

Guía del Docente

Curso de Administración de SQL Server (25 Horas)

Manual de referencia técnico y metodológico para el instructor. Incluye desglose pormenorizado de las 78 diapositivas, objetivos pedagógicos, explicaciones técnicas a bajo nivel de la arquitectura del motor, preguntas de dinamización para el aula, resolución paso a paso de los laboratorios y el checklist operativo de buenas prácticas del Administrador de Bases de Datos.

25 h

DURACIÓN LECTIVA

5

MÓDULOS (0 AL 4)

78

DIAPOSITIVAS CON GUION

14

PRÁCTICAS Y LABORATORIOS

77

SCRIPTS Y BLOQUES T-SQL

1

CASO PRÁCTICO
INTEGRADOR

Motor de Referencia:

Microsoft SQL Server 2022 (compatible con 2019 / 2016)

Entorno de Prácticas:

Contenedores Docker (mcr.microsoft.com/mssql/server:2022-latest) y SSMS

Material Complementario:

curso_sql_server_dba_25h.pptx (78 diapositivas panorámicas 16:9)

Repositorio del Proyecto:

training-sqlserver · Documento Maestro de Referencia

Estructura General del Curso

El curso está diseñado para cubrir de forma progresiva desde la arquitectura física del motor relacional hasta la alta disponibilidad y las operaciones críticas ante desastres. Cada módulo alterna bloques teóricos de 20–30 minutos con demostraciones y laboratorios guiados en instancias dedicadas.

Módulo	Título	Horas	Diapositivas	Enfoque Pedagógico
0	Bienvenida y Entorno	0,5 h	1 – 4	Objetivos, dinámica de clase y despliegue del entorno Docker/SSMS
1	Fundamentos y Arquitectura. Licenciamiento	5,5 h	5 – 20	Motor relacional, Buffer Pool, páginas/extensiones, WAL/VLF y Lab 1
2	Gestión y Seguridad	7,0 h	21 – 42	DDL, restricciones, procedimientos, DML analítico, permisos, roles y Lab 2
3	Optimización y Alta Disponibilidad	6,0 h	43 – 60	Índices B-Tree, estadísticas, planes de ejecución, DMVs, Always On y Lab 3
4	Mantenimiento, Copias de Seguridad y Caso Final	6,0 h	61 – 78	Backups, Point-in-Time, CHECKDB, Agent, incidencias, Lab 4 y Caso Integrador

MAPA DE CONTENIDOS POR MÓDULO

MÓDULO 0 · 0,5 H · DIAPS. 1–4

Bienvenida y Entorno de Trabajo

Presentación del curso, dinámica de aprendizaje práctico, reglas de seguridad y despliegue del contenedor SQL Server 2022 en Docker.

MÓDULO 1 · 5,5 H · DIAPS. 5–20

Arquitectura y Licenciamiento

Relational Engine vs Storage Engine, Buffer Pool, páginas de 8 KB, MDF/NDF/LDF, WAL y VLFs, bases del sistema, ediciones y licencias Core vs Server+CAL.

Lab 1: Ficheros y Filegroups.

MÓDULO 2 · 7,0 H · DIAPS. 21–42

Gestión y Seguridad

DDL robusto, vistas indexadas, Stored Procedures y plan cache sniffing, JOINS lógicos vs físicos, Window Functions, Logins, Users, Schemas, Roles y Ownership Chaining.

Lab 2A: Consultas y SP · **Lab 2B:** Permisos segregados.

MÓDULO 3 · 6,0 H · DIAPS. 43–60

Optimización y Alta Disponibilidad

Heaps vs Clustered, B-Tree, Non-Clustered y covering indexes, estadísticas y estimador de cardinalidad, planes de ejecución, DMVs de rendimiento, RTO/RPO y Always On AG.

Lab 3.1: Índices · **Lab 3.2:** Planes · **Lab 3.3:** DMVs.

MÓDULO 4 · 6,0 H · DIAPS. 61–78

Mantenimiento, Copias de Seguridad y Caso Final

Modelos Simple vs Full, cadena Full + Diff + Log con CHECKSUM, restauración Point-in-Time (STOPAT), DBCC CHECKDB, mantenimiento automatizado con Agent, alertas de gravedad 19–25, resolución de incidencias críticas (log lleno, TempDB saturada, deadlocks) y Checklist de Buenas Prácticas del DBA.

Lab 4: Preparación, desastre y restauración · **Caso Práctico Integrador Final** (Rúbrica completa).

0,5 H · DIAPOSITIVAS 1 – 4

MÓDULO 0: BIENVENIDA Y ENTORNO

Objetivos, dinámica de clase y despliegue del entorno Docker/SSMS

DIAPOSITIVA 01

CURSO DE FORMACIÓN · 25 H

Administración de SQL Server

0 · Bienvenida y Entorno

CONTENIDO CLAVE EN PANTALLA (PROYECCIÓN)

- De la arquitectura del motor a la alta disponibilidad: operar bases de datos en producción

OBJETIVO PEDAGÓGICO

Dar la bienvenida, situar el curso en el rol real de un DBA y fijar expectativas sobre lo que el alumno sabrá hacer al terminar.

GUION DEL DOCENTE Y EXPLICACIÓN TÉCNICA

- Presentación del instructor y de los alumnos:** ronda rápida (rol, experiencia previa con T-SQL, qué versión de SQL Server usan hoy en su empresa).
- Mensaje clave:** un DBA no "administra tablas", garantiza que los datos estén disponibles, íntegros, seguros y rápidos. Los cuatro adjetivos estructuran el curso.
- El curso son 25 horas repartidas en 4 módulos:** Fundamentos y arquitectura (5,5 h), Gestión y seguridad (7 h), Optimización y alta disponibilidad (6 h) y Mantenimiento y buenas prácticas (6 h), más 0,5 h de bienvenida y entorno.
- Cada módulo termina con un laboratorio guiado y el curso cierra con un caso práctico integrador que obliga a combinar todo:** diseño, seguridad, índices y backups.
- Versión de referencia:** SQL Server 2022 (compatible con 2019). Las diferencias relevantes con 2016/2017 y 2025 se señalan cuando importan.

PUNTOS DE INTERACCIÓN Y PREGUNTAS PARA LA CLASE

- ¿Quién ha sufrido alguna vez una caída de producción por falta de espacio, de backups o de índices? (Sirve de gancho para todo el curso.)
- ¿Qué esperáis poder hacer el último día que hoy no sabéis hacer?

DIAPOSITIVA 02

BIENVENIDA

Qué vamos a aprender: 25 horas, 4 módulos

0 · Bienvenida y Entorno

 CONTENIDO CLAVE EN PANTALLA (PROYECCIÓN)

- 0,5 h
- M0 · Bienvenida y entorno
- 5,5 h
- M1 · Fundamentos y arquitectura
- 7 h
- M2 · Gestión y seguridad
- 6 h
- M3 · Optimización y alta disponibilidad
- 6 h
- M4 · Mantenimiento y buenas prácticas
- Al terminar sabrás...
- Explicar cómo ejecuta el motor una consulta
- Crear objetos y un modelo de permisos seguro
- Leer un plan de ejecución y diagnosticar con DMVs
- ...y serás capaz de operar
- Diseñar backups según RPO/RTO y restaurar a un punto exacto
- Automatizar mantenimiento y resolver incidencias comunes
- Elegir la solución de HA/DR adecuada

 OBJETIVO PEDAGÓGICO

Que el alumno visualice el mapa completo del curso, el reparto horario y las competencias que adquirirá.

 GUION DEL DOCENTE Y EXPLICACIÓN TÉCNICA

- **Módulo 1 (5,5 h):** cómo es SQL Server por dentro. Sin entender Buffer Pool, páginas, ficheros y log no se puede diagnosticar nada después. Incluye ediciones y licenciamiento, que es donde más dinero se gasta o se pierde.
- **Módulo 2 (7 h):** crear objetos (tablas, restricciones, vistas, procedimientos), consultar con soltura (JOINS, window functions) y asegurar el acceso con el principio de mínimo privilegio.
- **Módulo 3 (6 h):** rendimiento (índices, estadísticas, planes de ejecución, DMVs) y alta disponibilidad / recuperación ante desastres (RTO, RPO, Log Shipping, FCI, Always On AG).
- **Módulo 4 (6 h):** lo que separa a un aficionado de un DBA: modelos de recuperación, backups, restauración point-in-time, DBCC CHECKDB, automatización con Agent y resolución de incidencias.
- **Los números no son rígidos:** si un laboratorio genera debate, se recorta teoría. El tiempo de laboratorio es el más valioso.
- Al final del curso el alumno podrá diseñar una estrategia de backup que cumpla un RPO dado, interpretar un plan de ejecución, y montar un esquema de permisos segregado.

 PUNTOS DE INTERACCIÓN Y PREGUNTAS PARA LA CLASE

- De los cuatro bloques, ¿cuál creéis que os va a costar más y por qué?
- ¿Qué tema que no aparece aquí os gustaría ver? (Anotarlo para el cierre del curso.)

DIAPOSITIVA 03

BIENVENIDA

Cómo trabajaremos

0 · Bienvenida y Entorno

 CONTENIDO CLAVE EN PANTALLA (PROYECCIÓN)

- Teoría breve
- Bloques de 20-30 min con esquemas, demostraciones en vivo y fragmentos T-SQL reales.
- Laboratorios guiados
- Cuatro laboratorios y un caso integrador. Cada alumno trabaja en su propia instancia.
- Participación
- Preguntas y debates en cada tema. Los errores son material de aprendizaje.

Regla de seguridad: los scripts del curso son destructivos a propósito. Comprueba siempre `SELECT @@SERVERNAME` SQL antes de ejecutarlos y nunca los lances contra producción.

- Material: diapositivas con guion completo en las notas, scripts T-SQL del curso y la Guía del Docente en Markdown.

 OBJETIVO PEDAGÓGICO

Acordar la dinámica: teoría breve, laboratorio inmediato y participación constante; fijar reglas de seguridad del entorno de práctica.

 GUION DEL DOCENTE Y EXPLICACIÓN TÉCNICA

- **Formato:** bloques de teoría de 20-30 minutos, siempre seguidos de una demostración o ejercicio. Un curso de administración se aprende con las manos en el teclado.
- **Los laboratorios se hacen sobre un contenedor Docker o una instancia local propia:** cada alumno puede romper su servidor sin afectar a nadie. Romper cosas a propósito (llenar el log, corromper una página) es parte del plan.
- **Regla de oro:** nunca ejecutar los scripts del curso contra un servidor de producción. Se pide confirmar el nombre del servidor (`SELECT @@SERVERNAME`) antes de cualquier script destructivo; es un hábito profesional.
- **Material:** diapositivas, scripts T-SQL del curso y la Guía del Docente en Markdown con todo el guion. Cada diapositiva de laboratorio incluye una guía de resolución en las notas.
- **Evaluación:** participación, entrega del Laboratorio 4 y resolución del caso práctico integrador final (con criterios explícitos que se verán al comienzo del caso).
- **Se anima a interrumpir con preguntas:** es más barato resolver una duda ahora que durante un incidente real.

 PUNTOS DE INTERACCIÓN Y PREGUNTAS PARA LA CLASE

- ¿Qué entorno tiene cada uno disponible: Windows con SSMS, macOS/Linux con Docker, ambos?
- ¿Qué os frena más al aprender administración: no tener un servidor donde practicar o miedo a romperlo?

DIAPOSITIVA 04

ENTORNO

Entorno de trabajo del curso

0 · Bienvenida y Entorno

 CONTENIDO CLAVE EN PANTALLA (PROYECCIÓN)

- SSMS (Windows)
- Cliente completo: objetos, planes de ejecución, Agent, Profiler.
- VS Code + extensión MSSQL
- Multiplataforma; sustituye a Azure Data Studio (fin de soporte 28/02/2026).

Docker o instalación **local**
Imagen oficial **2022-latest**, o Developer Edition en Windows.

SQL

- BASH · DOCKER

```
docker run -d --name sql2022 \
  -e "ACCEPT_EULA=Y" \
  -e "MSSQL_SA_PASSWORD=Curso#SQL2022!" \
  -p 1433:1433 \
  -v sqldata:/var/opt/mssql \
  mcr.microsoft.com/mssql/server:2022-latest
```

SQL

```
docker ps
docker logs sql2022 | tail -n 5
```

Si el contenedor se para: revisa docker logs. Casi siempre es una contraseña de sa que **no** cumple la política de complejidad.

SQL

 OBJETIVO PEDAGÓGICO

Dejar el entorno preparado: instancia SQL Server accesible y herramienta cliente conectada antes de empezar el Módulo 1.

 GUION DEL DOCENTE Y EXPLICACIÓN TÉCNICA

- **Tres piezas:** una instancia SQL Server (Docker o local), una herramienta cliente (SSMS en Windows; extensión MSSQL de VS Code en macOS/Linux) y un editor de scripts.
- **Azure Data Studio ha llegado al final de su soporte (28 de febrero de 2026):** Microsoft recomienda la extensión MSSQL de Visual Studio Code. Si algún alumno lo tiene instalado, funciona pero ya no recibe actualizaciones.
- **Docker:** la imagen oficial mcr.microsoft.com/mssql/server:2022-latest es multiplataforma. En Mac con Apple Silicon hay que ejecutar Docker Desktop con emulación x86_64 (--platform linux/amd64) y activar Rosetta en los ajustes; el arranque es más lento pero funciona.
- **Variables imprescindibles:** ACCEPT_EULA=Y y MSSQL_SA_PASSWORD con una contraseña que cumpla la política de complejidad (8+ caracteres, tres de cuatro categorías); si no, el contenedor arranca y se detiene sin avisar: se diagnostica con docker logs sql2022.
- **El volumen -v sqldata:/var/opt/mssql** persiste los archivos .mdf/.ldf si se elimina el contenedor. Se usará en el laboratorio de restauración del Módulo 4.
- **Opción local (Windows):** SQL Server 2022 Developer Edition (gratuita, con todas las funciones de Enterprise y sólo para desarrollo y pruebas) + SSMS 20/21.

 PUNTOS DE INTERACCIÓN Y PREGUNTAS PARA LA CLASE

- ¿Qué diferencia hay entre conectarse con la cuenta sa y con autenticación de Windows? (Anticipa el Módulo 2.)
- ¿Por qué crees que el contenedor se detiene si la contraseña de sa no cumple la política?

 INSTRUCCIONES DE LABORATORIO Y PAUTAS DE RESOLUCIÓN

- Arrancar el contenedor con el comando de la diapositiva y comprobar con docker ps que el estado es Up.
- **Conectar desde SSMS o VS Code:** servidor localhost,1433, usuario sa, marcar "Trust server certificate".
- Ejecutar SELECT @@VERSION, @@SERVERNAME; y guardar el resultado como evidencia.
- **Trampas:** puerto 1433 ocupado por otra instancia local (usar -p 14333:1433 y conectar a localhost,14333); contraseña con caracteres especiales mal escapados en el shell (usar comillas simples en zsh/bash).

5,5 H · DIAPOSITIVAS 5 – 20

MÓDULO 1: FUNDAMENTOS Y ARQUITECTURA. LICENCIAMIENTO

Motor relacional, Buffer Pool, páginas/extensiones, WAL/VLF y Lab 1

DIAPOSITIVA 05

MÓDULO 1 · ≈ 5,5 H

01 · Apertura del Módulo: Fundamentos y Arquitectura

1 · Fundamentos y Arquitectura. Licenciamiento

CONTENIDO CLAVE EN PANTALLA (PROYECCIÓN)

- Fundamentos y arquitectura
- Incluye licenciamiento: cómo es SQL Server por dentro y qué edición y licencia necesitas
- Historia y versiones
- Arquitectura del motor
- Páginas, extensiones y ficheros
- Instancias y bases del sistema
- Ediciones y licenciamiento
- Laboratorio 1

OBJETIVO PEDAGÓGICO

Introducir el Módulo 1 y justificar por qué empezamos por la arquitectura interna antes que por la sintaxis.

GUION DEL DOCENTE Y EXPLICACIÓN TÉCNICA

- **Idea fuerza:** casi todos los problemas que veremos más adelante (bloqueos, rendimiento, log lleno, restauraciones) se explican con cuatro conceptos: Buffer Pool, páginas, Transaction Log y tempdb. Este módulo los instala en la cabeza del alumno.
- **Recorrido:** historia y versiones → arquitectura → estructura física (páginas, extensiones, MDF/NDF/LDF, WAL y VLFs) → instancias y bases del sistema → ediciones y licenciamiento → herramientas → Laboratorio 1.
- **Duración estimada 5,5 horas:** ~3,5 de teoría con demostraciones y ~2 de laboratorio.

PUNTOS DE INTERACCIÓN Y PREGUNTAS PARA LA CLASE

- ¿Qué creéis que ocurre exactamente entre que escribís un INSERT y que el dato está "guardado"? Anotad las respuestas, las revisaremos al explicar el WAL.

DIAPOSITIVA 06

HISTORIA

Versiones de SQL Server

1 · Fundamentos y Arquitectura. Licenciamiento

CONTENIDO CLAVE EN PANTALLA (PROYECCIÓN)

- | | | |
|--|---|--|
| <ul style="list-style-type: none"> • 2012 | <ul style="list-style-type: none"> • Always On AG • Columnstore | <ul style="list-style-type: none"> • Secuencias • 2014 |
|--|---|--|
- In-Memory OLTP

Buffer Pool Ext.

Backup cifrado

SQL
- | | | |
|---|--|--|
| <ul style="list-style-type: none"> • 2016 • Query Store • Always Encrypted • Temporal tables • 2017 • Linux y Docker • Adaptive QP • Python | <ul style="list-style-type: none"> • 2019 • Intelligent QP • ADR • UTF-8 • 2022 • Ledger • PSP optimization • Contained AG | <ul style="list-style-type: none"> • 2025 • Tipo vector (IA) • Tipo json • Regex en T-SQL • Ciclo de vida: 5 años de soporte principal + 5 de soporte extendido. SQL Server 2016 ya no recibe parches de seguridad (jul-2026); 2022 es la referencia del curso. |
|---|--|--|

OBJETIVO PEDAGÓGICO

Situar las versiones actuales, sus hitos y su ciclo de vida para decidir cuál instalar o a cuál migrar.

GUIÓN DEL DOCENTE Y EXPLICACIÓN TÉCNICA

- SQL Server nace en 1989 (Sybase/Microsoft/Ashton-Tate). Los hitos modernos empiezan con 2005 (DMVs, TRY/CATCH, CLR), 2008 (compresión, Policy-Based Management) y 2012 (Always On AG y columnstore).
- **2014:** In-Memory OLTP (Hekaton) y Buffer Pool Extension. 2016: Query Store, Always Encrypted, Row-Level Security, temporal tables. 2017: primera versión en Linux y Docker; adaptive query processing.
- **2019:** Intelligent Query Processing (memory grant feedback, batch mode en rowstore), Accelerated Database Recovery (ADR) y UTF-8. 2022: Ledger, Parameter Sensitive Plan optimization, Contained Availability Groups y copia a S3.
- **2025:** incorpora el tipo vector y funciones de IA, el tipo json nativo y expresiones regulares en T-SQL. Conviene verificar en la página de ciclo de vida de Microsoft las fechas de soporte vigentes.
- **Ciclo de vida:** 5 años de soporte principal + 5 de soporte extendido (parches de seguridad). SQL Server 2016 terminó su soporte extendido en julio de 2026; 2017 termina en octubre de 2027, 2019 en enero de 2030 y 2022 en enero de 2033.
- Nivel de compatibilidad (COMPATIBILITY_LEVEL): 130=2016, 140=2017, 150=2019, 160=2022. Una BD restaurada en una versión nueva mantiene su nivel de compatibilidad hasta que se cambie; es clave en migraciones.

PUNTOS DE INTERACCIÓN Y PREGUNTAS PARA LA CLASE

- ¿Qué versión tenéis en producción y en qué fecha termina su soporte extendido?
- ¿Qué riesgo tiene subir el nivel de compatibilidad de una base de datos sin probarla? (Cambios en el optimizador/CE, regresiones de planes.)

DIAPOSITIVA 07

ARQUITECTURA

Arquitectura interna del motor

1 · Fundamentos y Arquitectura. Licenciamiento

 CONTENIDO CLAVE EN PANTALLA (PROYECCIÓN)

- | | | |
|---|--|--|
| <ul style="list-style-type: none"> • Aplicación cliente · SSMS · sqlcmd · .NET • TDS • SNI · SQL Network Interface • Shared Memory • TCP/IP • Named Pipes • RELATIONAL ENGINE (Query Processor) • Parser • Algebrizer • Optimizer • Query Executor | <ul style="list-style-type: none"> • STORAGE ENGINE • Access Methods • Buffer Manager • (Buffer Pool) • Transaction Manager • SQLOS · servicios del sistema operativo • Schedulers • Memory Manager • I/O • Locks • Disco • .mdf | <ul style="list-style-type: none"> • .ndf • (datos) • .ldf • (log WAL) • Relational Engine • Decide el cómo (plan) • Parser → Algebrizer → Optimizer • Cost-based con estadísticas • Storage Engine • Ejecuta el acceso a los datos • Buffer Pool y Access Methods • Log, bloqueos y transacciones |
|---|--|--|

 OBJETIVO PEDAGÓGICO

Que el alumno identifique las capas del motor (SNI, Relational Engine, Storage Engine, SQLOS) y sepa qué responsabilidad tiene cada una.

 GUION DEL DOCENTE Y EXPLICACIÓN TÉCNICA

- **Seguir el recorrido de arriba abajo con el puntero. El cliente se comunica mediante el protocolo TDS (Tabular Data Stream) sobre un protocolo de red:** Shared Memory (local), TCP/IP (el habitual, puerto 1433 por defecto) o Named Pipes. La capa SNI (SQL Network Interface) los abstrae.
- **Relational Engine (query processor):** el Parser comprueba la sintaxis; el Algebrizer resuelve nombres de objetos/tipos y produce un árbol de consulta; el Optimizer, basado en costes, elige un plan a partir de estadísticas; el Query Executor lo ejecuta operador a operador.
- **Storage Engine:** Access Methods (cómo leer/escribir filas, índices, heaps), Buffer Manager (gestiona el Buffer Pool: caché de páginas de datos en memoria) y Transaction Manager (log, bloqueos, aislamiento: ACID).
- **SQLOS es una capa propia de SQL Server (no es el sistema operativo):** planificación cooperativa con schedulers (uno por CPU lógica), gestión de memoria, I/O, sincronización y bloqueos. Explica los wait types (PAGEIOLATCH, CXPACKET, LCK_M_*).
- **A la derecha, el disco:** ficheros de datos (.mdf/.ndf) y de log (.ldf). El motor nunca escribe directamente en el disco de datos al hacer COMMIT: primero escribe en el log (WAL). Lo veremos en la diapositiva del Transaction Log.
- **Punto de vista del DBA:** cada capa suele ser origen de un tipo de problema: red (latencias, fallos de conexión), optimizador (planes malos), almacenamiento (E/S lenta, log), SQLOS (esperas de CPU/memoria).

 PUNTOS DE INTERACCIÓN Y PREGUNTAS PARA LA CLASE

- ¿En qué capa se decide si una consulta usa un Index Seek o un Index Scan? ¿Y quién lo ejecuta?
- Si el servidor sufre esperas PAGEIOLATCH_SH, ¿qué capas están implicadas?

DIAPOSITIVA 08

ARQUITECTURA

Ciclo de vida de una consulta

1 · Fundamentos y Arquitectura. Licenciamiento

 CONTENIDO CLAVE EN PANTALLA (PROYECCIÓN)

- 1
- Parse + Algebrize
- Sintaxis, resolución de objetos y permisos.
- 2
- Plan cache
- ¿Existe un plan reutilizable? Si sí, se salta la optimización.
- 3
- Optimize
- Elige el plan de menor coste estimado con estadísticas.
- 4
- Execute
- Operadores en iteradores piden filas y páginas.
- 5
- Buffer Pool + TDS
- Lectura lógica o física; resultado al cliente.
- Pruébalo: SET STATISTICS IO, TIME ON; muestra lecturas lógicas, físicas y tiempo de compilación frente a ejecución.

 OBJETIVO PEDAGÓGICO

Comprender el recorrido completo de una sentencia, qué se cachea (plan cache) y por qué la segunda ejecución es más barata.

 GUION DEL DOCENTE Y EXPLICACIÓN TÉCNICA

- **Paso 1 – Parse:** validación sintáctica; si hay error aquí nunca llega al optimizador. Paso 2 – Bind/Algebrize: resuelve tablas, columnas y tipos; comprueba permisos sobre los objetos.
- **Paso 3 – Optimize:** antes de optimizar, el motor busca en el plan cache un plan reutilizable con hash de la consulta. Si no existe, el optimizador explora alternativas (join order, tipos de join, acceso por índice) y elige el de menor coste estimado; no busca el óptimo absoluto, sino uno "suficientemente bueno" en tiempo limitado (Good Enough Plan Found).
- **Paso 4 – Execute:** el Query Executor es un modelo de iteradores (cada operador pide filas al siguiente). Pide a Access Methods las filas y estos al Buffer Manager las páginas.
- **Paso 5 – Lectura de páginas:** si la página está en el Buffer Pool es una lectura lógica; si no, se lee del disco (lectura física) y se sube al Buffer Pool. SET STATISTICS IO muestra ambas.
- **Resultado:** se envían filas al cliente por TDS en paquetes (4096 bytes por defecto). Un cliente lento consumiendo resultados provoca la espera ASYNC_NETWORK_IO, que no es un problema del servidor.
- El plan cache se vacía al reiniciar, con DBCC FREEPROCCACHE o por presión de memoria. Consultas ad hoc con literales distintos generan un plan por cada variante (plan cache bloat); la parametrización lo evita.

 PUNTOS DE INTERACCIÓN Y PREGUNTAS PARA LA CLASE

- ¿Por qué la primera ejecución de una consulta suele ser más lenta que las siguientes? (Compilación + lecturas físicas.)
- ¿Qué ventaja tiene que el plan quede cacheado y qué riesgo introduce? (Parameter sniffing, se verá en el Módulo 2.)

DIAPOSITIVA 09

ARQUITECTURA

1 · Fundamentos y Arquitectura. Licenciamiento

Buffer Pool y gestión de memoria

 CONTENIDO CLAVE EN PANTALLA (PROYECCIÓN)

- Buffer Pool
- Caché de páginas de datos de 8 KB. Lectura lógica (memoria) frente a física (disco).
- Plan Cache
- Planes compilados reutilizables. Se pierde al reiniciar o por presión de memoria.
- Checkpoint y Lazy Writer
- Checkpoint escribe páginas sucias periódicamente; Lazy Writer libera memoria bajo presión.
- T-SQL · LIMITAR LA MEMORIA
- EXEC sp_configure 'show advanced options', 1; RECONFIGURE;
- EXEC sp_configure 'max server memory (MB)', 12288; RECONFIGURE;
- Siempre limita max server memory en producción y deja memoria libre para el sistema operativo.

 OBJETIVO PEDAGÓGICO

Entender que SQL Server usa casi toda la memoria disponible por diseño, qué se guarda en ella y cómo limitarla correctamente.

 GUION DEL DOCENTE Y EXPLICACIÓN TÉCNICA

- **El Buffer Pool es la mayor parte de la memoria de la instancia:** caché de páginas de datos (8 KB). Leer de memoria cuesta microsegundos, de disco milisegundos. Por eso SQL Server "se come" la RAM: es su comportamiento normal, no una fuga.
- **El Plan Cache guarda planes compilados. Otras cachés:** Memory Grants (workspace para sorts y hashes), caché de metadatos y de bloqueos.
- **Página sucia (dirty page):** modificada en memoria y aún no escrita en el .mdf. Dos procesos las escriben: Checkpoint (periódico, controla el tiempo de recuperación; objetivo por defecto 60 s en BD nuevas) y Lazy Writer (libera páginas cuando hay presión de memoria).
- **max server memory:** por defecto 2.147.483.647 MB (sin límite). En producción hay que fijarlo dejando memoria al SO y otros procesos (regla de partida: 10-20% o al menos 4 GB, más si hay otras instancias o servicios). Sin límite, el SO puede paginar y el rendimiento se degrada.
- **Indicadores:** Page Life Expectancy (PLE) y Buffer Cache Hit Ratio; el segundo es engañoso. Lo más fiable: esperas PAGEIOLATCH y tasa de lecturas físicas.
- En virtualización, recomendar reservar la memoria (memory reservation) para evitar ballooning, que quita memoria al Buffer Pool sin que SQL Server lo sepa.

 PUNTOS DE INTERACCIÓN Y PREGUNTAS PARA LA CLASE

- El administrador del sistema dice "SQL Server usa el 95% de la RAM, hay una fuga". ¿Qué respondéis?
- ¿Qué diferencia hay entre Checkpoint y Lazy Writer?

 INSTRUCCIONES DE LABORATORIO Y PAUTAS DE RESOLUCIÓN

- **Opcional (demo):** consultar sys.dm_os_sys_memory y sys.dm_os_process_memory; cambiar max server memory y observar cómo cambia el valor de Target Server Memory en sys.dm_os_performance_counters.

DIAPOSITIVA 10

ALMACENAMIENTO

Estructura física: páginas y extensiones

1 · Fundamentos y Arquitectura. Licenciamiento

CONTENIDO CLAVE EN PANTALLA (PROYECCIÓN)

- Página = 8 KB (unidad mínima de E/S)
- Page Header · 96 bytes
- Filas de datos
- (crecen hacia abajo ↓)
- Espacio libre
- Row Offset Array
- (2 bytes/fila, crece hacia arriba ↑)
- Máx. 8060 bytes de datos por fila en página (límite en fila)
- Extensión = 8 páginas contiguas = 64 KB
- P1
- 8 KB
- P2
- 8 KB
- P3
- 8 KB
- P4
- 8 KB
- P5
- 8 KB
- P6
- 8 KB
- P7
- 8 KB
- P8
- 8 KB
- Mixta: páginas de distintos objetos · Uniforme: un solo objeto (desde la 9.ª página)
- Ficheros de datos agrupados en filegroups
- Filegroup PRIMARY (por defecto)
- Primario .mdf
- (catálogo + datos)
- Secundario .ndf
- (datos adicionales)
- Transaction log (.ldf) — secuencia circular de VLFs
- VLF 1
- VLF 2
- VLF 3
- VLF 4
- VLF 5
- VLF 6
- VLF 7
- VLF 8
- Gris: reutilizables · Naranja: activos (log activo) · Blanco: libres

OBJETIVO PEDAGÓGICO

Conocer la unidad básica de almacenamiento (página de 8 KB) y su agrupación en extensiones, y relacionarlas con ficheros y log.

GUION DEL DOCENTE Y EXPLICACIÓN TÉCNICA

- Página = 8 KB (8192 bytes), unidad mínima de E/S. Estructura: cabecera de 96 bytes (tipo de página, ids, LSN, contadores), área de filas, y un array de desplazamientos de fila al final (2 bytes por fila) que crece hacia arriba.
- Máximo 8060 bytes de datos de fila en página (no LOB). Si las columnas variables exceden, se mueven a páginas ROW_OVERFLOW; los LOB (varchar(max), xml, etc.) van a páginas LOB_DATA.
- Extensión = 8 páginas contiguas = 64 KB, unidad de asignación. Mixtas (comparten objetos distintos; las primeras 8 páginas de un objeto en versiones antiguas) y uniformes (un solo objeto). Desde SQL Server 2016, la opción por defecto en BD de usuario es asignar extensiones uniformes (MIXED_PAGE_ALLOCATION OFF).
- **Tipos de página de asignación:** PFS (espacio libre), GAM/SGAM (qué extensiones están libres/mixtas), IAM (qué extensiones pertenecen a un objeto). Importante en tempdb: contención en PFS/GAM/SGAM.
- **Demostración:** DBCC IND / sys.dm_db_database_page_allocations y DBCC PAGE (con TRACE FLAG 3604) para ver una página real; muy ilustrativo en el laboratorio avanzado.
- **Consecuencia práctica:** leer una fila implica leer una página entera. Filas anchas = menos filas por página = más E/S. El diseño de tablas importa.

PUNTOS DE INTERACCIÓN Y PREGUNTAS PARA LA CLASE

- ¿Cuántas filas de 200 bytes caben aproximadamente en una página? (≈ 40.)
- ¿Por qué un SELECT de una sola fila puede acabar leyendo 8 KB o más?

DIAPOSITIVA 11

ALMACENAMIENTO

Ficheros de datos y de log: MDF, NDF, LDF

1 · Fundamentos y Arquitectura. Licenciamiento

 CONTENIDO CLAVE EN PANTALLA (PROYECCIÓN)

- Primario · .mdf
- Uno por BD. Contiene el catálogo y, si no hay más, los datos. Filegroup PRIMARY.
- Secundarios · .ndf
- Opcionales. Reparten datos en varios discos y permiten restaurar por filegroup.
- Log · .ldf
- Registro secuencial de cambios (WAL). Con uno es suficiente y no pertenece a ningún filegroup.
- Autogrowth en MB, no en %: un 10% de 500 GB son 50 GB de golpe. Dimensiona el tamaño inicial para evitar crecer.
- IFI (Instant File Initialization): concede "Perform volume maintenance tasks" a la cuenta del servicio para crear y restaurar ficheros de datos mucho más rápido.

 OBJETIVO PEDAGÓGICO

Distinguir los tres tipos de fichero, el concepto de filegroup y las prácticas correctas de tamaño y crecimiento.

 GUION DEL DOCENTE Y EXPLICACIÓN TÉCNICA

- Toda base de datos tiene al menos un fichero de datos primario (.mdf, con el catálogo de la BD) y un fichero de log (.ldf). Los secundarios (.ndf) son opcionales y se agrupan en filegroups.
- **Filegroup:** unidad lógica de administración. PRIMARY es el predeterminado. Sirven para repartir E/S entre discos, restaurar parcialmente (piecemeal restore), colocar tablas calientes en almacenamiento rápido o particionar tablas.
- **Los ficheros de un mismo filegroup se llenan por el algoritmo proportional fill:** más espacio libre = más escrituras. Por eso conviene que tengan el mismo tamaño y el mismo crecimiento.
- **Autogrowth:** configúralo siempre en MB fijos (p. ej. 256-512 MB para datos grandes) y no en porcentaje, porque crecer un 10% de un fichero de 500 GB son 50 GB bloqueando operaciones. El tamaño inicial debe estimarse para evitar crecimientos frecuentes.
- **Instant File Initialization (IFI):** el privilegio "Perform volume maintenance tasks" evita poner a cero los ficheros de datos al crecer o restaurar (el log siempre se pone a cero, salvo mejoras recientes de 2022 para crecimientos pequeños). Reduce drásticamente el tiempo de restauración.
- **Un solo fichero de log por BD es suficiente:** se escribe de forma secuencial; añadir más ficheros no mejora el rendimiento, sólo complica la gestión.

 PUNTOS DE INTERACCIÓN Y PREGUNTAS PARA LA CLASE

- ¿Por qué un autogrowth del 10% es una mala práctica en un fichero grande?
- ¿Qué ventaja tiene separar tablas en distintos filegroups si todos los discos son el mismo array?

DIAPOSITIVA 12

ALMACENAMIENTO

Transaction Log: WAL y VLFs

1 · Fundamentos y Arquitectura. Licenciamiento

 CONTENIDO CLAVE EN PANTALLA (PROYECCIÓN)

- 1
- Se modifica la página
- Cambio en el Buffer Pool; la página queda sucia.
- 2
- Se escribe el log
- Registro con LSN en el log buffer.
- 3
- COMMIT
- Flush del log a disco. El cliente recibe OK.
- 4
- Checkpoint
- Más tarde, las páginas sucias se escriben al .mdf.
- T-SQL · VLFs DE LA BD ACTUAL

SQL

```
SELECT COUNT(*) AS vlf_total,
       SUM(CASE WHEN vlf_active = 1 THEN 1 ELSE 0 END) AS vlf_activos
FROM sys.dm_db_log_info(DB_ID());
```

- Miles de VLFs por autogrowths pequeños ralentizan arranque y recuperación. Dimensiona el log de una vez.

 OBJETIVO PEDAGÓGICO

Entender el protocolo Write-Ahead Logging, por qué COMMIT sólo espera al log, y cómo el log se divide en VLFs que se reutilizan.

 GUION DEL DOCENTE Y EXPLICACIÓN TÉCNICA

- **WAL (Write-Ahead Logging):** antes de modificar una página de datos en disco, el registro de log que describe el cambio debe estar en disco. Al hacer COMMIT, SQL Server sólo espera a que el log esté persistido (flush); las páginas de datos modificadas se escriben después por Checkpoint/Lazy Writer.
- Por eso, ante un fallo, la recuperación (crash recovery) rehace (REDO) transacciones confirmadas cuyas páginas no llegaron al disco y deshace (UNDO) las no confirmadas. Es la base de la durabilidad (D de ACID).
- El log es una secuencia lógica de registros identificados por un LSN. Internamente se divide en Virtual Log Files (VLFs). El motor reutiliza los VLFs ya no necesarios; un VLF no se puede reutilizar mientras contenga log activo (transacciones abiertas, replicación pendiente, backup de log pendiente, AG...).
- **Qué libera log:** en modelo SIMPLE, un checkpoint; en FULL/BULK_LOGGED, un backup de log. Si el log crece sin parar en FULL y no hay backups de log, la causa es esa (se verá con log_reuse_wait_desc en Módulo 4).
- Demasiados VLFs (miles) por autogrowths pequeños ralentizan el arranque, la recuperación y los backups de log. Se mide con sys.dm_db_log_info (SQL Server 2016 SP2 en adelante).
- **ADR (Accelerated Database Recovery, 2019):** mantiene un version store persistente para que la recuperación y el rollback sean casi instantáneos, independientemente del tamaño de la transacción.

 PUNTOS DE INTERACCIÓN Y PREGUNTAS PARA LA CLASE

- Si el servidor se apaga justo después de un COMMIT, ¿cómo es posible que el dato no se pierda aunque la página de datos no se haya escrito?
- ¿Por qué un SELECT grande no genera log pero un UPDATE grande sí?

DIAPOSITIVA 13

INSTANCIAS

Instancias: predeterminada y con nombre

1 · Fundamentos y Arquitectura. Licenciamiento

 CONTENIDO CLAVE EN PANTALLA (PROYECCIÓN)

- Predeterminada
- Nombre de servicio MSSQLSERVER. Se conecta por nombre de host, puerto TCP 1433.
- Con nombre
- SERVIDOR\INSTANCIA. Puerto dinámico resuelto por SQL Server Browser (UDP 1434).
- T-SQL · IDENTIFICAR LA INSTANCIA

```
SELECT @@SERVERNAME AS servidor,  
       SERVERPROPERTY('InstanceName') AS instancia,  
       SERVERPROPERTY('Edition') AS edicion,  
       SERVERPROPERTY('ProductVersion') AS version;
```

SQL

- En producción: fija puertos estáticos, usa cuentas de servicio dedicadas (gMSA) y configura max server memory en cada instancia.

 OBJETIVO PEDAGÓGICO

Entender qué es una instancia, cómo se identifican y conectan, y por qué se usan varias en un mismo servidor.

 GUION DEL DOCENTE Y EXPLICACIÓN TÉCNICA

- Una instancia es una copia independiente del motor (procesos, memoria, bases del sistema, cuentas, configuración). Un servidor puede alojar una instancia predeterminada (MSSQLSERVER) y hasta 50 con nombre.
- **Conexión:** la predeterminada se accede por nombre de servidor (puerto TCP 1433 por defecto); las nombradas como SERVIDOR\INSTANCIA, normalmente con puerto dinámico que resuelve el servicio SQL Server Browser (UDP 1434). En entornos con cortafuegos se recomienda fijar puerto estático.
- **Casos de uso:** aislar versiones o parches, separar entornos (desarrollo/preproducción), aislar cargas, separar administración y seguridad. Contrapartida: más memoria y CPU repartidas manualmente; hay que fijar max server memory por instancia.
- **SERVERPROPERTY devuelve metadatos:** InstanceName, Edition, ProductVersion, ProductLevel, IsClustered, etc. Es la forma estándar de inventariar instancias con T-SQL.
- **Cada instancia tiene su cuenta de servicio. Recomendación:** cuentas de dominio dedicadas o gMSA (Group Managed Service Accounts), nunca LocalSystem.
- **En contenedores Docker, cada contenedor es de hecho una instancia aislada:** es la forma más cómoda de tener varias versiones para pruebas.

 PUNTOS DE INTERACCIÓN Y PREGUNTAS PARA LA CLASE

- ¿En qué caso elegirías dos instancias en el mismo servidor frente a dos servidores?
- ¿Qué ocurre si el servicio SQL Server Browser está parado y te conectas a una instancia con nombre sin indicar puerto?

DIAPOSITIVA 14

INSTANCIAS

Bases de datos del sistema

1 · Fundamentos y Arquitectura. Licenciamiento

 CONTENIDO CLAVE EN PANTALLA (PROYECCIÓN)

- | | |
|---|--|
| <ul style="list-style-type: none"> • master • Catálogo de la instancia: logins, configuración, ubicación de las BD. Sin ella no arranca. • model • Plantilla de las nuevas BD y de tempdb en cada arranque. • msdb | <ul style="list-style-type: none"> • Agent: jobs y alertas; historial de backups; Database Mail. • tempdb • Objetos temporales, spills y version store. Se recrea al arrancar. Sin backup. • mssqlsystemresource • Sólo lectura y oculta. Contiene los objetos del sistema (sys.*). |
|---|--|

Backup obligatorio

SQL

- master, model y msdb. tempdb y Resource no se respaldan.

 OBJETIVO PEDAGÓGICO

Conocer la función de cada base de datos del sistema, su criticidad y qué incluir en la política de backups.

 GUION DEL DOCENTE Y EXPLICACIÓN TÉCNICA

- master: catálogo de la instancia (logins, configuración, endpoints, ubicación de ficheros de todas las BD). Si se pierde, la instancia no arranca. Backup obligatorio tras cualquier cambio de logins o configuración.
- model: plantilla para CREATE DATABASE; sus propiedades (modelo de recuperación, tamaño, autogrowth) se heredan. También lo es para tempdb en cada reinicio.
- msdb: SQL Server Agent (jobs, alertas, operadores), historial de backups y restauraciones, Database Mail, planes de mantenimiento. Backup obligatorio.
- tempdb: espacio de trabajo temporal (tablas #temp, variables de tabla, spills de sort/hash, version store, cursores). Se recrea en cada arranque, no se hace backup, y es compartida por toda la instancia: un consumidor puede afectar a todos. Buenas prácticas: varios ficheros de datos iguales (hasta 8 inicialmente), en almacenamiento rápido.
- mssqlsystemresource (Resource database): sólo lectura, oculta, contiene los objetos del sistema (sys.*). Se actualiza al parchear. No se ve en SSMS, pero existe en el disco.
- **Otras BD del sistema según configuración:** distribution (replicación), BD de SSISDB, etc. Importante no confundir "BD del sistema" con "objetos del sistema": no se crean tablas de usuario en master.

 PUNTOS DE INTERACCIÓN Y PREGUNTAS PARA LA CLASE

- ¿Qué ocurre con los jobs de mantenimiento si se pierde msdb y no hay backup?
- ¿Por qué tempdb no necesita backup, pero sí necesita planificación de capacidad?

DIAPOSITIVA 15

LICENCIAMIENTO

Ediciones de SQL Server 2022

1 · Fundamentos y Arquitectura. Licenciamiento

 CONTENIDO CLAVE EN PANTALLA (PROYECCIÓN)

- Regla práctica: empieza por Standard y justifica Enterprise con una función concreta. Developer para pruebas, nunca para producción.

Tabla Resumen en Diapositiva:

[object Object]undefined

 OBJETIVO PEDAGÓGICO*Elegir la edición adecuada según límites de cómputo, memoria y funcionalidades, y entender las restricciones de cada una.* GUION DEL DOCENTE Y EXPLICACIÓN TÉCNICA

- **Express:** gratuita, para aplicaciones pequeñas y aprendizaje. Límites: 1 socket o 4 cores (el menor), 1.410 MB para el Buffer Pool, 10 GB por base de datos; sin SQL Server Agent.
- **Standard:** para cargas departamentales. Hasta 4 sockets o 24 cores, 128 GB de Buffer Pool. Incluye Basic Availability Groups (una BD por AG, una réplica secundaria), compresión de backup, TDE (desde 2019) y cifrado.
- **Enterprise:** sin límites de cómputo (el máximo que permita el SO) y todas las funciones: particionado online, compresión de datos, Always On AG avanzados, rebuild online de índices, in-memory ampliado, Resource Governor, etc.
- **Developer:** mismas funciones que Enterprise, gratis pero sin derecho a producción. Es la edición ideal para el laboratorio y desarrollo. La edición Evaluation dura 180 días.
- **Web existe sólo para proveedores de hosting. Las cifras de límites pueden variar entre versiones:** validar siempre con la documentación oficial "Editions and supported features of SQL Server 2022".
- **Criterios de elección:** necesidad de HA avanzada, volumen de memoria/CPU, funciones de seguridad y cumplimiento, y coste. Standard cubre mucho más de lo que se cree; Enterprise se justifica por funciones concretas, no por prestigio.

 PUNTOS DE INTERACCIÓN Y PREGUNTAS PARA LA CLASE

- ¿Qué funcionalidad concreta justificaría pagar Enterprise en vuestra organización?
- ¿Por qué una base de datos de 40 GB no puede alojarse en Express aunque la aplicación sea pequeña?

DIAPOSITIVA 16

LICENCIAMIENTO

Modelos de licencia: Core vs Server + CAL

1 · Fundamentos y Arquitectura. Licenciamiento

 CONTENIDO CLAVE EN PANTALLA (PROYECCIÓN)

- | | |
|--|---|
| <ul style="list-style-type: none"> • Per Core (por núcleo) • Standard y Enterprise • Usuarios y dispositivos ilimitados • Mínimo 4 núcleos por procesador y VM • Paquetes de 2 núcleos • Server + CAL • Sólo Standard • Licencia de servidor + CAL por usuario o dispositivo | <ul style="list-style-type: none"> • Conviene con pocos usuarios identificados • No válido para acceso masivo desde internet • 4 • núcleos mínimos por procesador • 2 • núcleos por paquete de licencia • SA • Software Assurance: derechos de failover |
|--|---|

 OBJETIVO PEDAGÓGICO

Comprender los dos modelos de licenciamiento, sus reglas mínimas y cuándo conviene cada uno.

 GUION DEL DOCENTE Y EXPLICACIÓN TÉCNICA

- **Licenciamiento por núcleo (Per Core):** se licencian todos los núcleos físicos de cada procesador del servidor, con un mínimo de 4 licencias de núcleo por procesador; se venden en paquetes de 2 núcleos. No requiere CAL: usuarios y dispositivos ilimitados. Es el único modelo para Enterprise.
- **Servidor + CAL (Server + Client Access License):** sólo para Standard. Una licencia de servidor por instancia/servidor más una CAL por usuario o por dispositivo que accede. Conviene con pocos usuarios identificables y estables; con acceso por web/internet o muchos usuarios, suele salir peor.
- En máquinas virtuales se licencian los núcleos virtuales asignados (mínimo 4 por VM); con Enterprise y Software Assurance se puede licenciar todo el host y ejecutar un número ilimitado de VMs.
- **Software Assurance (SA):** da derecho a nuevas versiones, y para HA, el derecho a tener una réplica secundaria pasiva de failover sin licencia adicional (con ciertas condiciones: no atender consultas ni backups).
- **Los precios y condiciones cambian:** no se citan cifras en el curso; recomendar validar con un partner de licenciamiento (LAR) antes de comprar y documentar el inventario de licencias.
- **Error típico de auditoría:** instancias de Developer usadas en producción, o más núcleos licenciados de los que realmente corresponden. Un DBA debe poder demostrar el inventario.

 PUNTOS DE INTERACCIÓN Y PREGUNTAS PARA LA CLASE

- Un servidor con 1 procesador de 2 núcleos físicos, ¿cuántas licencias de núcleo hay que comprar con Per Core? (4, por el mínimo.)
- ¿Qué modelo elegirías para una aplicación pública accesible por Internet? ¿Por qué?

DIAPOSITIVA 17

HERRAMIENTAS

Herramientas de administración

1 · Fundamentos y Arquitectura. Licenciamiento

 CONTENIDO CLAVE EN PANTALLA (PROYECCIÓN)

- SSMS
- Referencia en Windows: objetos, planes gráficos, Agent, asistentes de backup/restore.
- VS Code + extensión MSSQL
- Multiplataforma, notebooks y Git. Sustituye a Azure Data Studio.
- sqlcmd
- Línea de comandos para scripts, CI y automatización.
- Extended Events
- Trazas ligeras. Sustituyen a SQL Server Profiler (obsoleto).

 OBJETIVO PEDAGÓGICO

Conocer las herramientas principales de trabajo del DBA y cuándo usar cada una.

 GUION DEL DOCENTE Y EXPLICACIÓN TÉCNICA

- **SSMS (SQL Server Management Studio):** herramienta de referencia en Windows. Explorador de objetos, editor T-SQL, planes de ejecución gráficos, Agent, Activity Monitor, asistentes de backup/restore, XEvents Profiler. Se actualiza independientemente del motor (versión 20/21); administra desde 2008 hasta 2022 y Azure SQL.
- **Extensión MSSQL de VS Code (y Azure Data Studio, hasta su fin de soporte el 28/02/2026):** multiplataforma, notebooks, extensiones, integración con Git. Menos funciones de administración que SSMS, pero suficiente para consultas y scripts.
- **sqlcmd:** utilidad de línea de comandos para ejecutar T-SQL o scripts en lotes, automatizable en shell o CI. Variante moderna go-sqlcmd (multiplataforma, instalable con winget/brew).
- **SQL Server Profiler y Extended Events:** Profiler (traza) está obsoleto para el motor y consume más recursos; Extended Events (XEvents) es su sustitución ligera. SSMS incluye XEvent Profiler y el visor de eventos en vivo.
- **Otras:** SQL Server Configuration Manager (servicios, protocolos, puertos), PowerShell con dbatools (módulo comunitario imprescindible), Database Engine Tuning Advisor, Query Store GUI y Azure Arc / portal para entornos híbridos.
- **Buena práctica:** administrar con scripts (T-SQL/PowerShell) versionados en lugar de sólo con asistentes gráficos: son repetibles, auditables y reducen errores humanos.

 PUNTOS DE INTERACCIÓN Y PREGUNTAS PARA LA CLASE

- ¿Qué ventaja tiene lanzar una tarea con sqlcmd frente a hacerla con el asistente de SSMS?
- ¿Por qué se desaconseja dejar una traza de Profiler contra un servidor en producción?

DIAPOSITIVA 18

HERRAMIENTAS

sqlcmd y Extended Events básicos

1 · Fundamentos y Arquitectura. Licenciamiento

 CONTENIDO CLAVE EN PANTALLA (PROYECCIÓN)

• BASH · SQLCMD

```
sqlcmd -S localhost,1433 -U sa -C \
-P "Curso#SQL2022!" \
-Q "SELECT @@VERSION"
```

SQL

```
sqlcmd -S .\SQLEXPRESS -E -b \
-i script.sql -o salida.txt
```

• T-SQL · EXTENDED EVENTS

```
CREATE EVENT SESSION [lentas] ON SERVER
ADD EVENT sqlserver.sql_statement_completed
(WHERE duration > 1000000) -- > 1 s (µs)
ADD TARGET package0.event_file
(SET filename = N'lentas.xel')
WITH (STARTUP_STATE = OFF);
GO
ALTER EVENT SESSION [lentas] ON SERVER
STATE = START;
```

SQL

- system_health ya está activa por defecto y recoge deadlocks y errores graves. Mírala primero tras un incidente.
- Filtra siempre con predicados para limitar la sobrecarga.

 OBJETIVO PEDAGÓGICO

Saber lanzar consultas con sqlcmd y crear una sesión básica de Extended Events para capturar sentencias lentas.

 GUION DEL DOCENTE Y EXPLICACIÓN TÉCNICA

- sqlcmd -S servidor -U usuario -P clave -Q "consulta" ejecuta una sentencia y sale. -E usa autenticación de Windows; -i ejecuta un fichero; -o redirige a salida; -b hace que falle con código de error si hay error (útil en scripts). -C confía en el certificado del servidor (necesario con ODBC 18 cuando el certificado es autofirmado).
- **Nunca dejar contraseñas en texto claro en scripts o en el historial de shell:** usar variables de entorno (SQLCMDPASSWORD) o autenticación integrada.
- **Extended Events:** arquitectura event-driven ligera. Una sesión tiene eventos (qué ocurre), acciones (datos adicionales), predicados (filtros) y targets (dónde se escribe: event_file, ring_buffer).
- En el ejemplo, sql_statement_completed con duration > 1.000.000 µs (1 segundo) y destino fichero .xel. Siempre filtrar con predicados para limitar sobrecarga. Se lee con sys.fn_xe_file_target_read_file o con el visor de SSMS.
- La sesión system_health viene activa por defecto: captura deadlocks, errores graves, esperas largas. Es el primer sitio donde mirar tras un incidente.
- **Para eliminar:** DROP EVENT SESSION [lentas] ON SERVER; Para ver las sesiones activas: sys.dm_xe_sessions.

 PUNTOS DE INTERACCIÓN Y PREGUNTAS PARA LA CLASE

- ¿Qué riesgo tiene capturar todos los eventos sin predicados en un servidor muy cargado?
- ¿Qué información útil esperaríais encontrar en system_health tras una caída de rendimiento?

 INSTRUCCIONES DE LABORATORIO Y PAUTAS DE RESOLUCIÓN

- Conectar con sqlcmd al contenedor y ejecutar SELECT @@VERSION; verificar el código de salida con echo \$?.
- Crear la sesión "lentas", ejecutar WAITFOR DELAY '00:00:02' desde otra conexión y abrir el visor de datos en vivo en SSMS.
- **Trampa:** el nombre del fichero .xel es relativo al directorio de log de la instancia; en Linux, ruta /var/opt/mssql/log/.

DIAPOSITIVA 19

LABORATORIO · BASE DE DATOS

1 · Fundamentos y Arquitectura. Licenciamiento

Lab 1 (I): BD con ficheros y filegroups

 CONTENIDO CLAVE EN PANTALLA (PROYECCIÓN)

- T-SQL · LABORATORIO 1

```
CREATE DATABASE Ventas
ON PRIMARY
(NAME = Ventas_sys, FILENAME = '/var/opt/mssql/data/Ventas.mdf',
 SIZE = 64MB, FILEGROWTH = 64MB),
FILEGROUP FG_DATOS
(NAME = Ventas_d1, FILENAME = '/var/opt/mssql/data/Ventas_d1.ndf',
 SIZE = 256MB, FILEGROWTH = 128MB),
(NAME = Ventas_d2, FILENAME = '/var/opt/mssql/data/Ventas_d2.ndf',
 SIZE = 256MB, FILEGROWTH = 128MB)
LOG ON
(NAME = Ventas_log, FILENAME = '/var/opt/mssql/log/Ventas.ldf',
 SIZE = 128MB, FILEGROWTH = 64MB);
GO
```

SQL

- Enunciado
- Crea la BD Ventas con los ficheros indicados
- Dos ficheros del mismo tamaño en FG_DATOS
- Autogrowth en MB, nunca en %
- Comprueba el resultado en SSMS (Propiedades → Archivos)

 OBJETIVO PEDAGÓGICO

Aplicar lo aprendido: crear una base de datos con filegroup propio, varios ficheros de datos y log dimensionado, usando T-SQL.

 GUION DEL DOCENTE Y EXPLICACIÓN TÉCNICA

- **Resumen de lo que debe lograr el alumno:** crear Ventas con filegroup PRIMARY (catálogo), filegroup FG_DATOS con dos ficheros de igual tamaño (proportional fill) y un fichero de log.
- **Insistir en la elección de tamaños:** inicial razonable y FILEGROWTH en MB. Es habitual que los alumnos pongan porcentajes o 1 MB por costumbre.
- **Las rutas de ejemplo corresponden al contenedor Linux (/var/opt/mssql/data y /log). En Windows serían p. ej. C:\SQLData.**

 PUNTOS DE INTERACCIÓN Y PREGUNTAS PARA LA CLASE

- ¿Qué pasaría si los dos ficheros de FG_DATOS tuvieran tamaños distintos? (Proportional fill: el más grande recibe más escrituras.)
- ¿Por qué conviene que PRIMARY contenga sólo el catálogo y los datos del usuario vayan a otro filegroup?

 INSTRUCCIONES DE LABORATORIO Y PAUTAS DE RESOLUCIÓN

- **Paso 1:** conectar a la instancia y comprobar SELECT @@SERVERNAME (hábito de seguridad).
- **Paso 2:** ejecutar el CREATE DATABASE Ventas de la diapositiva. Si da error de ruta, comprobar que el directorio existe y que el usuario mssql tiene permisos.
- **Paso 3:** verificar en SSMS (Propiedades de la BD → Archivos) que aparecen 4 ficheros y 2 filegroups.
- **Trampas habituales:** comillas simples tipográficas copiadas de Word; olvidar la coma entre ficheros del mismo filegroup; usar rutas de Windows en el contenedor Linux.
- **Solución:** ver el script completo de la diapositiva; si el alumno ya tiene una BD Ventas, hacer DROP DATABASE Ventas primero (con USE master).

DIAPOSITIVA 20

LABORATORIO · BASE DE DATOS

1 · Fundamentos y Arquitectura. Licenciamiento

Lab 1 (II): verificar y hacer crecer

 CONTENIDO CLAVE EN PANTALLA (PROYECCIÓN)

• T-SQL · VERIFICACIÓN

```
ALTER DATABASE Ventas MODIFY FILEGROUP FG_DATOS DEFAULT;  
GO  
SELECT name, physical_name, size*8/1024 AS size_mb,  
       growth*8/1024 AS growth_mb, is_percent_growth  
FROM Ventas.sys.database_files;  
  
SELECT name, is_default FROM Ventas.sys.filegroups;
```

SQL

- Comprueba
- Cuatro ficheros, dos filegroups, FG_DATOS predeterminado.
- Vigila
- is_percent_growth = 0 en todos los ficheros.
- Cierre del módulo: Buffer Pool, páginas y extensiones, Transaction Log (WAL) y tempdb explican la mayoría de incidencias que veremos en los módulos siguientes.

 OBJETIVO PEDAGÓGICO

Verificar la estructura creada con las vistas de catálogo, establecer el filegroup predeterminado y comprobar el comportamiento del crecimiento.

 GUION DEL DOCENTE Y EXPLICACIÓN TÉCNICA

- Tras crear la BD, fijar FG_DATOS como filegroup predeterminado: las tablas nuevas irán allí por defecto y PRIMARY queda reservado para el catálogo. Es una práctica muy extendida.
- sys.database_files muestra tamaño en páginas de 8 KB (de ahí size*8/1024 para MB) y las propiedades de crecimiento; is_percent_growth debe ser 0 en todos.
- **Para ver el efecto del crecimiento:** crear una tabla ancha en FG_DATOS e insertar más datos que los 512 MB iniciales; los ficheros crecerán en bloques de 128 MB y se puede consultar en sys.database_files o en el reporte de eventos de SSMS.
- **Cierre del Módulo 1:** repasar los cuatro conceptos que han aparecido hoy (Buffer Pool, páginas/extensiones, Transaction Log/WAL, tempdb) y conectarlos con los módulos siguientes.

 PUNTOS DE INTERACCIÓN Y PREGUNTAS PARA LA CLASE

- ¿Qué diferencia observáis entre el tamaño reservado y el espacio realmente usado? (FILEPROPERTY(name,'SpaceUsed').)
- ¿Qué haríais si un fichero de datos alcanza el límite de disco? (Añadir fichero en otra unidad, no shrink indiscriminado.)

 INSTRUCCIONES DE LABORATORIO Y PAUTAS DE RESOLUCIÓN

- **Paso 1:** ejecutar ALTER DATABASE Ventas MODIFY FILEGROUP FG_DATOS DEFAULT.
- **Paso 2:** ejecutar las dos consultas de verificación y comprobar que size_mb es 64/256/256/128 y que growth está en MB.
- **Paso 3 (extra):** crear una tabla dbo.Prueba(id INT IDENTITY, relleno CHAR(8000)) y insertar 100.000 filas con un WHILE; revisar si ha crecido algún fichero.
- **Trampas:** querer cambiar el filegroup predeterminado a uno que no contiene ficheros (error); confundir size (páginas) con MB.
- **Resolución:** las consultas de la diapositiva con size*8/1024 devuelven MB; si growth = 16384 y is_percent_growth = 0, equivale a 128 MB (16384 páginas x 8 KB).

7,0 H · DIAPOSITIVAS 21 – 42

MÓDULO 2: GESTIÓN Y SEGURIDAD

DDL, restricciones, procedimientos, DML analítico, permisos, roles y Lab 2

DIAPOSITIVA 21

INTRODUCCIÓN MÓDULO

2 · Gestión y Seguridad

02 · Apertura del Módulo: Gestión y Seguridad

CONTENIDO CLAVE EN PANTALLA (PROYECCIÓN)

- MÓDULO 2 · ~7 h
- Gestión y Seguridad
- DDL, DML y modelo de seguridad de SQL Server
- Tablas, restricciones y vistas
- Procedimientos almacenados y plan cache
- JOINS, subconsultas y window functions
- Logins, users, roles y permisos
- Ownership chaining y mínimo privilegio
- Laboratorio 2: consultas y permisos

OBJETIVO PEDAGÓGICO

Situar al grupo en el recorrido del módulo: primero se construyen y consultan los datos (DDL y DML) y después se protegen (seguridad). Las dos mitades comparten la misma BD de ejemplo, Ventas.

GUION DEL DOCENTE Y EXPLICACIÓN TÉCNICA

- Este módulo es el más práctico del curso (~7 h): alterna teoría breve con T-SQL que los alumnos ejecutan en su instancia (Docker o local).
- **Hilo conductor:** la base de datos Ventas con dbo.Cliente y dbo.Pedido. Todos los ejemplos, el Laboratorio 2A y el 2B reutilizan esas tablas, de modo que cada concepto se apoya en el anterior.
- **Bloque 1 (DDL):** tablas con restricciones, vistas estándar e indexadas, procedimientos almacenados y cómo el motor cachea y recompila planes (Plan Cache).
- **Bloque 2 (DML avanzado):** orden lógico del SELECT, JOINS lógicos y sus tres algoritmos físicos (Nested Loops, Merge, Hash), subconsultas, CTE y window functions.
- **Bloque 3 (seguridad):** autenticación, jerarquía Login → User → Rol → Permiso, GRANT/DENY/REVOKE, ownership chaining, mínimo privilegio y auditoría básica.
- **Aviso de planificación:** Laboratorio 2A tras DML (~1 h) y Laboratorio 2B al final de seguridad (~1 h). Si el grupo va justo de tiempo, se recortan ejercicios opcionales, nunca los pasos de verificación.
- **Requisito previo:** SSMS o Azure Data Studio conectados a una instancia 2019/2022 con permisos de sysadmin (para crear logins y bases de datos).

PUNTOS DE INTERACCIÓN Y PREGUNTAS PARA LA CLASE

- ¿Qué parte de vuestro trabajo actual depende más de que el modelo de datos esté bien definido (integridad) y cuál de que esté bien protegido?
- ¿Alguna vez habéis visto una aplicación conectándose con sa o con un usuario db_owner? ¿Qué podría salir mal?

DIAPOSITIVA 22

DDL

Creación de tablas y restricciones

2 · Gestión y Seguridad

CONTENIDO CLAVE EN PANTALLA (PROYECCIÓN)

- T-SQL · BD Ventas

SQL

```
CREATE TABLE dbo.Cliente (
  ClienteId INT IDENTITY(1,1) NOT NULL,
  Email NVARCHAR(150) NOT NULL,
  Nombre NVARCHAR(100) NOT NULL,
  Alta DATETIME2(0) NOT NULL
  CONSTRAINT DF_Cliente_Alta DEFAULT SYSDATETIME(),
  CONSTRAINT PK_Cliente PRIMARY KEY (ClienteId),
  CONSTRAINT UQ_Cliente_Email UNIQUE (Email)
);

CREATE TABLE dbo.Pedido (
  PedidoId INT IDENTITY(1,1) NOT NULL,
  ClienteId INT NOT NULL,
  Fecha DATE NOT NULL,
  Total DECIMAL(12,2) NOT NULL,
  CONSTRAINT PK_Pedido PRIMARY KEY (PedidoId),
  CONSTRAINT FK_Pedido_Cliente FOREIGN KEY (ClienteId)
    REFERENCES dbo.Cliente (ClienteId),
  CONSTRAINT CK_Pedido_Total CHECK (Total >= 0)
);
```

- Las cinco restricciones
- PK: identifica la fila; crea índice clustered
- FK: integridad referencial; sin índice automático
- UNIQUE: valores no repetidos (índice único)
- CHECK: regla de dominio por fila
- DEFAULT: valor si se omite la columna

OBJETIVO PEDAGÓGICO

Que el alumno sepa declarar tablas con las cinco restricciones básicas (PK, FK, CHECK, UNIQUE, DEFAULT) y entienda qué hace el motor internamente con cada una.

GUION DEL DOCENTE Y EXPLICACIÓN TÉCNICA

- Mostrar el script de dbo.Cliente y dbo.Pedido línea a línea. Se nombran explícitamente todas las restricciones (PK_, FK_, UQ_, CK_, DF_): si no, SQL Server genera nombres con sufijo aleatorio (p. ej. PK__Cliente__A1B2C3) que complican los scripts de despliegue y las comparaciones entre entornos.
- PRIMARY KEY:** identifica la fila, obliga a NOT NULL y crea un índice único; por defecto es CLUSTERED salvo que ya exista uno. Elegir una clave estrecha, estática y creciente (IDENTITY) reduce Page Splits y el tamaño de todos los índices non-clustered, que incluyen la clave del clustered.
- UNIQUE:** crea un índice único non-clustered. Admite un solo NULL por columna (a diferencia de la norma ANSI). Para «único salvo NULL» se usa un índice filtrado: CREATE UNIQUE INDEX ... WHERE Col IS NOT NULL.
- FOREIGN KEY:** garantiza integridad referencial, pero SQL Server NO crea el índice en la columna hija. Sin índice, un DELETE en el padre obliga a escanear la tabla hija. Es el error de diseño más frecuente.
- CHECK:** se evalúa en INSERT/UPDATE; si es «trusted», el optimizador la usa para eliminar ramas imposibles del plan. DEFAULT: se aplica cuando la columna se omite en el INSERT; es una restricción a nivel de columna.
- Tipos de datos:** DECIMAL(12,2) para importes (nunca FLOAT/MONEY por redondeos), DATETIME2(0) en lugar de DATETIME (más preciso y compacto), NVARCHAR sólo si se necesita Unicode.
- Consultar el catálogo:** sys.key_constraints, sys.foreign_keys, sys.check_constraints, sys.default_constraints; o EXEC sp_help N'dbo.Pedido'.

PUNTOS DE INTERACCIÓN Y PREGUNTAS PARA LA CLASE

- ¿Qué pasaría con un DELETE de un cliente que tiene 2 millones de pedidos si la FK no tiene índice de apoyo?
- ¿Es mejor validar una regla de negocio en la aplicación, con un CHECK o con ambas? ¿Quién protege la BD de otra aplicación que escriba directamente?

DIAPOSITIVA 23

BUENAS PRÁCTICAS

2 · Gestión y Seguridad

Errores comunes en el diseño DDL

 CONTENIDO CLAVE EN PANTALLA (PROYECCIÓN)

- Tabla sin clave primaria
- Un heap sin PK admite duplicados y complica replicación y CDC. Define siempre una PK.
- FK sin índice de apoyo
- SQL Server no indexa la FK: un DELETE en el padre escanea la tabla hija y bloquea.
- Restricciones WITH NOCHECK
- La restricción queda «no confiable» y el optimizador deja de aprovecharla.
- VARCHAR mezclado con NVARCHAR
- La conversión implícita puede impedir un Index Seek y dispara la CPU.
- GUID aleatorio como clustered
- NEWID() fragmenta el índice con Page Splits. Prefiere IDENTITY o NEWSEQUENTIALID().
- Tipos sobredimensionados
- Más páginas, más I/O y más memoria en el Buffer Pool para el mismo dato.

 OBJETIVO PEDAGÓGICO

Interiorizar los seis errores de diseño más habituales en producción y su consecuencia medible en rendimiento o integridad.

 GUION DEL DOCENTE Y EXPLICACIÓN TÉCNICA

- **Tabla sin PK (heap):** permite duplicados, complica replicación transaccional, CDC y los UPDATE/DELETE con forwarded records. Salvo staging, toda tabla debe tener PK.
- **FK sin índice en la columna hija:** provoca scans y bloqueos largos al borrar o actualizar en el padre. Detectarlo con sys.foreign_keys cruzado con sys.index_columns.
- **WITH NOCHECK:** añade la restricción sin validar los datos existentes y la marca como no confiable (sys.foreign_keys.is_not_trusted = 1). El optimizador no puede usarla y pierde optimizaciones como la eliminación de joins. Reparar con ALTER TABLE ... WITH CHECK CHECK CONSTRAINT.
- **VARCHAR vs NVARCHAR:** NVARCHAR usa 2 bytes por carácter; mezclarlos en un JOIN o WHERE genera conversión implícita (CONVERT_IMPLICIT en el plan) y puede impedir un Index Seek por precedencia de tipos.
- **GUID aleatorio como clustered key (NEWID()):** inserciones por toda la B-Tree, Page Splits, fragmentación y Buffer Pool ineficiente. Alternativas: IDENTITY, SEQUENCE o NEWSEQUENTIALID().
- **Tipos sobredimensionados (BIGINT, VARCHAR(MAX), CHAR(500)):** más páginas de 8 KB, más I/O y más memoria de Buffer Pool para el mismo dato; y VARCHAR(MAX) no se puede indexar como clave.
- **Regla práctica:** el diseño físico se revisa con la herramienta de los datos, no con intuición: sys.dm_db_index_usage_stats y los planes de ejecución.

 PUNTOS DE INTERACCIÓN Y PREGUNTAS PARA LA CLASE

- ¿Cuál de estos errores creéis que existe hoy en alguna de vuestras bases de datos? ¿Cómo lo detectaríais con una consulta al catálogo?
- ¿Por qué un GUID aleatorio es mala idea como clustered key, pero puede ser aceptable como clave no agrupada?

DIAPOSITIVA 24

VISTAS

Vistas estándar frente a vistas indexadas

2 · Gestión y Seguridad

 CONTENIDO CLAVE EN PANTALLA (PROYECCIÓN)

- Vista estándar
- Consulta guardada, sin datos propios
- Se expande dentro de la consulta externa
- Simplifica joins y centraliza permisos
- Coste = el de la consulta subyacente
- Vista indexada
- Resultado materializado en un índice
- Se mantiene en cada INSERT, UPDATE y DELETE
- Acelera agregaciones repetitivas
- Exige SCHEMABINDING y penaliza escrituras
- T-SQL · vista estándar

SQL

```
CREATE VIEW dbo.vPedidoCliente AS
SELECT c.ClienteId, c.Nombre, p.PedidoId, p.Fecha, p.Total
FROM   dbo.Cliente AS c JOIN dbo.Pedido AS p ON p.ClienteId = c.ClienteId;
```

 OBJETIVO PEDAGÓGICO

Distinguir una vista estándar (consulta almacenada, sin datos) de una vista indexada (resultado materializado) y saber cuándo conviene cada una.

 GUION DEL DOCENTE Y EXPLICACIÓN TÉCNICA

- **Vista estándar:** sólo se guarda la definición en sys.sql_modules. Al consultarla, el optimizador la expande (view expansion) dentro de la consulta externa y genera un único plan; no hay penalización ni ganancia de rendimiento por sí misma.
- **Usos:** abstraer joins complejos, ocultar columnas sensibles y dar un punto de seguridad estable (GRANT SELECT sobre la vista sin dar acceso a la tabla; el ownership chaining lo hace posible).
- **Anidar vistas sobre vistas es un antipatrón:** cada capa añade joins que el optimizador debe simplificar y los planes se vuelven difíciles de leer.
- **Vista indexada (materializada):** al crear sobre ella un UNIQUE CLUSTERED INDEX, SQL Server almacena físicamente el resultado y lo mantiene sincronizado en cada INSERT/UPDATE/DELETE de las tablas base (coste en escrituras).
- En ediciones Enterprise/Developer el optimizador puede usar la vista indexada aunque la consulta no la nombre (matching automático); en Standard hay que usar la pista WITH (NOEXPAND).
- **Cuándo usarla:** agregaciones costosas y muy repetidas sobre tablas de lectura intensiva (informes, dashboards). Cuándo no: tablas con alta tasa de escritura, porque cada DML paga el mantenimiento.
- **Consulta útil:** SELECT name, type_desc FROM sys.views; y sys.indexes WHERE object_id = OBJECT_ID('dbo.vVentasCliente').

 PUNTOS DE INTERACCIÓN Y PREGUNTAS PARA LA CLASE

- ¿Una vista estándar mejora el rendimiento de una consulta? ¿Por qué sí o por qué no?
- Si una tabla recibe 5 000 inserciones por segundo, ¿pondrías una vista indexada sobre ella? ¿Qué medirías antes de decidir?

DIAPOSITIVA 25

VISTAS

Vistas indexadas: SCHEMABINDING y requisitos

2 · Gestión y Seguridad

CONTENIDO CLAVE EN PANTALLA (PROYECCIÓN)

- T-SQL · vista indexada

SQL

```
CREATE VIEW dbo.vVentasCliente
WITH SCHEMABINDING
AS
SELECT  p.ClienteId,
        COUNT_BIG(*) AS NumPedidos,
        SUM(p.Total) AS TotalVentas
FROM    dbo.Pedido AS p
GROUP BY p.ClienteId;
GO
CREATE UNIQUE CLUSTERED INDEX IX_vVentasCliente
ON      dbo.vVentasCliente (ClienteId);
GO
SELECT * FROM dbo.vVentasCliente WITH (NOEXPAND);
```

- Coste: cada DML sobre dbo.Pedido también actualiza la vista.
- Requisitos principales
- SCHEMABINDING y nombres de dos partes
- COUNT_BIG(*) si hay GROUP BY
- Sin OUTER JOIN, subconsultas, DISTINCT, TOP, UNION ni MIN/MAX
- Primer índice: UNIQUE CLUSTERED
- Opciones SET: ARITHABORT, ANSI_NULLS y QUOTED_IDENTIFIER en ON
- Standard: usar NOEXPAND

OBJETIVO PEDAGÓGICO

Crear correctamente una vista indexada conociendo sus restricciones y sus costes de mantenimiento.

GUION DEL DOCENTE Y EXPLICACIÓN TÉCNICA

- WITH SCHEMABINDING enlaza la vista a las tablas base:** impide ALTER/DROP de columnas usadas mientras la vista exista. Obliga a nombres de dos partes (dbo.Pedido) y prohíbe SELECT *.
- Si la vista agrupa (GROUP BY), debe incluir COUNT_BIG(*): permite al motor mantener el resultado de forma incremental al borrar filas. SUM sólo sobre expresiones no nulas (aquí Total es NOT NULL).
- Construcciones prohibidas:** OUTER JOIN, subconsultas, DISTINCT, TOP, UNION, MIN/MAX/AVG, ORDER BY, CTE, tablas derivadas y auto-joins. Expresiones no deterministas (GETDATE) tampoco.
- El primer índice debe ser UNIQUE CLUSTERED; después se pueden crear índices non-clustered adicionales sobre la vista.
- Opciones SET requeridas al crearla y al modificar las tablas base:** ANSI_NULLS, ANSI_PADDING, ANSI_WARNINGS, ARITHABORT, CONCAT_NULL_YIELDS_NULL y QUOTED_IDENTIFIER en ON; NUMERIC_ROUNDABORT en OFF. SSMS las trae bien por defecto; ciertos drivers antiguos no (error 1934).
- Consumo:** en Standard hay que escribir WITH (NOEXPAND); en Enterprise el optimizador puede usarla automáticamente. Verificar en el plan: ¿aparece un Clustered Index Scan sobre la vista o se expande a las tablas base?
- Advertencia de rendimiento:** cada INSERT sobre dbo.Pedido provoca también una modificación en el índice de la vista, con su bloqueo y su log. Medir con SET STATISTICS IO y sys.dm_db_index_operational_stats.

PUNTOS DE INTERACCIÓN Y PREGUNTAS PARA LA CLASE

- ¿Por qué creéis que se exige COUNT_BIG(*) cuando hay GROUP BY?
- Un desarrollador quiere borrar una columna de dbo.Pedido y recibe un error de dependencia. ¿Qué lo causa y cómo lo localizaríais?

DIAPOSITIVA 26

PROGRAMABILIDAD

2 · Gestión y Seguridad

Procedimientos almacenados

 CONTENIDO CLAVE EN PANTALLA (PROYECCIÓN)

- T-SQL · procedimiento

```
CREATE OR ALTER PROCEDURE dbo.usp_PedidosPorCliente
    @ClienteId INT,
    @Desde DATE = '19000101'
AS
BEGIN
    SET NOCOUNT ON;
    SELECT p.PedidoId, p.Fecha, p.Total
    FROM    dbo.Pedido AS p
    WHERE   p.ClienteId = @ClienteId
    AND     p.Fecha >= @Desde
    ORDER BY p.Fecha DESC;
END;
GO
EXEC dbo.usp_PedidosPorCliente @ClienteId = 42;
```

SQL

- Por qué usarlos
- Plan reutilizable en el Plan Cache
- EXECUTE sin dar acceso a las tablas
- Lógica encapsulada con TRY...CATCH
- SET NOCOUNT ON reduce tráfico
- Parámetros tipados frente a SQL concatenado

 OBJETIVO PEDAGÓGICO

Entender qué aporta un procedimiento almacenado (rendimiento, encapsulación y seguridad) y escribir uno con las buenas prácticas básicas.

 GUION DEL DOCENTE Y EXPLICACIÓN TÉCNICA

- Un procedimiento almacenado (stored procedure, SP) es código T-SQL con nombre guardado en la BD. Se compila en la primera ejecución y el plan queda en el Plan Cache para reutilizarse.
- **Parámetros tipados:** evitan la concatenación de cadenas y, con ella, la inyección SQL. Un parámetro con DEFAULT lo hace opcional.
- **SET NOCOUNT ON suprime los mensajes «n filas afectadas»:** reduce tráfico de red y evita que algunos drivers interpreten esos mensajes como resultados.
- CREATE OR ALTER (desde SQL Server 2016 SP1) conserva permisos y metadatos al modificar, a diferencia de DROP + CREATE que obliga a reasignar GRANTS.
- **Seguridad:** se puede dar EXECUTE sobre el procedimiento sin dar permisos sobre las tablas gracias al ownership chaining (se verá en la parte de seguridad). Es la «API» de la BD.
- **Gestión de errores y transacciones:** TRY...CATCH, THROW y SET XACT_ABORT ON para que cualquier error aborta y revierta la transacción abierta.
- **Metadatos útiles:** sys.procedures, sys.sql_modules (definición), sys.dm_exec_procedure_stats (veces ejecutado, tiempo, lecturas lógicas por procedimiento).

 PUNTOS DE INTERACCIÓN Y PREGUNTAS PARA LA CLASE

- ¿Qué ventajas tiene dar EXECUTE sobre un procedimiento frente a dar SELECT/INSERT sobre las tablas?
- ¿En qué se diferencia un procedimiento almacenado de una consulta enviada como cadena desde la aplicación (ad hoc)?

DIAPOSITIVA 27

PROGRAMABILIDAD

2 · Gestión y Seguridad

Plan caching, sniffing y recompilaciones

CONTENIDO CLAVE EN PANTALLA (PROYECCIÓN)

- 1
- Compilar
- El optimizador crea el plan con estadísticas y el valor del parámetro.
- 2
- Cachear
- El plan se guarda en el Plan Cache, localizado por hash del texto.
- 3
- Reutilizar
- Las siguientes ejecuciones se saltan la compilación.
- 4
- Recompilar
- Cambios de esquema, estadísticas o SET options lo invalidan.
- T-SQL · mitigaciones y DMV

SQL

```
-- Mitigaciones de parameter sniffing
EXEC dbo.usp_PedidosPorCliente @ClienteId = 42 WITH RECOMPILE;
-- En la consulta: OPTION (RECOMPILE) / OPTION (OPTIMIZE FOR UNKNOWN)
SELECT TOP (5) usecounts, size_in_bytes, objtype
FROM sys.dm_exec_cached_plans ORDER BY usecounts DESC;
```

- Sniffing: el primer valor compilado fija el plan para todos los demás.

OBJETIVO PEDAGÓGICO

Comprender el ciclo compilar → cachear → reutilizar → recompilar y reconocer un problema de parameter sniffing con sus mitigaciones.

GUION DEL DOCENTE Y EXPLICACIÓN TÉCNICA

- **Compilar es caro (CPU):** parse, binding (algebraizer), optimización basada en costes con estadísticas. El plan resultante se guarda en el Plan Cache, parte del Buffer Pool, y se localiza por un hash del texto de la consulta más atributos (SET options, esquema por defecto).
- **Reutilizar evita volver a compilar:** es la gran ventaja de procedimientos y consultas parametrizadas. Las consultas ad hoc con literales distintos generan un plan cada una (cache bloat); mitigación: parametrizar, «optimize for ad hoc workloads» o PARAMETERIZATION FORCED.
- **Parameter sniffing:** en la primera ejecución el optimizador «espía» el valor real del parámetro y construye un plan óptimo para ese valor. Si los datos están sesgados (un cliente con 60 % de los pedidos), ese plan (Index Seek + Key Lookup) puede ser desastroso para el valor frecuente (necesita Scan).
- **No es un bug, es un compromiso. Mitigaciones:** OPTION (RECOMPILE) por consulta, OPTION (OPTIMIZE FOR UNKNOWN) o OPTIMIZE FOR (@p = valor), WITH RECOMPILE en el procedimiento, y Query Store con plan forcing (2016+). SQL Server 2022 añade Parameter Sensitive Plan (PSP) optimization con nivel de compatibilidad 160.
- **Recompilación:** el plan se invalida por cambios de esquema, actualización de estadísticas (umbral dinámico en compat. 130+), cambios de SET options, sp_recompile o ALTER del objeto. Desde 2005 la recompilación es a nivel de sentencia, no de todo el procedimiento.
- **DMVs:** sys.dm_exec_cached_plans (usecounts, size_in_bytes), sys.dm_exec_query_stats, sys.dm_exec_sql_text(plan_handle) y sys.dm_exec_query_plan(plan_handle). Contadores: «SQL Re-Compilations/sec» y «Batch Requests/sec».
- **DBCC FREEPROCCACHE vacía la caché:** sólo en entornos de prueba o con un plan_handle concreto; en producción provoca una tormenta de compilaciones.

PUNTOS DE INTERACCIÓN Y PREGUNTAS PARA LA CLASE

- Un procedimiento va en 20 ms para unos clientes y en 40 s para otros. ¿Qué hipótesis plantearíais y qué miraríais primero en el plan?
- ¿Por qué vaciar el Plan Cache «para arreglarlo» sólo oculta el problema durante un tiempo?

DIAPOSITIVA 28

DML AVANZADO

2 · Gestión y Seguridad

SELECT complejos: orden lógico de ejecución

 CONTENIDO CLAVE EN PANTALLA (PROYECCIÓN)

- 1
- FROM
- Origen y JOINS
- 2
- WHERE
- Filtra filas
- 3
- GROUP BY
- Agrupa filas
- 4
- HAVING
- Filtra grupos
- 5
- SELECT
- Columnas y OVER
- 6
- ORDER BY
- Ordena y TOP
- T-SQL · ventas por cliente y año

```
SELECT c.Nombre, YEAR(p.Fecha) AS Anio, SUM(p.Total) AS Ventas
FROM dbo.Cliente AS c
JOIN dbo.Pedido AS p ON p.ClienteId = c.ClienteId
WHERE p.Fecha >= '20240101'
GROUP BY c.Nombre, YEAR(p.Fecha)
HAVING SUM(p.Total) > 1000
ORDER BY Ventas DESC;
```

SQL

Alias: **ORDER BY** puede usar alias de **SELECT** porque se evalúa después; **WHERE** no.

SQL

 OBJETIVO PEDAGÓGICO

Aprender que el orden de escritura de un **SELECT** no es el orden en que se evalúa, y usarlo para razonar sobre alias, filtros y agregados.

 GUION DEL DOCENTE Y EXPLICACIÓN TÉCNICA

- **El orden lógico (logical query processing) es:** FROM/JOIN → WHERE → GROUP BY → HAVING → SELECT (incluidas window functions) → DISTINCT → ORDER BY → TOP/OFFSET. Es un modelo conceptual: el optimizador puede reordenar físicamente mientras el resultado sea equivalente.
- **Consecuencia 1:** WHERE no puede usar alias definidos en SELECT (todavía no existen), pero ORDER BY sí. Consecuencia 2: WHERE filtra filas antes de agrupar; HAVING filtra grupos después de agregar.
- **Consecuencia 3:** las window functions se evalúan en la fase SELECT, por eso no pueden aparecer en WHERE ni en GROUP BY; para filtrarlas hay que envolverlas en una CTE o subconsulta.
- **Rendimiento:** filtrar pronto reduce filas. Predicados SARGable (Search ARGument ABLE) como Fecha >= '20240101' permiten Index Seek; envolver la columna en una función (YEAR(Fecha) = 2024) obliga a escanear.
- Evitar SELECT *: aumenta lecturas, rompe vistas y procedimientos al cambiar el esquema e impide usar índices cubrientes (covering).
- El ejemplo agrupa por cliente y año. Pedir a los alumnos que predigan el número de columnas del resultado y qué ocurre si se mueve SUM(p.Total) > 1000 del HAVING al WHERE (error: función de agregado no válida en WHERE).
- Mostrar el plan (Ctrl+M) para ver cómo el Stream Aggregate o Hash Match Aggregate aparece antes del Sort.

 PUNTOS DE INTERACCIÓN Y PREGUNTAS PARA LA CLASE

- ¿Por qué puedo escribir ORDER BY Ventas DESC usando el alias, pero no WHERE Ventas > 1000?
- ¿Qué diferencia práctica hay entre filtrar en WHERE y en HAVING cuando la condición no usa agregados?

DIAPOSITIVA 29

DML AVANZADO

2 · Gestión y Seguridad

JOINS: tipos lógicos y trampas habituales

 CONTENIDO CLAVE EN PANTALLA (PROYECCIÓN)

- Trampa: filtrar la tabla derecha en el WHERE convierte el LEFT JOIN en INNER.
- T-SQL · LEFT JOIN

SQL

```
-- 1) Anti-join: clientes sin pedidos
SELECT c.ClienteId, c.Nombre
FROM   dbo.Cliente AS c
LEFT JOIN dbo.Pedido AS p
      ON p.ClienteId = c.ClienteId
WHERE  p.PedidoId IS NULL;
```

```
-- 2) Filtra la tabla derecha en el ON
SELECT c.Nombre, p.PedidoId
FROM   dbo.Cliente AS c
LEFT JOIN dbo.Pedido AS p
      ON p.ClienteId = c.ClienteId
      AND p.Fecha >= '20240101';
```

Tabla Resumen en Diapositiva:

[object Object]undefined

 OBJETIVO PEDAGÓGICO

Elegir el tipo de JOIN lógico adecuado y evitar la trampa clásica de filtrar la tabla externa en el WHERE.

 GUION DEL DOCENTE Y EXPLICACIÓN TÉCNICA

- INNER JOIN devuelve sólo filas con coincidencia. LEFT JOIN conserva todas las filas de la izquierda y rellena con NULL; es la base del anti-join (clientes sin pedidos con WHERE p.PedidoId IS NULL).
- RIGHT JOIN es equivalente a un LEFT con las tablas invertidas; se evita por legibilidad. FULL JOIN se usa en conciliaciones entre dos fuentes. CROSS JOIN genera el producto cartesiano (n x m filas); útil para calendarios y combinaciones, peligroso si es accidental.
- **Trampa principal:** un predicado sobre la tabla derecha en el WHERE (p.Fecha >= ...) descarta las filas con NULL y convierte el LEFT JOIN en INNER JOIN. Si se quiere conservar los clientes sin pedidos en el rango, el filtro va en la cláusula ON.
- **Trampa 2:** duplicación de filas por relaciones 1:N. Sumar un importe del lado «uno» después de un JOIN a «varios» multiplica el total; agregar antes del JOIN o usar APPLY.
- **Trampa 3:** JOIN con columnas de tipos distintos (varchar vs nvarchar, int vs varchar) → conversión implícita y scans. Las claves de join deben tener el mismo tipo y collation.
- **Alternativas semánticas:** EXISTS / NOT EXISTS para semi y anti-joins (no multiplican filas); NOT IN falla silenciosamente si la subconsulta devuelve algún NULL.
- Pedir que ejecuten ambas consultas de la derecha y comparen el número de filas devuelto.

 PUNTOS DE INTERACCIÓN Y PREGUNTAS PARA LA CLASE

- ¿Cuántas filas devuelve un CROSS JOIN entre 1 000 clientes y 365 días? ¿Y entre dos tablas de 1 millón?
- Si queréis clientes sin pedidos en 2024 (aunque tengan en otros años), ¿dónde pondríais el filtro de fecha y por qué?

DIAPOSITIVA 30

DML AVANZADO

2 · Gestión y Seguridad

JOINS: algoritmos físicos del optimizador

 CONTENIDO CLAVE EN PANTALLA (PROYECCIÓN)

- Nested Loops
- Por cada fila externa busca en la interna
- Ideal: externa pequeña + Index Seek
- Típico en OLTP
- Merge Join
- Recorre dos entradas ordenadas
- Exige igualdad y orden (índice o Sort)
- Eficiente con grandes volúmenes
- Hash Match
- Tabla hash con la entrada pequeña
- No exige índices ni orden
- Usa memory grant; puede derramar a tempdb
- En el plan: compara Estimated frente a Actual Rows. Estadísticas obsoletas provocan joins inadecuados.

 OBJETIVO PEDAGÓGICO

Reconocer en un plan de ejecución los tres operadores físicos de join, saber cuándo es adecuado cada uno y qué señales indican una mala elección.

 GUION DEL DOCENTE Y EXPLICACIÓN TÉCNICA

- Un JOIN lógico se implementa con un operador físico que el optimizador elige por coste, según cardinalidad estimada, índices y orden disponible.
- **Nested Loops:** por cada fila de la entrada externa (outer) busca coincidencias en la interna (inner). Brilla cuando la externa es pequeña y la interna tiene un índice para Index Seek. Coste $\approx$ filas externas $\times$ coste del seek. Es el típico de consultas OLTP.
- **Merge Join:** recorre en paralelo dos entradas ordenadas por la clave de join, una sola pasada. Requiere condición de igualdad y orden (de un índice o de un operador Sort, caro). Muy eficiente con volúmenes grandes ya ordenados.
- **Hash Match:** construye una tabla hash en memoria con la entrada más pequeña (build) y «sondea» con la grande (probe). No necesita índices ni orden; es el caballo de batalla de cargas analíticas. Consume memory grant: si se subestima, se derrama a tempdb (Hash Spill, aviso en el plan).
- **Señales de problema:** Nested Loops con millones de iteraciones (estimación de filas muy baja), warnings de spill en Hash/Sort, diferencias grandes entre Estimated y Actual Rows (estadísticas obsoletas o parameter sniffing).
- **Pistas de join (OPTION (HASH JOIN), LOOP JOIN, MERGE JOIN) existen pero se usan sólo como último recurso diagnóstico:** fijan el algoritmo y ocultan el problema de fondo.
- **Novedades:** Adaptive Joins (2017+, batch mode) deciden entre Hash y Nested Loops en tiempo de ejecución; en 2019+ el batch mode también funciona sobre tablas rowstore. DMVs: sys.dm_exec_query_memory_grants y sys.dm_exec_query_stats.

 PUNTOS DE INTERACCIÓN Y PREGUNTAS PARA LA CLASE

- ¿Por qué un Hash Match suele aparecer en consultas analíticas y un Nested Loops en consultas OLTP?
- Si veis un Nested Loops con 5 millones de ejecuciones en la entrada interna, ¿qué sospecharíais sobre las estimaciones?

DIAPOSITIVA 31

DML AVANZADO

Subconsultas, EXISTS y CTE

2 · Gestión y Seguridad

 CONTENIDO CLAVE EN PANTALLA (PROYECCIÓN)

- T-SQL · subconsulta y EXISTS

```
-- Subconsulta escalar
SELECT PedidoId, Total
FROM   dbo.Pedido
WHERE  Total > (SELECT AVG(Total)
               FROM   dbo.Pedido);

-- EXISTS (correlacionada)
SELECT c.Nombre
FROM   dbo.Cliente AS c
WHERE  EXISTS (SELECT 1 FROM dbo.Pedido AS p
               WHERE p.ClienteId = c.ClienteId);
```

SQL

- T-SQL · CTE

```
WITH VentasCliente AS (
    SELECT ClienteId, SUM(Total) AS Ventas
    FROM   dbo.Pedido
    GROUP BY ClienteId
)
SELECT TOP (5) c.Nombre, v.Ventas
FROM   VentasCliente AS v
JOIN   dbo.Cliente AS c ON c.ClienteId = v.ClienteId
ORDER BY v.Ventas DESC;
```

SQL

- Recuerda: NOT IN con algún NULL devuelve cero filas: usa NOT EXISTS. Una CTE no materializa datos: se expande como una vista en línea.

 OBJETIVO PEDAGÓGICO

Usar subconsultas escalares y correlacionadas, EXISTS y CTE de forma legible sin penalizar el rendimiento.

 GUION DEL DOCENTE Y EXPLICACIÓN TÉCNICA

- Subconsulta escalar:** devuelve un único valor y se usa como expresión (Total > (SELECT AVG(Total) ...)). Si devolviera más de una fila, error 512. No correlacionada: se evalúa una vez.
- Subconsulta correlacionada:** referencia columnas de la consulta externa (p.ClienteId = c.ClienteId). Lógicamente se evalúa por cada fila, pero el optimizador la transforma casi siempre en un semi-join (Left Semi Join) con un coste mucho menor.
- EXISTS se detiene en la primera coincidencia y devuelve verdadero/falso; es ideal para «existe algún pedido». IN sobre una lista es equivalente en la mayoría de los casos, pero NOT IN devuelve cero filas si la subconsulta contiene un NULL: preferir NOT EXISTS.
- CTE (Common Table Expression, WITH ... AS):** mejora la legibilidad y permite recursividad (jerarquías). En SQL Server NO se materializa: se expande como una vista en línea cada vez que se referencia; si se usa 3 veces, se ejecuta 3 veces. Para reutilizar un resultado costoso, usar una tabla temporal #tabla.
- CROSS APPLY / OUTER APPLY:** permiten invocar una subconsulta correlacionada o una función con valores de tabla por cada fila (típico: «los 3 últimos pedidos de cada cliente»).
- Rendimiento:** comprobar siempre el plan; una subconsulta en la lista de SELECT correlacionada puede ejecutarse fila a fila (Nested Loops con muchas iteraciones). Reescribir como JOIN agregado suele ser más estable.
- Ejercicio rápido:** reescribir la consulta EXISTS con un JOIN y comparar los planes y el número de lecturas lógicas con SET STATISTICS IO ON.

 PUNTOS DE INTERACCIÓN Y PREGUNTAS PARA LA CLASE

- ¿Qué devuelve WHERE ClienteId NOT IN (SELECT ClienteId FROM ...) si la subconsulta contiene un NULL? ¿Y con NOT EXISTS?
- ¿Una CTE guarda el resultado en memoria o en tempdb? ¿Cómo lo comprobaríais en el plan de ejecución?

DIAPOSITIVA 32

DML AVANZADO

2 · Gestión y Seguridad

Agregados y window functions básicas

CONTENIDO CLAVE EN PANTALLA (PROYECCIÓN)

- T-SQL · window functions

```
SELECT ClienteId, Fecha, Total,
  ROW_NUMBER() OVER (PARTITION BY ClienteId
                     ORDER BY Fecha DESC) AS rn,
  RANK()        OVER (ORDER BY Total DESC) AS ranking,
  SUM(Total)    OVER (PARTITION BY ClienteId
                     ORDER BY Fecha
                     ROWS UNBOUNDED PRECEDING) AS acumulado,
  LAG(Total)    OVER (PARTITION BY ClienteId
                     ORDER BY Fecha) AS anterior
FROM dbo.Pedido;
```

SQL

- GROUP BY vs OVER: GROUP BY colapsa filas; OVER mantiene el detalle. Indica ROWS en el frame para evitar el worktable de RANGE.

Tabla Resumen en Diapositiva:

[object Object]undefined

OBJETIVO PEDAGÓGICO

Diferenciar GROUP BY (colapsa filas) de las window functions (mantienen el detalle) y aplicar ROW_NUMBER, RANK, SUM OVER y LAG.

GUION DEL DOCENTE Y EXPLICACIÓN TÉCNICA

- Funciones de agregado (COUNT, SUM, AVG, MIN, MAX) con GROUP BY colapsan el conjunto en una fila por grupo. Las window functions calculan sobre una «ventana» de filas relacionadas pero devuelven una fila por cada fila de entrada, conservando el detalle.
- Sintaxis:** función() OVER (PARTITION BY ... ORDER BY ... [ROWS|RANGE ...]). PARTITION BY define los grupos (como un GROUP BY sin colapsar); ORDER BY define el orden dentro de la partición; la cláusula de frame define las filas incluidas.
- ROW_NUMBER(): numera 1, 2, 3 sin empates (los desempates son no deterministas si el ORDER BY no es único). RANK(): los empates comparten posición y deja huecos (1, 1, 3); DENSE_RANK() no deja huecos.
- SUM(...) OVER (PARTITION BY ... ORDER BY ...):** total acumulado (running total). Advertencia de rendimiento: si no se especifica frame, el valor por defecto es RANGE UNBOUNDED PRECEDING, que usa un worktable en disco; especificar ROWS UNBOUNDED PRECEDING es mucho más rápido y suele ser lo que se desea.
- LAG(col, n, default) y LEAD(col, n):** acceden a filas anterior/siguiente sin auto-join (disponibles desde SQL Server 2012). Útiles para variaciones periodo a periodo.
- Las window functions sólo se permiten en SELECT y ORDER BY; para filtrar por rn = 1 («último pedido por cliente») hay que usar CTE/subconsulta. Un índice con PARTITION BY y ORDER BY como claves (en ese orden) evita el operador Sort.
- En el plan buscar los operadores Segment, Sequence Project y Window Spool (si aparece Window Spool, revisar el frame).

PUNTOS DE INTERACCIÓN Y PREGUNTAS PARA LA CLASE

- ¿Qué diferencia hay entre ROW_NUMBER y RANK cuando dos pedidos tienen el mismo importe?
- ¿Cómo obtendríais el último pedido de cada cliente usando una window function? ¿Y con GROUP BY?

DIAPOSITIVA 33

LABORATORIO · CONSULTAS Y SP

2 · Gestión y Seguridad

Laboratorio 2A: consultas y procedimientos

CONTENIDO CLAVE EN PANTALLA (PROYECCIÓN)

- 1
- Preparar datos
- Crea Ventas con Cliente y Pedido y carga 50 000 pedidos sesgados.
- 2
- Consulta analítica
- Top 3 pedidos por cliente con ROW_NUMBER y SUM OVER.
- 3
- Procedimiento
- Crea usp_TopPedidosCliente con parámetros y SET NOCOUNT ON.
- 4
- Plan y sniffing
- Ejecuta con cliente 1 y con otro; compara planes con Ctrl+M.
- T-SQL · punto de partida del paso 2

```
WITH R AS (SELECT ClienteId, PedidoId, Total,
ROW_NUMBER() OVER (PARTITION BY ClienteId
ORDER BY Total DESC) AS rn
FROM dbo.Pedido)
SELECT * FROM R WHERE rn <= 3;
```

SQL

- Trampa: no puedes filtrar rn en el mismo SELECT; envuélvelo en una CTE.

OBJETIVO PEDAGÓGICO

Aplicar DML avanzado (window functions, CTE) y encapsular el resultado en un procedimiento almacenado, observando plan caching y parameter sniffing en vivo.

GUION DEL DOCENTE Y EXPLICACIÓN TÉCNICA

- **Duración orientativa:** 60–70 min. Los alumnos trabajan individualmente o por parejas sobre la BD Ventas creada en las diapositivas de DDL.
- **Objetivo técnico:** ver con sus propios ojos que (1) las window functions resuelven en una consulta lo que antes requería auto-joins, (2) un procedimiento reutiliza plan y (3) el mismo plan puede ser bueno para un parámetro y malo para otro.
- **El docente recorre los puestos al paso 3:** es donde más se atascan (CTE antes del SELECT, ORDER BY dentro de la CTE, nombres de columna).
- **Criterio de éxito:** el procedimiento devuelve las 3 mejores ventas de cada cliente y el alumno sabe explicar por qué el plan para @Clienteld = 1 (60 % de las filas) es diferente del de un cliente raro.
- **Cierre:** puesta en común de 5 minutos; pedir a un grupo que enseñe su plan con el Key Lookup y a otro el plan con Clustered Index Scan.

PUNTOS DE INTERACCIÓN Y PREGUNTAS PARA LA CLASE

- ¿Por qué el plan de la segunda ejecución es idéntico al de la primera aunque el parámetro sea distinto?
- ¿Qué cambia en el plan al añadir OPTION (RECOMPILE)? ¿Qué coste tiene hacerlo en cada ejecución?

INSTRUCCIONES DE LABORATORIO Y PAUTAS DE RESOLUCIÓN

- **Requisito:** contenedor SQL Server 2022 en marcha (docker run -e ACCEPT_EULA=Y -e MSSQL_SA_PASSWORD= -p 1433:1433 mcr.microsoft.com/mssql/server:2022-latest) y SSMS conectado a localhost,1433.
- Paso 1 — Datos: CREATE DATABASE Ventas; USE Ventas; crear dbo.Cliente y dbo.Pedido con los scripts de la diapositiva 22. Cargar 500 clientes: INSERT dbo.Cliente (Email, Nombre) SELECT TOP (500) CONCAT('c', n, '@demo.es'), CONCAT(N'Cliente ', n) FROM (SELECT ROW_NUMBER() OVER (ORDER BY (SELECT NULL)) AS n FROM sys.all_objects a CROSS JOIN sys.all_objects b) AS t ORDER BY n;
- Paso 1b — Pedidos sesgados: INSERT dbo.Pedido (Clienteld, Fecha, Total) SELECT TOP (50000) CASE WHEN ABS(CHECKSUM(NEWID())) % 10 < 6 THEN 1 ELSE 2 + ABS(CHECKSUM(NEWID())) % 499 END, DATEADD(DAY, -ABS(CHECKSUM(NEWID())) % 730, CAST(GETDATE() AS date)), CAST(10 + ABS(CHECKSUM(NEWID())) % 990 AS decimal(12,2)) FROM sys.all_objects a CROSS JOIN sys.all_objects b; Después: CREATE INDEX IX_Pedido_Cliente ON dbo.Pedido (Clienteld) INCLUDE (Fecha, Total);
- Paso 2 — Consulta analítica: top 3 pedidos por importe de cada cliente con ROW_NUMBER() OVER (PARTITION BY Clienteld ORDER BY Total DESC) dentro de una CTE y WHERE rn <= 3 fuera. Añadir una columna con SUM(Total) OVER (PARTITION BY Clienteld) para el % sobre el total del cliente.
- Paso 3 — Procedimiento: CREATE OR ALTER PROCEDURE dbo.usp_TopPedidosCliente @Clienteld INT, @N INT = 3 con SET NOCOUNT ON y SELECT TOP (@N) ... ORDER BY Total DESC. Probar EXEC con @Clienteld = 1 y luego con 250.
- Paso 4 — Plan y sniffing: activar plan real (Ctrl+M) y SET STATISTICS IO ON. Ejecutar primero con un cliente raro (Seek + Key Lookup) y luego con el cliente 1 (compara lecturas lógicas). Ver el plan cacheado: SELECT usecounts, plan_handle FROM sys.dm_exec_cached_plans CROSS APPLY sys.dm_exec_sql_text(plan_handle) WHERE text LIKE '%usp_TopPedidosCliente%'. Repetir con EXEC ... WITH RECOMPILE y comparar.
- **Trampas habituales:** (a) ORDER BY dentro de una CTE sin TOP → error 1033; (b) filtrar rn en el mismo SELECT donde se calcula → error 4108 (usar CTE); (c) ejecutar CREATE PROCEDURE sin GO previo cuando hay más sentencias en el lote; (d) olvidar USE Ventas y crear los objetos en master.

- **Guía de resolución:** si el alumno no ve diferencia de planes, confirmar que el sesgo existe (`SELECT Clienteld, COUNT(*) FROM dbo.Pedido GROUP BY Clienteld ORDER BY 2 DESC`) y que el índice `IX_Pedido_Cliente` está creado; vaciar la caché sólo con `DBCC FREEPROCCACHE` en el contenedor de laboratorio.

DIAPOSITIVA 34

SEGURIDAD

Modelo de seguridad de SQL Server

2 · Gestión y Seguridad

 CONTENIDO CLAVE EN PANTALLA (PROYECCIÓN)

- NIVEL DE INSTANCIA (servidor)
- NIVEL DE BASE DE DATOS
- Autenticación
- Windows o SQL Server
- Login
- Principal de servidor
- User
- Principal de base de datos
- Roles y permisos
- GRANT · DENY · REVOKE
- Objetos
- Schemas, tablas, SP, vistas
- Recuerda: un login abre la puerta de la instancia; los permisos los decide el user en cada base de datos.

 OBJETIVO PEDAGÓGICO

Presentar la cadena completa de seguridad (autenticación → login → user → roles/permisos → objetos) y la frontera entre nivel de instancia y de base de datos.

 GUION DEL DOCENTE Y EXPLICACIÓN TÉCNICA

- **Una petición atraviesa varias capas en orden.** 1) **Autenticación:** ¿quién eres? Windows (Kerberos/NTLM) o SQL Server (usuario y contraseña). 2) **Login:** principal a nivel de servidor que permite entrar a la instancia (sys.server_principals).
- **3) User:** principal de base de datos mapeado a un login (sys.database_principals); sin user (o guest) no se accede a la BD aunque el login sea válido. 4) **Roles y permisos:** GRANT/DENY/REVOKE sobre objetos, schemas o la propia BD. 5) **Objetos:** tablas, vistas, procedimientos, agrupados en schemas.
- **Los permisos se evalúan jerárquicamente:** servidor → base de datos → schema → objeto → columna. Un permiso concedido en un nivel alto (p. ej. SELECT sobre la BD) se hereda por los inferiores, salvo DENY.
- **Principals:** logins, users, roles de servidor/BD y roles de aplicación. Securables: servidor, BD, schema, objeto. Permission: CONNECT, SELECT, EXECUTE, ALTER, CONTROL, etc.
- **Idea clave para el DBA:** el login abre la puerta de la instancia, pero no da acceso a ningún dato; ahí entran el user, los roles y los permisos. Por eso «tener un login» y «tener acceso» no son lo mismo.
- **Otras capas complementarias fuera de esta diapositiva:** cifrado en tránsito (TLS), TDE, Always Encrypted, Row-Level Security y Dynamic Data Masking. Se mencionan, no se desarrollan.
- **Vistas de catálogo de referencia:** sys.server_principals, sys.database_principals, sys.server_permissions, sys.database_permissions y sys.fn_my_permissions.

 PUNTOS DE INTERACCIÓN Y PREGUNTAS PARA LA CLASE

- ¿Puede un login con credenciales válidas conectarse a la instancia y aun así no ver ninguna base de datos de usuario? ¿Por qué?
- ¿Qué diferencia hay entre un permiso concedido a nivel de schema y uno concedido tabla a tabla en una base de datos con 500 tablas?

DIAPOSITIVA 35

SEGURIDAD

Autenticación: Windows frente a SQL Server

2 · Gestión y Seguridad

 CONTENIDO CLAVE EN PANTALLA (PROYECCIÓN)

- T-SQL · modo y política

```
-- 1 = sólo Windows, 0 = modo mixto
SELECT SERVERPROPERTY('IsIntegratedSecurityOnly') AS SoloWindows;
SELECT name, is_policy_checked, is_expiration_checked
FROM sys.sql_logins;
```

SQL

- Modo mixto: requiere reiniciar el servicio y deja sa expuesta: deshabíltala.

Tabla Resumen en Diapositiva:

[object Object]undefined

 OBJETIVO PEDAGÓGICO

Comparar los dos modos de autenticación, saber cuándo habilitar el modo mixto y cuáles son sus riesgos.

 GUION DEL DOCENTE Y EXPLICACIÓN TÉCNICA

- **Autenticación de Windows (integrada):** SQL Server confía en la identidad validada por Active Directory (Kerberos, o NTLM como alternativa). Se gestionan contraseñas, caducidad, bloqueo y MFA en el directorio, y se pueden crear logins para grupos de AD, lo que simplifica el alta y baja de personas.
- **Autenticación de SQL Server:** el login y el hash de la contraseña (SHA-512 con sal desde 2012) se guardan en master (sys.sql_logins). Sirve para aplicaciones heredadas, servidores fuera de dominio, instancias en Linux o contenedores sin AD y herramientas de terceros.
- **Modo de autenticación del servidor:** «Sólo Windows» o «Mixto» (Windows + SQL). Se cambia en Propiedades del servidor → Seguridad y requiere reiniciar el servicio. SERVERPROPERTY('IsIntegratedSecurityOnly') indica el modo (1 = sólo Windows).
- **En modo mixto la cuenta sa pasa a estar activa:** es el objetivo número uno de los ataques de fuerza bruta. Buenas prácticas: contraseña robusta, cambiar el nombre (ALTER LOGIN sa WITH NAME = ...) o deshabilitarla (ALTER LOGIN sa DISABLE).
- Para logins SQL activar CHECK_POLICY = ON (aplica la política de contraseñas de Windows) y CHECK_EXPIRATION según el caso. Visible en sys.sql_logins (is_policy_checked, is_expiration_checked).
- **Problema del doble salto (double hop):** con Kerberos mal configurado (SPN ausente), las conexiones remotas caen a NTLM o fallan. Verificar con SELECT auth_scheme FROM sys.dm_exec_connections WHERE session_id = @@SPID.
- En Docker/Linux (contenedor del laboratorio) la autenticación inicial es SQL (sa); Windows/AD se habilita con adutil en SQL Server 2019+ pero queda fuera del alcance del curso.

 PUNTOS DE INTERACCIÓN Y PREGUNTAS PARA LA CLASE

- ¿Qué ventajas operativas tiene crear un login para un grupo de Active Directory en lugar de uno por persona?
- ¿Qué haríais con la cuenta sa en un servidor en modo mixto expuesto a varias aplicaciones? ¿Y si una aplicación sólo soporta SQL Auth?

DIAPOSITIVA 36

SEGURIDAD

Logins, Users y Schemas

2 · Gestión y Seguridad

 CONTENIDO CLAVE EN PANTALLA (PROYECCIÓN)

- LOGIN lg_analista
- Nivel de instancia (master)
- USER analista
- Nivel de BD: Ventas, mismo SID
- ROL rol_lectura_rpt
- Agrupa permisos
- SCHEMA rpt
- Objetos y permisos
- T-SQL · login, schema y user

```
-- 1) Instancia: el login
CREATE LOGIN lg_analista
  WITH PASSWORD = N'C0mpl3ja!2024', CHECK_POLICY = ON;

-- 2) Base de datos: schema y user mapeado
USE Ventas;
GO
CREATE SCHEMA rpt AUTHORIZATION dbo;
GO
CREATE USER analista FOR LOGIN lg_analista
  WITH DEFAULT_SCHEMA = rpt;
```

SQL

Huérfano: tras un restore, **ALTER USER ... WITH LOGIN = ...** repara el SID.

SQL

 OBJETIVO PEDAGÓGICO

Distinguir login (instancia) de user (base de datos), entender su mapeo por SID y usar los schemas como espacio de nombres y unidad de permisos.

 GUION DEL DOCENTE Y EXPLICACIÓN TÉCNICA

- Login = principal de servidor; User = principal de base de datos. Se enlazan por el SID (Security Identifier). CREATE USER ... FOR LOGIN guarda el SID del login en sys.database_principals.
- Un login puede tener un user distinto en cada BD; sin user en la BD (y guest deshabilitado) no hay acceso. Cada BD tiene el user especial dbo (propietario) y los de sistema (guest, INFORMATION_SCHEMA, sys).
- **Usuario huérfano (orphaned user):** tras restaurar una BD en otra instancia, el SID del user no coincide con ningún login. Detectar con sp_change_users_login 'Report' o comparando SIDs; reparar con ALTER USER ... WITH LOGIN = login. Alternativa moderna: bases de datos contenidas (CONTAINMENT = PARTIAL) con usuarios con contraseña propios.
- Schema = espacio de nombres y contenedor de seguridad dentro de la BD. No es lo mismo que el user (desde 2005 están separados): el propietario del schema puede cambiarse sin renombrar objetos.
- **Un permiso concedido a nivel de schema (GRANT SELECT ON SCHEMA::rpt)** cubre los objetos actuales y los futuros; es la forma más mantenible de segregar permisos (rpt para informes, app para procedimientos de aplicación, dbo para administración).
- DEFAULT_SCHEMA del user determina cómo se resuelven nombres sin calificar; mejor calificar siempre (dbo.Pedido) para evitar resoluciones ambiguas y ahorrar recompilaciones en el plan cache.
- **Cuidado con CREATE LOGIN ... WITH PASSWORD en scripts versionados:** nunca contraseñas reales en el control de código. Para Windows: CREATE LOGIN [DOMINIO\ana] FROM WINDOWS (sólo con AD).

 PUNTOS DE INTERACCIÓN Y PREGUNTAS PARA LA CLASE

- ¿Qué ocurre si restauráis una copia de Ventas en otro servidor donde el login lg_analista existe pero se creó con otro SID?
- ¿Qué ventaja tiene organizar los objetos en schemas (rpt, app, dbo) frente a tenerlo todo en dbo?

DIAPOSITIVA 37

SEGURIDAD

Roles fijos y roles personalizados

2 · Gestión y Seguridad

CONTENIDO CLAVE EN PANTALLA (PROYECCIÓN)

- T-SQL · rol personalizado

```
CREATE ROLE rol_lectura_rpt;
GRANT SELECT ON SCHEMA::rpt
    TO rol_lectura_rpt;

CREATE ROLE rol_operador;
GRANT EXECUTE ON SCHEMA::app
    TO rol_operador;

ALTER ROLE rol_lectura_rpt
    ADD MEMBER analista;
```

SQL

- Mejor práctica: evita db_owner y sysadmin en aplicaciones; usa roles personalizados con permisos sobre un schema.

Tabla Resumen en Diapositiva:

[object Object]undefined

OBJETIVO PEDAGÓGICO

Conocer los roles fijos más relevantes y justificar por qué se prefieren roles personalizados con permisos sobre schemas.

GUION DEL DOCENTE Y EXPLICACIÓN TÉCNICA

- Roles fijos de servidor:** sysadmin (control total, ignora DENY), securityadmin (administra logins; casi equivale a sysadmin por escalada), dbcreator (crea/restaura BDs), serveradmin, processadmin, diskadmin, bulkadmin, setupadmin y public. SQL Server 2022 añade roles granulares como ##MS_ServerStateReader## para monitorización sin sysadmin.
- Roles fijos de BD:** db_owner (control total de la BD), db_datareader/db_datawriter (SELECT/INSERT-UPDATE-DELETE sobre todas las tablas), db_ddladmin (DDL), db_securityadmin, db_accessadmin, db_backupoperator y los db_denydatareader/db_denydatawriter.
- El rol public existe en servidor y BD, y todos los principals pertenecen a él:** no se le debe conceder nada que no se quiera dar a todo el mundo.
- Problema de los roles fijos:** son «todo o nada» (db_datareader lee todas las tablas presentes y futuras). Para el mínimo privilegio se definen roles personalizados (CREATE ROLE) y se les concede permisos sobre un schema o procedimientos concretos.
- Gestión:** ALTER ROLE rol ADD MEMBER usuario (desde 2012; sp_addrolemember está obsoleto). En SQL Server 2012+ también existen roles de servidor definidos por el usuario (CREATE SERVER ROLE).
- Los roles pueden anidarse (un rol miembro de otro), pero la anidación excesiva complica las auditorías. Convención:** rol_<función>, p. ej. rol_lectura_rpt, rol_operador.
- Auditar membresías:** SELECT r.name AS rol, m.name AS miembro FROM sys.database_role_members rm JOIN sys.database_principals r ON r.principal_id = rm.role_principal_id JOIN sys.database_principals m ON m.principal_id = rm.member_principal_id; y sys.server_role_members para servidor.

PUNTOS DE INTERACCIÓN Y PREGUNTAS PARA LA CLASE

- ¿Por qué un login miembro de securityadmin debe tratarse casi como un sysadmin?
- ¿Qué ocurre con las tablas creadas el próximo mes si añadimos a un usuario a db_datareader? ¿Y si le damos SELECT sobre un schema?

DIAPOSITIVA 38

SEGURIDAD

GRANT, DENY y REVOKE

2 · Gestión y Seguridad

 CONTENIDO CLAVE EN PANTALLA (PROYECCIÓN)

- GRANT
- Concede un permiso; WITH GRANT OPTION permite delegarlo.
- DENY
- Prohíbe el permiso y gana siempre al GRANT, también el heredado.
- REVOKE
- Quita un GRANT o un DENY previo; no equivale a denegar.
- T-SQL · permisos sobre dbo.Pedido

```
GRANT SELECT, INSERT ON dbo.Pedido TO rol_operador;
DENY DELETE ON dbo.Pedido TO rol_operador;
REVOKE INSERT ON dbo.Pedido FROM rol_operador;
```

SQL

```
-- Permisos efectivos del contexto actual
```

```
SELECT * FROM sys.fn_my_permissions(N'dbo.Pedido', N'OBJECT');
```

- Precedencia: DENY > GRANT > sin permiso. Los roles se acumulan, salvo DENY.

 OBJETIVO PEDAGÓGICO

Aplicar correctamente las tres sentencias de permisos y razonar sobre la precedencia entre ellas y entre roles.

 GUION DEL DOCENTE Y EXPLICACIÓN TÉCNICA

- GRANT concede un permiso (SELECT, INSERT, UPDATE, DELETE, EXECUTE, ALTER, CONTROL...) sobre un securable. WITH GRANT OPTION permite al receptor delegarlo; usarlo con extrema cautela.
- **DENY prohíbe explícitamente el permiso y tiene precedencia sobre cualquier GRANT, incluso heredado de un rol o de un nivel superior. Excepciones:** no afecta a sysadmin ni al user dbo.
- **REVOKE elimina un GRANT o un DENY previo y devuelve al estado «sin permiso explícito»:** no equivale a denegar, y el usuario puede seguir teniendo el permiso por otro camino (rol o schema).
- **Orden de evaluación:** DENY > GRANT > sin permiso (denegado implícito). Los permisos de roles se acumulan, salvo un DENY en cualquiera de ellos.
- **Los permisos se pueden aplicar a niveles distintos:** servidor, BD, schema (GRANT SELECT ON SCHEMA::rpt), objeto, columna (GRANT SELECT (Nombre, Email) ON dbo.Cliente). Favorecer schema y rol.
- **Verificación:** sys.fn_my_permissions('dbo.Pedido', 'OBJECT') para el contexto actual; combinar con EXECUTE AS USER = 'x' para probar como otro usuario (siempre cerrar con REVERT). Catálogo: sys.database_permissions.
- **Error típico del alumno:** usar DENY como mecanismo principal. DENY es una excepción, no una estrategia: complica el análisis de permisos efectivos. Mejor construir la lista de GRANT mínima.

 PUNTOS DE INTERACCIÓN Y PREGUNTAS PARA LA CLASE

- Un usuario pertenece a dos roles: uno con GRANT SELECT sobre dbo.Pedido y otro con DENY SELECT. ¿Qué ocurre al consultar? ¿Y si se hace REVOKE del DENY?
- ¿Cómo comprobarais los permisos efectivos de un usuario sin iniciar sesión como él?

DIAPOSITIVA 39

SEGURIDAD

Ownership chaining (cadena de propiedad)

2 · Gestión y Seguridad

 CONTENIDO CLAVE EN PANTALLA (PROYECCIÓN)

- Cadena intacta (mismo propietario)
- Usuario operador
- app.usp_AltaPedido
- owner: dbo
- dbo.Pedido
- owner: dbo
- Sólo se valida
- EXECUTE
- Cadena rota (propietarios distintos)
- Usuario operador
- app.usp_Otro
- owner: usr_app
- dbo.Pedido
- owner: dbo
- Se exige permiso
- sobre la tabla
- Utilidad: da EXECUTE sobre el procedimiento, no acceso a las tablas: es una API controlada.
- Límites: el SQL dinámico rompe la cadena; entre bases de datos exige DB_CHAINING.

 OBJETIVO PEDAGÓGICO

Entender cómo la cadena de propiedad permite exponer datos sólo a través de procedimientos o vistas y cuándo se rompe.

 GUION DEL DOCENTE Y EXPLICACIÓN TÉCNICA

- Cuando un objeto (vista, procedimiento) accede a otro con el mismo propietario, SQL Server comprueba sólo el permiso sobre el primer objeto (p. ej. EXECUTE) y omite la comprobación sobre los objetos subyacentes. Eso es el ownership chaining.
- **Aplicación práctica:** el operador recibe EXECUTE sobre app.usp_AltaPedido y no tiene ningún permiso sobre dbo.Pedido. El procedimiento escribe en la tabla porque el propietario del schema app y de dbo es el mismo (dbo): la cadena es intacta.
- **La cadena se rompe cuando el propietario cambia a lo largo de ella (p. ej. procedimiento propiedad de otro user):** entonces se evalúan los permisos del llamador sobre el objeto siguiente y falla con error 229 si no los tiene.
- **Limitaciones:** sólo cubre SELECT, INSERT, UPDATE, DELETE y EXECUTE; no cubre DDL (TRUNCATE TABLE, CREATE, ALTER) ni, importante, SQL dinámico: EXEC(@sql) y sp_executesql se ejecutan en otro contexto y rompen la cadena.
- Entre bases de datos, el cross-database ownership chaining está desactivado por defecto (DB_CHAINING OFF) y es un riesgo si se activa: un propietario de BD con db_owner puede alcanzar objetos de otras BDs.
- **Alternativas explícitas y más controlables:** EXECUTE AS OWNER en el módulo (cambio de contexto) o firma de procedimientos con certificados (module signing).
- **Comprobación en laboratorio:** EXECUTE AS USER = 'operador'; EXEC app.usp_AltaPedido ...; SELECT * FROM dbo.Pedido; (error 229). Documentarlo: es la forma más clara de demostrarlo a los alumnos.

 PUNTOS DE INTERACCIÓN Y PREGUNTAS PARA LA CLASE

- ¿Por qué un procedimiento con SQL dinámico (EXEC(@sql)) falla con «permiso denegado» aunque el usuario tenga EXECUTE sobre él?
- ¿Qué riesgo introduce activar cross-database ownership chaining entre dos bases de datos con administradores distintos?

DIAPOSITIVA 40

BUENAS PRÁCTICAS

2 · Gestión y Seguridad

Mínimo privilegio y auditoría básica

 CONTENIDO CLAVE EN PANTALLA (PROYECCIÓN)

- Mínimo privilegio
- Permisos a roles, no a usuarios sueltos
- GRANT sobre schemas o procedimientos
- Cuentas de aplicación ≠ administración
- sa deshabilitada y sysadmin revisado
- Auditoría básica
- SQL Server Audit a archivo o Event Log
- Login auditing: fallos y éxitos
- DMVs: sys.dm_exec_sessions, sys.server_principals
- Triggers DDL / Extended Events
- T-SQL · SQL Server Audit

SQL

```
CREATE SERVER AUDIT Aud_Seguridad
TO FILE (FILEPATH = N'/var/opt/mssql/audit/');
CREATE SERVER AUDIT SPECIFICATION Aud_Seg_Spec
FOR SERVER AUDIT Aud_Seguridad
ADD (FAILED_LOGIN_GROUP) WITH (STATE = ON);
ALTER SERVER AUDIT Aud_Seguridad WITH (STATE = ON);
```

- Lectura: consulta los eventos con sys.fn_get_audit_file.

 OBJETIVO PEDAGÓGICO

Traducir el principio de mínimo privilegio en reglas concretas y conocer las tres herramientas básicas de auditoría de SQL Server.

 GUION DEL DOCENTE Y EXPLICACIÓN TÉCNICA

- **Mínimo privilegio:** cada principal recibe sólo los permisos imprescindibles, durante el tiempo imprescindible. En la práctica: permisos a roles (no a usuarios sueltos), GRANT sobre schemas o procedimientos, cuentas separadas para aplicaciones y para administración, y revisión periódica de miembros de sysadmin y db_owner.
- **Cuentas de servicio:** ejecutar el motor con una cuenta de bajo privilegio (idealmente gMSA o cuenta virtual), no como administrador del dominio. Deshabilitar sa y el user guest; quitar CONNECT a guest en las BDs de usuario.
- **SQL Server Audit:** objeto de auditoría (destino: archivo, Application Log o Security Log) + especificación a nivel de servidor o de BD con grupos de acciones (FAILED_LOGIN_GROUP, DATABASE_ROLE_MEMBER_CHANGE_GROUP, SCHEMA_OBJECT_ACCESS_GROUP...). Disponible en todas las ediciones a nivel de servidor; las especificaciones de BD son para todas desde 2016 SP1.
- **Login auditing:** Propiedades del servidor → Seguridad → Auditoría de inicio de sesión (Ninguno, Sólo erróneos, Sólo correctos, Ambos); escribe en el Error Log. Barato y suficiente para detectar ataques de fuerza bruta (error 18456).
- **DMVs y catálogo para revisión:** sys.dm_exec_sessions (quién está conectado y desde dónde), sys.dm_exec_connections (auth_scheme), sys.server_principals, sys.database_permissions y sys.dm_server_audit_status.
- **Leer el resultado de la auditoría:** SELECT * FROM sys.fn_get_audit_file('var/opt/mssql/audit/*.sqlaudit', DEFAULT, DEFAULT). Extended Events y triggers DDL cubren cambios de esquema.
- **Advertencia:** una auditoría demasiado amplia (p. ej. SELECT sobre tablas calientes) genera mucho volumen y penaliza el rendimiento; auditar lo que importa y proteger el destino contra manipulación.

 PUNTOS DE INTERACCIÓN Y PREGUNTAS PARA LA CLASE

- ¿Qué eventos auditaríais primero en un servidor de producción con presupuesto limitado de I/O?
- ¿Cómo detectaríais que un usuario ha sido añadido a db_owner la semana pasada sin que nadie lo notificara?

DIAPOSITIVA 41

LABORATORIO · PERMISOS

2 · Gestión y Seguridad

Laboratorio 2B: permisos segregados

CONTENIDO CLAVE EN PANTALLA (PROYECCIÓN)

- 1
- Crear principals
- Dos logins, sus users en Ventas y los schemas rpt y app.
- 2
- Definir roles
- rol_lectura_rpt sobre rpt y rol_operador con EXECUTE sobre app.
- 3
- Asignar y denegar

Añade miembros y un DENY **SELECT** sobre dbo.Pedido al operador.

SQL

- 4
- Probar
- EXECUTE AS USER, REVERT y sys.fn_my_permissions.
- T-SQL · verificación

```
EXECUTE AS USER = 'operador';
EXEC app.usp_AltaPedido @ClienteId = 1, @Total = 99.90; -- OK
SELECT TOP (1) * FROM dbo.Pedido; -- error 229
REVERT;
```

SQL

- Trampa: olvidar REVERT deja la sesión suplantada.

OBJETIVO PEDAGÓGICO

Diseñar e implementar un esquema de permisos segregados (analista de lectura y operador de aplicación) siguiendo el mínimo privilegio, y verificarlo con EXECUTE AS.

GUIÓN DEL DOCENTE Y EXPLICACIÓN TÉCNICA

- **Duración orientativa:** 60 min. Escenario: dos perfiles sobre la BD Ventas. El analista sólo consulta informes (schema rpt); el operador sólo da de alta pedidos mediante un procedimiento (schema app) y NO debe poder leer ni modificar dbo.Pedido directamente.
- **Objetivo técnico:** poner en práctica login → user → rol → permiso sobre schema, y demostrar ownership chaining (EXECUTE sobre el procedimiento basta para insertar).
- **Criterio de éxito:** el analista puede hacer SELECT en rpt.vVentasCliente pero no en dbo.Pedido; el operador ejecuta app.usp_AltaPedido con éxito pero recibe el error 229 al hacer SELECT directo.
- **El docente debe insistir en el patrón de verificación:** probar siempre lo que NO debe poder hacerse. Una prueba sólo de éxito no demuestra el mínimo privilegio.
- **Cierre:** revisar entre todos la salida de sys.database_permissions y sys.database_role_members como inventario de permisos del esquema creado.

PUNTOS DE INTERACCIÓN Y PREGUNTAS PARA LA CLASE

- ¿Qué permisos tiene realmente el operador? Demostrad con sys.fn_my_permissions y con pruebas negativas.
- ¿Qué pasaría si el procedimiento usara SQL dinámico para insertar? ¿Cómo lo resolveríais sin dar permisos sobre la tabla?

INSTRUCCIONES DE LABORATORIO Y PAUTAS DE RESOLUCIÓN

- **Requisito:** BD Ventas con dbo.Cliente, dbo.Pedido y datos (Laboratorio 2A) y sesión con rol sysadmin.
- Paso 1 — Principals: CREATE LOGIN lg_analista WITH PASSWORD = 'N', CHECK_POLICY = ON; y lo mismo con lg_operador. USE Ventas; CREATE SCHEMA rpt AUTHORIZATION dbo; (en su propio lote con GO) y CREATE SCHEMA app AUTHORIZATION dbo; CREATE USER analista FOR LOGIN lg_analista WITH DEFAULT_SCHEMA = rpt; CREATE USER operador FOR LOGIN lg_operador WITH DEFAULT_SCHEMA = app;
- Paso 2 — Objetos: CREATE VIEW rpt.vVentasCliente AS SELECT c.ClienteId, c.Nombre, COUNT(*) AS NumPedidos, SUM(p.Total) AS Ventas FROM dbo.Cliente c JOIN dbo.Pedido p ON p.ClienteId = c.ClienteId GROUP BY c.ClienteId, c.Nombre; y CREATE PROCEDURE app.usp_AltaPedido @ClienteId INT, @Total DECIMAL(12,2) AS BEGIN SET NOCOUNT ON; INSERT dbo.Pedido (ClienteId, Fecha, Total) VALUES (@ClienteId, CAST(GETDATE() AS date), @Total); END;
- Paso 3 — Roles y permisos: CREATE ROLE rol_lectura_rpt; GRANT SELECT ON SCHEMA::rpt TO rol_lectura_rpt; ALTER ROLE rol_lectura_rpt ADD MEMBER analista; CREATE ROLE rol_operador; GRANT EXECUTE ON SCHEMA::app TO rol_operador; ALTER ROLE rol_operador ADD MEMBER operador; DENY SELECT ON dbo.Pedido TO operador;
- Paso 4 — Verificación: EXECUTE AS USER = 'operador'; EXEC app.usp_AltaPedido @ClienteId = 1, @Total = 99.90; (debe funcionar); SELECT TOP (1) * FROM dbo.Pedido; (error 229); REVERT; Repetir con EXECUTE AS USER = 'analista': SELECT * FROM rpt.vVentasCliente; (ok) y SELECT * FROM dbo.Cliente; (error 229). Inventario: SELECT * FROM sys.database_permissions WHERE grantee_principal_id IN (DATABASE_PRINCIPAL_ID('rol_lectura_rpt'), DATABASE_PRINCIPAL_ID('rol_operador'));
- Paso 5 (opcional) — Conexión real: crear conexiones nuevas en SSMS (Autenticación SQL) con cada login y repetir las pruebas.
- **Trampas habituales:** (a) CREATE SCHEMA no es la primera instrucción del lote → error 111, separar con GO; (b) olvidar REVERT tras EXECUTE AS: las pruebas siguientes se ejecutan como el usuario suplantado; (c) el DENY directo sobre dbo.Pedido NO bloquea el procedimiento porque la cadena de propiedad omite la comprobación; si el alumno espera que falle, explicar la regla; (d) usar un login sin user en Ventas → error 916 (la entidad de seguridad no puede acceder a la BD).

- **Guía de resolución:** si el operador no puede ejecutar el procedimiento, comprobar con `sys.fn_my_permissions('app.usp_AltaPedido', 'OBJECT')` y que el schema `app` pertenece a `dbo` (`sys.schemas.principal_id`); si la cadena no funciona, verificar que ambos objetos tienen el mismo propietario efectivo.

DIAPOSITIVA 42

RESUMEN

Resumen del Módulo 2: puntos clave

2 · Gestión y Seguridad

 CONTENIDO CLAVE EN PANTALLA (PROYECCIÓN)

- DDL e integridad
- PK, FK, CHECK, UNIQUE, DEFAULT
- FK con índice de apoyo
- Tipos ajustados al dato
- Vistas y SP
- Vistas indexadas con SCHEMABINDING
- Plan cache y sniffing
- SP como API segura
- DML analítico
- Orden lógico del SELECT
- Nested, Merge y Hash
- OVER, RANK y LAG
- Seguridad
- Login, user, rol, permiso
- DENY gana a GRANT
- Mínimo privilegio y auditoría
- Siguiendo módulo: índices y B-Tree, planes de ejecución, DMVs de rendimiento y alta disponibilidad (RTO/RPO).

 OBJETIVO PEDAGÓGICO

Consolidar los cuatro bloques del módulo y conectar con el siguiente (rendimiento y alta disponibilidad).

 GUION DEL DOCENTE Y EXPLICACIÓN TÉCNICA

- **DDL e integridad:** las cinco restricciones son la primera línea de calidad de datos; el índice de apoyo de las FK y los tipos ajustados evitan problemas de rendimiento futuros.
- **Vistas y procedimientos:** la vista indexada materializa agregaciones (coste en escritura); los procedimientos reutilizan plan y son la «API» de la BD, pero exigen vigilar el parameter sniffing y las recompilaciones.
- **DML analítico:** el orden lógico del SELECT explica alias, WHERE y HAVING; los tres algoritmos de join (Nested Loops, Merge, Hash) se reconocen en el plan; las window functions evitan auto-joins; usa ROWS en el frame.
- **Seguridad:** login (instancia) → user (BD) → rol → permiso sobre schema/objeto; DENY gana a GRANT; el ownership chaining permite exponer sólo procedimientos; mínimo privilegio y auditoría (SQL Server Audit, login auditing, DMVs).
- Hacer una ronda de «una cosa que aplicaré el lunes» (1 min por persona) y recoger dudas pendientes para repasarlas antes del siguiente módulo.
- **Puente al Módulo 3:** hemos visto que la elección de join, el Index Seek y el sniffing dependen de índices y estadísticas. El siguiente módulo explica cómo se construyen (B-Tree), cómo leer planes de ejecución y cómo detectar cuellos de botella con DMVs.

 PUNTOS DE INTERACCIÓN Y PREGUNTAS PARA LA CLASE

- ¿Qué decisión de este módulo tendría mayor impacto en vuestra BD de producción esta semana?
- ¿Qué concepto de seguridad os parece más difícil de explicar a un desarrollador y cómo lo haríais?

6,0 H · DIAPOSITIVAS 43 – 60

MÓDULO 3: OPTIMIZACIÓN Y ALTA DISPONIBILIDAD

Índices B-Tree, estadísticas, planes de ejecución, DMVs, Always On y Lab 3

DIAPOSITIVA 43

INTRODUCCIÓN MÓDULO

03 · Apertura del Módulo: Optimización y Alta Disponibilidad

3 · Optimización y Alta Disponibilidad

CONTENIDO CLAVE EN PANTALLA (PROYECCIÓN)

- MÓDULO 3 · 6 h
- Optimización y Alta Disponibilidad
- Índices, planes de ejecución, DMVs y fundamentos de HA/DR
- Arquitectura de índices y B-Tree
- Fragmentación y estadísticas
- Lectura de planes de ejecución
- Monitorización con DMVs
- RTO, RPO y comparativa HA/DR
- Laboratorio 3

OBJETIVO PEDAGÓGICO

Situar al alumnado en el siguiente bloque del curso: pasar de "la base de datos funciona" a "la base de datos funciona rápido y no se cae". Se presenta el mapa del módulo y los resultados esperados.

GUION DEL DOCENTE Y EXPLICACIÓN TÉCNICA

- **Este módulo dura unas 6 horas y se divide en tres bloques:** (1) optimización a nivel de datos (índices, fragmentación y estadísticas), (2) diagnóstico (planes de ejecución y DMVs) y (3) continuidad de negocio (HA/DR).
- **Mensaje clave:** en rendimiento NO se adivina, se mide. Todo el módulo sigue el ciclo medir → hipótesis → cambio → volver a medir, usando STATISTICS IO/TIME, el plan real y las Dynamic Management Views (DMVs).
- **Conectar con el Módulo 1:** lo visto sobre páginas de 8 KB, extensiones y el Buffer Pool explica por qué un índice reduce lecturas lógicas; lo visto sobre el Transaction Log será la base de Log Shipping y Always On.
- Advertir de que la mayoría de problemas de rendimiento en producción se resuelven con un buen diseño de índices y estadísticas actualizadas, antes de tocar hardware o parámetros del servidor.
- En HA/DR veremos conceptos y comparativa; la configuración paso a paso de un Availability Group queda fuera del alcance del laboratorio, pero se deja claro qué pregunta de negocio responde cada tecnología.
- **Entorno del laboratorio:** SQL Server 2019/2022 en Docker o instancia local, base de datos Ventas (dbo.Cliente, dbo.Pedido, dbo.Producto) y SSMS. Verificar que todos pueden conectarse antes de empezar.

PUNTOS DE INTERACCIÓN Y PREGUNTAS PARA LA CLASE

- ¿Cuál ha sido la consulta más lenta que habéis sufrido en producción y cómo se averiguó la causa?
- ¿Qué diferencia hay entre "alta disponibilidad" y "copia de seguridad"? Recogemos ideas y las contrastamos al final del módulo.

DIAPOSITIVA 44

ÍNDICES

Heap vs Clustered Index

3 · Optimización y Alta Disponibilidad

 CONTENIDO CLAVE EN PANTALLA (PROYECCIÓN)

- Heap (sin índice clustered)
- Filas en páginas sin orden lógico (se localizan por IAM)
- Lectura completa = Table Scan; búsqueda por RID
- Forwarded Records tras UPDATE que hace crecer la fila
- Útil en staging y cargas masivas puntuales
- Clustered Index
- La hoja del B-Tree contiene las filas, ordenadas por la clave
- Sólo uno por tabla (la PRIMARY KEY lo crea por defecto)
- Clave ideal: estrecha, única, estática y creciente
- Sin Forwarded Records, pero con posibles Page Splits
- Regla práctica: en OLTP casi toda tabla debe tener un Clustered Index estrecho y creciente (por ejemplo INT IDENTITY). Cada columna de esa clave se copia en todos los índices no clustered.

 OBJETIVO PEDAGÓGICO

Comprender cómo SQL Server organiza físicamente una tabla (Heap o Clustered Index) y por qué esa decisión condiciona todos los demás índices y el coste de cada consulta.

 GUION DEL DOCENTE Y EXPLICACIÓN TÉCNICA

- **Una tabla sin Clustered Index es un Heap:** las filas se guardan en páginas de 8 KB sin ningún orden lógico; el motor las localiza mediante las páginas IAM (Index Allocation Map). La única forma de leerlo entero es un Table Scan.
- **Un Clustered Index ordena físicamente la tabla por la clave:** el nivel hoja del B-Tree ES la tabla (data pages). Sólo puede haber uno por tabla. Al crear una PRIMARY KEY sin indicar otra cosa, SQL Server crea por defecto un Clustered Index único.
- **Problema típico del Heap:** los Forwarded Records. Si un UPDATE hace crecer una fila y no cabe en su página, la fila se mueve y deja un puntero de reenvío; cada lectura posterior cuesta una E/S adicional. Se detecta en sys.dm_db_index_physical_stats con forwarded_record_count (modo DETAILED).
- Un buen candidato a clave clustered es estrecho (menos bytes en todos los NCI, porque la clave clustered se copia en cada fila de cada índice no clustered), único (si no, SQL Server añade un uniquifier de 4 bytes), estático (cambiarla mueve la fila) y creciente (INT/BIGINT IDENTITY evita Page Splits en mitad del índice).
- Un GUID aleatorio (NEWID()) como clave clustered provoca Page Splits constantes y fragmentación; si hace falta un GUID, usar NEWSEQUENTIALID() o separar el GUID de la clave clustered.
- **Cuándo un Heap sí tiene sentido:** tablas de staging para cargas masivas (BULK INSERT, bcp) donde se escribe mucho y se lee una sola vez; incluso ahí conviene medir.
- **Consulta útil para ver qué tiene cada tabla:** SELECT OBJECT_NAME(object_id), type_desc FROM sys.indexes WHERE index_id IN (0,1); index_id 0 = Heap, 1 = Clustered.

 PUNTOS DE INTERACCIÓN Y PREGUNTAS PARA LA CLASE

- ¿Qué le ocurre a una tabla con 20 índices no clustered si elegimos como clave clustered una columna NVARCHAR(200)?
- Si una tabla de auditoría sólo recibe INSERT y nunca se consulta por clave, ¿Heap o Clustered? Argumentad el coste de cada opción.

DIAPPOSITIVA 45

ÍNDICES

Estructura B-Tree de un índice

3 · Optimización y Alta Disponibilidad

CONTENIDO CLAVE EN PANTALLA (PROYECCIÓN)

- | | | |
|---------------------|-----------------------|---|
| • Raíz (Root) | • 10–49 | • Bravo→42 |
| • Nivel intermedio | • Filas | • Casas→8 |
| • Nivel hoja (Leaf) | • 50–99 | • Díaz→61 |
| • Clustered Index | • Filas | • Mora→23 |
| • 50 100 | • 100+ | • Núñez→5 |
| • 10 30 | • Non-Clustered Index | • Torres→90 |
| • 60 80 | • M T | • Vega→33 |
| • 110 130 | • A F | • La hoja = páginas de datos (la tabla ES el índice). Hojas enlazadas en lista doble. |
| • Filas | • M P | • La hoja guarda clave + puntero (clave clustered o RID) → Key Lookup. |
| • 1–9 | • T W | |
| • Filas | • Alonso→17 | |

OBJETIVO PEDAGÓGICO

Visualizar el B-Tree como estructura común de Clustered y Non-Clustered Index para entender por qué un Index Seek necesita pocas lecturas y qué contiene realmente el nivel hoja.

GUION DEL DOCENTE Y EXPLICACIÓN TÉCNICA

- **Todos los índices rowstore se almacenan como B-Tree balanceado:** una página raíz (Root), cero o más niveles intermedios y el nivel hoja (Leaf). Un Seek recorre raíz → intermedio → hoja: típicamente 3–4 lecturas lógicas incluso en tablas de millones de filas.
- **Clustered Index (izquierda):** las hojas SON las páginas de datos, con todas las columnas de la fila. Las páginas de una misma hoja se enlazan en una lista doblemente enlazada, lo que permite recorrer rangos en orden (ORDER BY, BETWEEN) sin volver a la raíz.
- **Non-Clustered Index (derecha):** la hoja guarda las columnas de la clave del índice (más las INCLUDE) y un localizador de fila: la clave del Clustered Index o, en un Heap, el RID (File:Page:Slot). Por eso hace falta un Key Lookup/RID Lookup para recuperar el resto de columnas.
- **La profundidad del árbol crece logarítmicamente:** con claves de 8 bytes caben unos 600 punteros por página intermedia; con 3 niveles se direccionan cientos de millones de filas. Claves anchas reducen el fan-out y aumentan la profundidad.
- Se puede inspeccionar con sys.dm_db_index_physical_stats (columna index_depth) y con DBCC IND/DBCC PAGE (no documentados oficialmente) para mostrar páginas reales en clase.
- **Impacto de escritura:** cada INSERT/UPDATE/DELETE debe mantener el Clustered Index y todos los NCI afectados; de ahí que "más índices" no sea gratis.

PUNTOS DE INTERACCIÓN Y PREGUNTAS PARA LA CLASE

- ¿Cuántas lecturas lógicas esperáis para un Seek por clave en una tabla de 100 millones de filas con clave INT? ¿Y para un Scan completo?
- Si el NCI guarda la clave clustered como puntero, ¿qué pasa con todos los NCI cuando cambiamos el valor de la clave clustered de una fila?

DIAPOSITIVA 46

ÍNDICES

Non-Clustered, Key Lookup y covering

3 · Optimización y Alta Disponibilidad

CONTENIDO CLAVE EN PANTALLA (PROYECCIÓN)

• T-SQL · CREAR ÍNDICES

SQL

```
-- Consulta frecuente
SELECT PedidoId, FechaPedido, Total
FROM dbo.Pedido
WHERE ClienteId = 4217;

-- 1) Sólo clave: Seek + Key Lookup por fila
CREATE NONCLUSTERED INDEX IX_Pedido_Cliente
ON dbo.Pedido (ClienteId);

-- 2) Covering: sin Key Lookup
CREATE NONCLUSTERED INDEX IX_Pedido_Cliente_Cov
ON dbo.Pedido (ClienteId)
INCLUDE (FechaPedido, Total);
```

- Qué ocurre por dentro
- La hoja del NCI guarda clave + puntero a la fila
- Key Lookup: 1 lectura aleatoria por fila
- INCLUDE: columnas sólo en la hoja, sin ordenar
- Más índices = más coste en INSERT/UPDATE/DELETE
- Orden de la clave: igualdad primero, rango después, y columnas de lectura en INCLUDE.

OBJETIVO PEDAGÓGICO

Explicar el Key Lookup como el coste oculto de los Non-Clustered Index y enseñar cómo un índice covering con INCLUDE lo elimina.

GUIÓN DEL DOCENTE Y EXPLICACIÓN TÉCNICA

- **Un Non-Clustered Index (NCI) es una estructura adicional que apunta a las filas. Si la consulta pide columnas que no están en el índice, el motor hace un Key Lookup (en Heap, RID Lookup):** por cada fila encontrada en el NCI hace una lectura aleatoria al Clustered Index.
- El plan típico es Index Seek (NCI) + Nested Loops + Key Lookup (Clustered). Con pocas filas es barato; con miles de filas el coste se multiplica y el optimizador puede preferir un Clustered Index Scan directamente (tipping point, normalmente cuando el Lookup afectaría a ~25-33 % de las páginas de la tabla).
- **Un índice covering contiene todas las columnas que necesita la consulta, de modo que se resuelve sólo con el NCI. Con INCLUDE las columnas extra se guardan únicamente en el nivel hoja:** no forman parte de la clave, no ordenan y no cuentan para el límite de clave (1700 bytes en NCI desde SQL Server 2016, 900 en clustered).
- **Orden de columnas de la clave:** primero las de igualdad (=), luego la de rango (>, <, BETWEEN) y al final las usadas sólo para ordenar. La selectividad importa, pero el patrón de acceso manda.
- **Advertencia:** cada columna INCLUDE aumenta el tamaño del índice y el coste de cada escritura. No se debe "cubrir" cualquier consulta; se cubre la consulta crítica y frecuente.
- WITH (DROP_EXISTING = ON) permite redefinir un índice en una sola operación, evitando reconstruir los NCI dos veces; ONLINE = ON requiere Enterprise Edition.

PUNTOS DE INTERACCIÓN Y PREGUNTAS PARA LA CLASE

- ¿Por qué un SELECT * rara vez puede ser cubierto por un índice? ¿Qué alternativa propondrías al desarrollador?
- Vemos un Key Lookup con 80 % del coste del plan: ¿añadimos INCLUDE siempre? ¿Qué comprobarías antes (nº de filas, columnas, frecuencia)?

DIAPOSITIVA 47

ÍNDICES

Filtered, columnstore y missing indexes

3 · Optimización y Alta Disponibilidad

 CONTENIDO CLAVE EN PANTALLA (PROYECCIÓN)

- T-SQL · FILTERED Y COLUMNSTORE

```
-- Filtered: sólo pedidos abiertos
CREATE NONCLUSTERED INDEX IX_Pedido_Abierto
ON dbo.Pedido (FechaPedido)
INCLUDE (Total)
WHERE Estado = 1;
-- Columnstore (analítica / DW)
CREATE NONCLUSTERED COLUMNSTORE INDEX
NCCI_Pedido ON dbo.Pedido (ClienteId, Total);
```

SQL

- Cuándo usar cada uno
- Filtered: subconjuntos pequeños, predicado fijo
- Columnstore: agregaciones sobre millones de filas
- Filtered + parámetros: puede ignorarse sin RECOMPILE
- T-SQL · MISSING INDEX DMVS

```
SELECT TOP (5)
d.statement AS tabla,
d.equality_columns, d.inequality_columns,
d.included_columns,
s.user_seeks, s.avg_user_impact
FROM sys.dm_db_missing_index_details AS d
JOIN sys.dm_db_missing_index_groups AS g
ON g.index_handle = d.index_handle
JOIN sys.dm_db_missing_index_group_stats AS s
ON s.group_handle = g.index_group_handle
ORDER BY s.user_seeks * s.avg_user_impact DESC;
```

SQL

- Con cautela: son sugerencias, no órdenes. Pueden duplicar índices y se reinician con el servicio. Valídalas con el plan real.

 OBJETIVO PEDAGÓGICO

Conocer dos tipos especiales de índice (filtered y columnstore) y aprender a usar con cautela las sugerencias de índices que expone el propio motor.

 GUION DEL DOCENTE Y EXPLICACIÓN TÉCNICA

- Filtered Index:** es un NCI con cláusula WHERE que indexa sólo un subconjunto de filas (por ejemplo, pedidos abiertos, Estado = 1). Es más pequeño, más barato de mantener y sus estadísticas son más precisas para ese subconjunto.
- Limitaciones:** el predicado debe ser determinista y simple; el optimizador sólo lo usa si la consulta garantiza el predicado. Con consultas parametrizadas (WHERE Estado = @e) puede no usarse sin OPTION (RECOMPILE). Requiere ciertas opciones SET (QUOTED_IDENTIFIER, ANSI_NULLS ON) en la sesión.
- Columnstore (sólo mención en este curso):** almacena por columnas, comprimido y procesado en modo batch. Es ideal para analítica y Data Warehouse (agregaciones sobre millones de filas); no para búsquedas puntuales OLTP. Existe clustered columnstore y nonclustered columnstore; desde 2016 se puede combinar con OLTP (operational analytics).
- Missing Index DMVs (sys.dm_db_missing_index_details, _groups, _group_stats): el optimizador registra los índices que le habrían ayudado. Son sugerencias, no órdenes: ignoran el orden óptimo de las columnas, pueden proponer índices casi duplicados, no consideran el coste de escritura y se pierden al reiniciar el servicio.
- Procedimiento sano:** usar las DMVs como punto de partida, validar con el plan real de la consulta, consolidar con índices existentes (sys.dm_db_index_usage_stats muestra user_seeks/scans/lookups frente a user_updates) y medir antes/después.
- Un índice con muchos user_updates y cero lecturas es candidato a eliminarse, pero hay que observar un ciclo de negocio completo (cierre mensual, informes anuales) y recordar que estas cifras se reinician al reiniciar la instancia.

 PUNTOS DE INTERACCIÓN Y PREGUNTAS PARA LA CLASE

- ¿Qué casos de vuestro negocio tienen un subconjunto "caliente" de filas (pedidos abiertos, usuarios activos) donde un filtered index aportaría valor?
- Si el servidor lleva 3 días encendido, ¿es fiable concluir que un índice no se usa? ¿Qué información adicional necesitaríais?

DIAPOSITIVA 48

MANTENIMIENTO

Fragmentación, Page Splits y fillfactor

3 · Optimización y Alta Disponibilidad

 CONTENIDO CLAVE EN PANTALLA (PROYECCIÓN)

- T-SQL · MEDIR FRAGMENTACIÓN

```
SELECT OBJECT_NAME(ips.object_id) AS Tabla,
       i.name AS Indice,
       ips.avg_fragmentation_in_percent AS Frag,
       ips.page_count
FROM sys.dm_db_index_physical_stats
     (DB_ID(), NULL, NULL, NULL, N'LIMITED') AS ips
JOIN sys.indexes AS i
  ON i.object_id = ips.object_id
 AND i.index_id = ips.index_id
WHERE ips.page_count > 1000
 AND ips.avg_fragmentation_in_percent > 5
ORDER BY Frag DESC;
```

SQL

- T-SQL · REORGANIZE / REBUILD

```
ALTER INDEX IX_Pedido_Cliente ON dbo.Pedido REORGANIZE;
ALTER INDEX IX_Pedido_Cliente ON dbo.Pedido
REBUILD WITH (FILLFACTOR = 90);
```

SQL

- Page Split
- Página llena + inserción en medio: se parte en dos
- Deja páginas a medias y genera log extra
- Claves aleatorias (GUID) lo agravan

Tabla Resumen en Diapositiva:

[object Object]undefined

 OBJETIVO PEDAGÓGICO

Entender qué es la fragmentación de índices, cómo se mide con `sys.dm_db_index_physical_stats` y cómo decidir entre `REORGANIZE` y `REBUILD` apoyándose en el fillfactor.

 GUION DEL DOCENTE Y EXPLICACIÓN TÉCNICA

- Fragmentación lógica (`avg_fragmentation_in_percent`): porcentaje de páginas hoja cuyo orden físico no coincide con el orden lógico de la clave. Afecta sobre todo a las lecturas de rango (read-ahead menos eficiente). Fragmentación interna: páginas poco llenas (`avg_page_space_used_in_percent` baja), que desperdician espacio y memoria del Buffer Pool.
- Page Split:** cuando se inserta o se actualiza una fila en una página hoja llena, el motor reserva una página nueva, mueve ~50 % de las filas y enlaza la nueva página. Es costoso (log, bloqueos) y deja ambas páginas a medio llenar. Claves aleatorias (GUID) o `UPDATES` que hacen crecer filas los multiplican.
- Se mide con `sys.dm_db_index_physical_stats`; el modo `LIMITED` es rápido (sólo niveles superiores) y suficiente para `avg_fragmentation_in_percent`; `DETAILED` recorre todo y es caro: no usarlo en horario de producción en tablas grandes.
- Regla de partida de Microsoft:** ignorar índices de menos de ~1000 páginas; entre 5 y 30 % `ALTER INDEX ... REORGANIZE` (siempre online, compacta hojas, interrumpible); por encima de 30 % `ALTER INDEX ... REBUILD` (recrea el índice, actualiza estadísticas con `FULLSCAN`, puede ser `ONLINE` sólo en Enterprise). Son umbrales de partida, no leyes.
- `FILLFACTOR` deja un porcentaje libre en cada página hoja al reconstruir (por ejemplo 90) para absorber inserciones sin Page Split. Con claves crecientes lo habitual es 100; bájalo sólo en índices con inserciones aleatorias, y comprueba que reduce Page Splits (contador Page Splits/sec, `sys.dm_db_index_operational_stats`: `leaf_allocation_count`).
- En almacenamiento SSD/flash la fragmentación lógica penaliza menos; la densidad de página y las estadísticas siguen siendo relevantes. Un `REBUILD` masivo diario puede ser peor (log, bloqueos, réplicas) que el problema que intenta resolver.

 PUNTOS DE INTERACCIÓN Y PREGUNTAS PARA LA CLASE

- Un índice de 300 páginas con 60 % de fragmentación: ¿lo reconstruiríais? ¿Por qué?
- ¿Qué efecto tendría un `FILLFACTOR = 70` en una tabla de solo lectura? ¿Y en una con inserciones aleatorias?
- ¿Cómo comprobaríais, tras cambiar el fillfactor, que de verdad disminuyen los Page Splits?

DIAPOSITIVA 49

OPTIMIZADOR

Estadísticas y cardinality estimator

3 · Optimización y Alta Disponibilidad

 CONTENIDO CLAVE EN PANTALLA (PROYECCIÓN)

• T-SQL · INSPECCIONAR ESTADÍSTICAS

```
-- Opciones a nivel de base de datos
SELECT name, is_auto_create_stats_on,
       is_auto_update_stats_on,
       is_auto_update_stats_async_on
FROM sys.databases WHERE name = N'Ventas';

-- Encabezado, densidad e histograma
DBCC SHOW_STATISTICS (N'dbo.Pedido',
                     N'IX_Pedido_Cliente');

-- Actualización manual
UPDATE STATISTICS dbo.Pedido IX_Pedido_Cliente
WITH FULLSCAN;
```

SQL

Auto-create / Auto-update

SQL

- Se refrescan tras muchos cambios: umbral dinámico $\sqrt{(1000 \cdot \text{filas})}$ con compat. 130+.
- Histograma
- Hasta 200 pasos sobre la 1.ª columna de la clave: filas estimadas por valor.
- Cardinality Estimator
- El CE nuevo (compat. 120+) cambia estimaciones; compara con el Legacy CE.

 OBJETIVO PEDAGÓGICO

Comprender que el optimizador elige el plan a partir de estimaciones de cardinalidad basadas en estadísticas, y saber inspeccionarlas, mantenerlas y reconocer cuándo fallan.

 GUION DEL DOCENTE Y EXPLICACIÓN TÉCNICA

- Las estadísticas describen la distribución de valores de una o varias columnas. El Query Optimizer las usa para estimar cuántas filas devolverá cada operador (cardinalidad) y, con ello, elegir Seek vs Scan, tipo de JOIN y concesión de memoria. Estadísticas malas = planes malos.
- Cada objeto de estadísticas tiene un encabezado (filas, filas muestreadas, última actualización), un vector de densidad y un histograma de hasta 200 pasos sobre la PRIMERA columna de la clave. Se inspecciona con DBCC SHOW_STATISTICS; para ver la fecha y el nº de modificaciones, sys.dm_db_stats_properties.
- AUTO_CREATE_STATISTICS crea estadísticas de columna (WA_Sys...) cuando una consulta filtra por una columna sin ellas; AUTO_UPDATE_STATISTICS las refresca cuando el número de cambios supera un umbral: clásico 500 + 20 % de las filas; con nivel de compatibilidad 130 o superior, umbral dinámico aproximado $\sqrt{(1000 \times \text{filas})}$, mucho más sensible en tablas grandes. AUTO_UPDATE_STATISTICS_ASYNC evita bloquear la consulta que dispara la actualización.
- Las estadísticas del índice se actualizan con FULLSCAN al hacer REBUILD, no con REORGANIZE; el muestreo por defecto de UPDATE STATISTICS puede ser insuficiente en datos muy sesgados: considerar WITH FULLSCAN o PERSIST_SAMPLE_PERCENT (2016 SP1+/2019).
- **Cardinality Estimator (CE):** con nivel de compatibilidad 120+ se usa el CE nuevo (2014), que cambió supuestos de correlación y contención entre predicados. Si una migración provoca regresiones, se puede comparar con LEGACY_CARDINALITY_ESTIMATION (database scoped configuration) o el hint USE HINT.
- **Síntoma clásico:** en el plan real, Estimated Number of Rows muy distinto de Actual Number of Rows. Causas habituales: estadísticas obsoletas, parameter sniffing, variables locales, tablas variable, columnas correlacionadas.

 PUNTOS DE INTERACCIÓN Y PREGUNTAS PARA LA CLASE

- ¿Por qué un REBUILD de índice "arregla" a veces un plan lento aunque la fragmentación fuera baja?
- Una tabla de 50 millones de filas con auto-update clásico necesitaría 10 millones de cambios para actualizar estadísticas: ¿qué consecuencias tiene y cómo lo mitiga el umbral dinámico?

DIAPOSITIVA 50

PLANES DE EJECUCIÓN

Planes: Scan, Seek y Key Lookup

3 · Optimización y Alta Disponibilidad

 CONTENIDO CLAVE EN PANTALLA (PROYECCIÓN)

- Scan
- Lee todas las páginas del objeto
- Coste proporcional al tamaño (E/S)
- Correcto en tablas pequeñas o lecturas masivas
- Sospechoso: predicado no SARGable
- Seek
- Navega el B-Tree hasta la clave
- Pocas lecturas lógicas
- Ideal con predicados selectivos
- Ojo: devolver millones de filas no es barato
- Key Lookup
- Un acceso al clustered por fila del NCI
- Lecturas aleatorias que se multiplican
- Plan típico: Nested Loops + Lookup
- Solución: INCLUDE o reescribir la consulta
- Un Scan no es siempre un problema: hay que cruzarlo con el nº de filas, el tamaño de la tabla y las lecturas lógicas reales antes de decidir crear un índice.

 OBJETIVO PEDAGÓGICO

Reconocer los tres operadores de acceso a datos más frecuentes en un plan de ejecución y saber cuándo son normales y cuándo son una señal de alarma.

 GUION DEL DOCENTE Y EXPLICACIÓN TÉCNICA

- **Table Scan (Heap) / Clustered Index Scan / Index Scan:** leen todas las páginas del objeto. Coste proporcional al número de páginas. No es malo por definición: en tablas pequeñas o cuando la consulta necesita la mayoría de las filas, un Scan secuencial con read-ahead es lo más eficiente.
- **Index Seek:** navega el B-Tree desde la raíz hasta el primer valor que cumple el predicado y lee sólo el rango necesario. Requiere un predicado SARGable (Search ARGument ABLE): columna sola a un lado, sin funciones ni conversiones implícitas (WHERE YEAR(Fecha) = 2024 anula el Seek; mejor un rango Fecha >= ... AND Fecha < ...).
- Un Seek también puede ser caro si devuelve millones de filas o se ejecuta muchas veces dentro de un Nested Loops (mirar Number of Executions y Actual Rows en las propiedades del operador).
- **Key Lookup (Clustered) / RID Lookup (Heap):** obtiene las columnas que el NCI no contiene. Va acompañado de Nested Loops. Si el nº de ejecuciones es alto, es el candidato nº 1 para un índice covering (INCLUDE).
- **Coste CPU y E/S:** cada operador muestra Estimated I/O Cost y Estimated CPU Cost; el porcentaje del plan es una estimación relativa del optimizador, calculada con un modelo de hardware de referencia, y NO equivale a tiempo real. Para tiempo real, usar el plan real y SET STATISTICS TIME.
- **Orden de lectura:** de derecha a izquierda y de arriba abajo (el flujo de datos va hacia la izquierda); el grosor de las flechas indica el volumen de filas.

 PUNTOS DE INTERACCIÓN Y PREGUNTAS PARA LA CLASE

- Veis un Clustered Index Scan sobre una tabla de 200 filas: ¿hay que arreglarlo? ¿Y si la tabla tiene 200 millones?
- ¿Por qué WHERE CONVERT(VARCHAR(10), FechaPedido, 120) = '2024-03-15' impide el Index Seek? ¿Cómo lo reescribiríais?

DIAPOSITIVA 51

PLANES DE EJECUCIÓN

Leer un plan: coste, warnings y estimados

3 · Optimización y Alta Disponibilidad

CONTENIDO CLAVE EN PANTALLA (PROYECCIÓN)

- T-SQL · MEDIR E/S Y TIEMPO

```
SET STATISTICS IO ON;
SET STATISTICS TIME ON;
```

```
SELECT PedidoId, Total
FROM dbo.Pedido
WHERE ClienteId = 4217;
```

```
-- Messages: Table 'Pedido'. Scan count 1,
-- logical reads 3, physical reads 0
-- CPU time = 0 ms, elapsed time = 1 ms
```

SQL

- Métrica clave: las lecturas lógicas son estables; el tiempo varía con la caché y la carga.
- Qué mirar en el plan
- Estimated vs Actual: gran diferencia = estadísticas obsoletas o parameter sniffing
- Warnings: spill a tempdb, conversión implícita, falta de estadísticas
- Coste %: estimación relativa, no tiempo real
- Query Store: planes e historial para detectar regresiones (2016+)

OBJETIVO PEDAGÓGICO

Aprender a obtener y leer un plan de ejecución real, medir E/S y tiempo con STATISTICS IO/TIME y detectar las desviaciones y avisos que indican un problema.

GUIÓN DEL DOCENTE Y EXPLICACIÓN TÉCNICA

- Plan estimado (Ctrl+L): se genera sin ejecutar la consulta; sólo contiene estimaciones. Plan real (Ctrl+M antes de ejecutar): incluye métricas reales (Actual Rows, Actual Executions, tiempos) y avisos de ejecución como spills.
- SET STATISTICS IO ON muestra por tabla:** Scan count, logical reads (páginas leídas de memoria), physical reads y read-ahead reads. Las lecturas lógicas son la métrica más estable para comparar versiones de una consulta, porque no dependen de la caché ni de la carga del servidor.
- SET STATISTICS TIME ON informa de CPU time y elapsed time; el elapsed incluye esperas (bloqueos, E/S). Comparar siempre con la caché caliente y repetir varias ejecuciones; evitar DBCC DROPCLEANBUFFERS en producción.
- Estimated vs Actual Rows:** una diferencia de un orden de magnitud o más es la principal pista de estadísticas obsoletas, parameter sniffing o de un predicado que el CE no sabe estimar. Se ven en las propiedades de cada operador (F4).
- Warnings (triángulo amarillo):** Sort/Hash Spill a tempdb por concesión de memoria insuficiente, conversión implícita (CONVERT_IMPLICIT) que anula Seeks, columnas sin estadísticas, Missing Index. Un spill indica memory grant mal estimado.
- Query Store (SQL Server 2016+; mención): guarda por base de datos las consultas, sus planes y métricas históricas; permite detectar regresiones de plan y forzar un plan anterior. Se activa con ALTER DATABASE ... SET QUERY_STORE = ON y está activado por defecto en bases nuevas desde SQL Server 2022.

PUNTOS DE INTERACCIÓN Y PREGUNTAS PARA LA CLASE

- ¿Por qué elegimos lecturas lógicas y no el tiempo transcurrido para comparar dos versiones de una consulta?
- El plan muestra Estimated Rows = 1 y Actual Rows = 900.000: ¿qué hipótesis plantearíais y qué comprobaríais primero?

DIAPOSITIVA 52

MONITORIZACIÓN

DMVs: consultas costosas y sesiones activas

3 · Optimización y Alta Disponibilidad

 CONTENIDO CLAVE EN PANTALLA (PROYECCIÓN)

- T-SQL · TOP CONSULTAS POR CPU

```
SELECT TOP (5)
    qs.execution_count,
    qs.total_worker_time/qs.execution_count AS cpu_us,
    qs.total_logical_reads,
    LEFT(st.text, 100) AS batch_text
FROM sys.dm_exec_query_stats AS qs
CROSS APPLY sys.dm_exec_sql_text(qs.sql_handle) st
-- Plan: sys.dm_exec_query_plan(qs.plan_handle)
ORDER BY qs.total_worker_time DESC;
```

SQL

- T-SQL · PETICIONES ACTIVAS

```
SELECT r.session_id, r.status, r.command,
    r.wait_type, r.wait_time,
    r.blocking_session_id,
    r.cpu_time, r.logical_reads,
    DB_NAME(r.database_id) AS bd,
    LEFT(t.text, 100) AS sql_text
FROM sys.dm_exec_requests AS r
CROSS APPLY sys.dm_exec_sql_text(r.sql_handle) t
WHERE r.session_id <> @@SPID;
```

SQL

- Datos volátiles: las DMVs acumulan desde el último reinicio y query_stats sólo ve planes en caché. Necesitas VIEW SERVER STATE (VIEW SERVER PERFORMANCE STATE en 2022).

 OBJETIVO PEDAGÓGICO

Usar las DMVs sys.dm_exec_query_stats y sys.dm_exec_requests junto con sys.dm_exec_sql_text/query_plan para localizar las consultas más costosas y ver qué está ejecutándose ahora mismo.

 GUION DEL DOCENTE Y EXPLICACIÓN TÉCNICA

- Las Dynamic Management Views exponen el estado interno del motor sin instalar nada. Requieren el permiso VIEW SERVER STATE (VIEW SERVER PERFORMANCE STATE en SQL Server 2022) y sus datos son volátiles: se reinician al reiniciar la instancia.
- sys.dm_exec_query_stats acumula, por sentencia en la caché de planes, execution_count, total_worker_time (CPU, µs), total_elapsed_time, total_logical_reads/writes y últimos valores. Se une con sys.dm_exec_sql_text(sql_handle) para obtener el texto y con sys.dm_exec_query_plan(plan_handle) para el plan XML.
- Importante:** sólo ve consultas cuyo plan sigue en caché; los planes expulsados por memoria o recompilados desaparecen, y las consultas con OPTION (RECOMPILE) no aparecen. Para histórico fiable, Query Store.
- Ordenar por total_worker_time identifica lo que más CPU consume en conjunto; por total_logical_reads, lo que más memoria/E-S toca; por promedio (dividir por execution_count) las consultas individualmente lentas. Una consulta de 5 ms ejecutada un millón de veces puede pesar más que una de 5 s ejecutada una vez.
- sys.dm_exec_requests muestra las peticiones activas ahora: status, command, wait_type, wait_time, blocking_session_id, cpu_time, logical_reads, database_id y sql_handle. Es el primer sitio al que mirar cuando "va lento ahora".
- Buenas prácticas:** filtrar session_id > 50 o unir con sys.dm_exec_sessions (is_user_process = 1), excluir @@SPID y limitar columnas de texto (LEFT/SUBSTRING) para no devolver lotes enormes a SSMS.

 PUNTOS DE INTERACCIÓN Y PREGUNTAS PARA LA CLASE

- ¿Por qué ordenar sólo por duración media puede ocultar la consulta que más daño hace al servidor?
- Reiniciamos el servicio ayer y hoy sys.dm_exec_query_stats está casi vacío: ¿qué conclusiones podemos (y no) extraer?

DIAPOSITIVA 53

MONITORIZACIÓN

DMVs: esperas y bloqueos

3 · Optimización y Alta Disponibilidad

 CONTENIDO CLAVE EN PANTALLA (PROYECCIÓN)

- T-SQL · TOP WAITS ACUMULADOS

```
SELECT TOP (10)
    wait_type,
    waiting_tasks_count,
    wait_time_ms,
    signal_wait_time_ms,
    CAST(100.0 * wait_time_ms /
        SUM(wait_time_ms) OVER() AS DECIMAL(5,2)) AS pct
FROM sys.dm_os_wait_stats
WHERE wait_type NOT LIKE N'SLEEP%'
    AND wait_type NOT IN (N'LAZYWRITER_SLEEP',
        N'XE_TIMER_EVENT')
ORDER BY wait_time_ms DESC;
```

SQL

- Consejo: toma dos snapshots y calcula la diferencia.
- T-SQL · TAREAS EN ESPERA AHORA

```
SELECT wt.session_id, wt.wait_type,
    wt.wait_duration_ms,
    wt.blocking_session_id,
    wt.resource_description
FROM sys.dm_os_waiting_tasks AS wt
WHERE wt.session_id > 50
ORDER BY wt.wait_duration_ms DESC;
```

SQL

Tabla Resumen en Diapositiva:

[object Object]undefined

 OBJETIVO PEDAGÓGICO

Aplicar el método de Wait Statistics: identificar los tipos de espera dominantes con sys.dm_os_wait_stats y localizar bloqueos con sys.dm_os_waiting_tasks.

 GUION DEL DOCENTE Y EXPLICACIÓN TÉCNICA

- Cada hilo (worker) de SQL Server pasa por estados RUNNING, RUNNABLE y SUSPENDED. Cuando no puede avanzar (espera un bloqueo, una E/S, memoria...) registra una espera. Analizar dónde espera el servidor indica cuál es el cuello de botella dominante.
- sys.dm_os_wait_stats es acumulativa desde el inicio de la instancia (o desde DBCC SQLPERF('sys.dm_os_wait_stats', CLEAR)): wait_time_ms incluye signal_wait_time_ms (tiempo en cola esperando CPU después de que el recurso esté listo). Si el signal wait supera ~20–25 % del total hay presión de CPU.
- Hay que filtrar las esperas benignas (SLEEP_*, LAZYWRITER_SLEEP, XE_TIMER_EVENT, BROKER_*, WAITFOR...). La lista real es larga; los scripts de referencia de la comunidad (por ejemplo, los de Paul Randal) mantienen un listado actualizado. La consulta de la diapositiva es una simplificación didáctica.
- Interpretación rápida:** PAGEIOLATCH_SH/EX = lecturas de disco lentas o falta de memoria; WRITELOG = latencia del disco de log; LCK_M_* = bloqueos entre sesiones; SOS_SCHEDULER_YIELD = presión de CPU; CXPACKET/CXCONSUMER = paralelismo (no siempre un problema por sí solo); ASYNC_NETWORK_IO = el cliente tarda en consumir resultados.
- sys.dm_os_waiting_tasks es una vista instantánea de las tareas que esperan AHORA: session_id, wait_type, wait_duration_ms, blocking_session_id y resource_description (por ejemplo, el recurso bloqueado). Permite reconstruir cadenas de bloqueo y localizar el head blocker (el que bloquea y no está bloqueado).
- Método sano:** tomar dos snapshots de wait_stats separados por unos minutos y calcular la diferencia; los valores acumulados desde hace semanas diluyen el problema actual.

 PUNTOS DE INTERACCIÓN Y PREGUNTAS PARA LA CLASE

- El 60 % de las esperas es PAGEIOLATCH_SH: ¿qué dos líneas de actuación distintas se os ocurren (E/S y memoria/consultas)?
- ¿Por qué una consulta puede tardar 30 s con CPU time de 200 ms? ¿Dónde lo veríais?

DIAPOSITIVA 54

HA / DR

RTO y RPO: tiempo y datos en juego

3 · Optimización y Alta Disponibilidad

 CONTENIDO CLAVE EN PANTALLA (PROYECCIÓN)

- RPO: datos que puedes perder
- RTO: tiempo hasta restablecer
- Último punto recuperable
- (commit replicado o backup)
- Fallo
- (caída de nodo o sitio)
- Servicio restablecido
- (aplicación operativa)
- RPO $\approx$ 0
- FCI o AG síncrono: ningún commit confirmado se pierde
- RTO: segundos
- Failover automático con FCI o AG síncrono y listener
- RPO: minutos

Log Shipping: pérdida = intervalo de backup del log

SQL

 OBJETIVO PEDAGÓGICO

Definir RTO y RPO como los dos requisitos de negocio que determinan qué tecnología de HA/DR elegir, y distinguir alta disponibilidad de recuperación ante desastres.

 GUION DEL DOCENTE Y EXPLICACIÓN TÉCNICA

- **RPO (Recovery Point Objective):** cuántos datos, medidos en tiempo, puede perder el negocio. RPO = 15 min significa que tras un fallo se aceptan hasta 15 minutos de transacciones perdidas. RPO $\approx$ 0 exige que cada commit esté ya en otro lugar antes de confirmarse (replicación síncrona).
- **RTO (Recovery Time Objective):** cuánto tiempo puede estar caído el servicio hasta restablecerse (incluye detección, decisión, failover y recuperación de la BD). RTO = 1 min exige failover automático; RTO = 4 h permite restaurar desde backup.
- Alta Disponibilidad (HA) protege frente a fallos locales (nodo, instancia, disco) con RTO/RPO bajos, normalmente en el mismo CPD. Disaster Recovery (DR) protege frente a la pérdida de un sitio entero, normalmente con replicación asíncrona a otra ubicación; el RPO suele ser mayor que cero.
- Cuanto más cercanos a cero son RTO y RPO, mayor es el coste (licencias Enterprise, hardware duplicado, red de baja latencia, complejidad operativa). Los requisitos los define el negocio, no el DBA; el DBA los traduce a arquitectura.
- **La replicación y HA NO sustituyen a los backups:** un DROP TABLE accidental o la corrupción lógica se replican al secundario. Los backups (Módulo 4) siguen siendo la defensa frente a errores humanos y ransomware.
- **Para un ejercicio rápido:** dar tres sistemas (tienda online, ERP interno, informes mensuales) y pedir que propongan RPO y RTO razonables y su justificación económica.

 PUNTOS DE INTERACCIÓN Y PREGUNTAS PARA LA CLASE

- ¿Qué RPO y RTO fijaríais para el sistema de pedidos de una tienda online en campaña de Black Friday y para un datamart de informes mensuales?
- Si el RPO es 0 y el RTO 30 s, ¿qué opciones tecnológicas siguen disponibles? ¿Y si además hay que cubrir la pérdida de todo el CPD?

DIAPOSITIVA 55

HA / DR

Comparativa de tecnologías HA/DR

3 · Optimización y Alta Disponibilidad

 CONTENIDO CLAVE EN PANTALLA (PROYECCIÓN)

- Elige por RTO/RPO, edición y presupuesto: Standard ofrece Basic AG, FCI de 2 nodos y Log Shipping; Enterprise añade AG completos.

Tabla Resumen en Diapositiva:

[object Object]undefined

 OBJETIVO PEDAGÓGICO

Comparar Log Shipping, Replicación, Failover Cluster Instances y Always On Availability Groups según alcance, failover, pérdida de datos y requisitos, para saber elegir cada una.

 GUION DEL DOCENTE Y EXPLICACIÓN TÉCNICA

- **Log Shipping:** automatiza el backup del Transaction Log de la primaria, su copia y su restauración en una o varias secundarias (NORECOVERY o STANDBY de sólo lectura). Es sencillo, barato, funciona en Standard y tiene RPO igual al intervalo de backup (minutos). El failover es manual: hay que restaurar la cola de log, recuperar la BD y redirigir las aplicaciones.
- **Replicación (Snapshot, Transactional, Merge):** distribuye datos a nivel de tablas/artículos, no de base de datos completa. Sirve para informes descargados, consolidación y escenarios de datos distribuidos; no es una solución de HA/DR completa (no replica logins, jobs, ni el esquema entero).
- **Failover Cluster Instance (FCI):** protege la INSTANCIA entera sobre un Windows Server Failover Cluster (WSFC) con almacenamiento compartido (SAN, S2D, SMB). Una sola copia de los datos: si el almacenamiento falla, todo falla. Failover automático (segundos a pocos minutos, incluida la recuperación de la BD), RPO $\approx$ 0. Disponible en Standard con 2 nodos.
- **Always On Availability Groups (AG):** protege un conjunto de bases de datos replicando el log a hasta 8 réplicas secundarias (Enterprise), síncronas (RPO 0, failover automático) o asíncronas (RPO > 0, failover manual forzado). No necesita almacenamiento compartido; las réplicas pueden ser legibles y usarse para backups o informes (Enterprise).
- **Un AG no replica objetos a nivel de instancia (logins, SQL Agent jobs, linked servers):** hay que sincronizarlos aparte. FCI y AG pueden combinarse (FCI como réplicas de un AG).
- **Criterios de elección:** RTO/RPO exigidos, edición y licencias, topología de red y almacenamiento, capacidad del equipo para operar un WSFC y presupuesto. No hay una opción "mejor" en abstracto.

 PUNTOS DE INTERACCIÓN Y PREGUNTAS PARA LA CLASE

- Una pyme con SQL Server Standard, un solo CPD y RTO de 30 min: ¿qué tecnología propondrías y qué limitaciones explicarías a la dirección?
- ¿Por qué un FCI no protege frente a la corrupción del almacenamiento compartido? ¿Cómo lo complementarías?

Always On AG: topología y quorum

3 · Optimización y Alta Disponibilidad

CONTENIDO CLAVE EN PANTALLA (PROYECCIÓN)

- | | | |
|--|--|--|
| <ul style="list-style-type: none"> • Aplicaciones cliente • Listener (nombre virtual + IP) • Secundaria síncrona • SQLNODE2 · legible • Réplica primaria • SQLNODE1 · R/W • Secundaria asíncrona • SQLNODE3 · DR • Síncrona · RPO = 0 | <ul style="list-style-type: none"> • Failover automático • Envía el log de transacciones • Asíncrona · RPO > 0 • Failover manual • Windows Server Failover Cluster (WSFC) • Quorum: voto por nodo + Witness (File Share / Cloud / Disk) | <ul style="list-style-type: none"> • Basic AG (Standard) • 2 réplicas, 1 BD por AG, sin lectura en la secundaria. • Distributed AG • Une dos AG en clústeres distintos: DR entre sitios y migraciones. • Quorum y Testigo • Mayoría de votos contra split-brain; testigo en nodos pares. |
|--|--|--|

OBJETIVO PEDAGÓGICO

Interpretar la topología de un Always On Availability Group (réplicas, modos de disponibilidad, listener y quorum) y conocer las variantes Basic AG y Distributed AG.

GUION DEL DOCENTE Y EXPLICACIÓN TÉCNICA

- **Un AG se construye sobre un Windows Server Failover Cluster (WSFC) (o, en Linux, Pacemaker). Los nodos votan para decidir qué miembros siguen en servicio:** es el quorum. Con un número par de nodos se añade un testigo (File Share Witness, Cloud Witness en Azure o Disk Witness) para evitar el split-brain, es decir, dos nodos creyéndose primarios.
- **La réplica primaria acepta lecturas y escrituras y envía el log de transacciones (log blocks) a las secundarias. En modo de disponibilidad síncrono (synchronous-commit) el commit no se confirma al cliente hasta que el log está endurecido en la secundaria:** RPO 0 a cambio de latencia añadida; permite failover automático. En modo asíncrono el commit no espera: RPO > 0 y sólo failover manual forzado (con posible pérdida de datos).
- **El listener es un nombre de red virtual (VNN) y una o varias IP que siguen a la réplica primaria:** las aplicaciones se conectan a él y tras un failover se reconectan a la nueva primaria (usar MultiSubnetFailover=True en la cadena de conexión). Con ApplicationIntent=ReadOnly y read-only routing se puede dirigir lectura a secundarias legibles (Enterprise).
- **Basic Availability Groups (Standard Edition, desde 2016 SP1):** un único par de réplicas, una sola base de datos por AG, sin lectura en la secundaria ni backups en ella. Es el sustituto de Database Mirroring, obsoleto.
- **Distributed Availability Group (mención):** un AG que contiene otros dos AG en clústeres WSFC distintos; útil para DR entre sitios, migraciones sin downtime y cambios de versión/SO. Enterprise Edition.
- **Monitorización:** sys.dm_hadr_availability_replica_states, sys.dm_hadr_database_replica_states (log_send_queue_size, redo_queue_size, synchronization_state_desc) y el Dashboard de Always On en SSMS.
- **Advertencia de rendimiento:** una réplica síncrona con red lenta o disco de log saturado en la secundaria penaliza a TODAS las transacciones de la primaria (esperas HADR_SYNC_COMMIT).

PUNTOS DE INTERACCIÓN Y PREGUNTAS PARA LA CLASE

- ¿Qué ocurriría con las escrituras en la primaria si la réplica síncrona pierde conectividad? ¿Cómo lo gestiona el AG?
- Tenéis dos nodos en un mismo CPD y quorum de mayoría de nodos: ¿qué pasa si cae uno? ¿Cómo ayuda un Cloud Witness?

DIAPOSITIVA 57

LABORATORIO · LABORATORIO 3.1

3 · Optimización y Alta Disponibilidad

Lab 3.1 · Índices y medición de lecturas

CONTENIDO CLAVE EN PANTALLA (PROYECCIÓN)

- 1
- Preparar datos
- Genera 500 000 pedidos en la base de datos Ventas.
- 2
- Línea base
- Activa STATISTICS IO y plan real (Ctrl+M); anota las lecturas.
- 3
- Crear índices
- Crea el índice simple y luego la versión con INCLUDE.
- 4
- Comparar
- Repite la consulta y compara operador y lecturas lógicas.
- T-SQL · LÍNEA BASE

```
USE Ventas;
SET STATISTICS IO, TIME ON;
SELECT PedidoId, FechaPedido, Total
FROM dbo.Pedido
WHERE ClienteId = 4217
AND FechaPedido >= '20240101';
-- Anota: operador y logical reads
```

SQL

- T-SQL · CREAR Y MEJORAR EL ÍNDICE

```
CREATE NONCLUSTERED INDEX IX_Pedido_Cli_Fecha
ON dbo.Pedido (ClienteId, FechaPedido);
-- Lookup: añade INCLUDE y recrea
CREATE NONCLUSTERED INDEX IX_Pedido_Cli_Fecha
ON dbo.Pedido (ClienteId, FechaPedido)
INCLUDE (Total)
WITH (DROP_EXISTING = ON);
```

SQL

OBJETIVO PEDAGÓGICO

Crear índices Non-Clustered y covering sobre dbo.Pedido y cuantificar su efecto comparando operador, lecturas lógicas y plan antes y después.

GUION DEL DOCENTE Y EXPLICACIÓN TÉCNICA

- **Duración orientativa:** 35–40 minutos. Se trabaja en parejas con SSMS conectado a la instancia (Docker: localhost,1433).
- **Idea central del laboratorio:** el mismo SELECT pasa de un Clustered Index Scan, a un Index Seek con Key Lookup, y finalmente a un Index Seek puro cuando el índice es covering. Cada paso se mide con STATISTICS IO.
- **Recaltar la disciplina de medir:** anotar en una tabla (operador, logical reads, CPU/elapsed) para la versión sin índice, con índice de clave y con índice covering.
- **Aviso de rendimiento:** crear índices sobre 500.000 filas es rápido en el laboratorio, pero en producción es una operación pesada (bloqueos de esquema, log, E/S); se haría fuera de horario o con ONLINE = ON (Enterprise).

PUNTOS DE INTERACCIÓN Y PREGUNTAS PARA LA CLASE

- ¿Cuánto bajan las lecturas lógicas entre el Scan y el Seek? ¿Y entre Seek+Lookup y covering?
- ¿Qué coste adicional tiene ahora un INSERT en dbo.Pedido? ¿Cómo lo medirías?

INSTRUCCIONES DE LABORATORIO Y PAUTAS DE RESOLUCIÓN

- PASO 0 · Preparar datos (si no existen ya de módulos anteriores). Ejecutar en SSMS: CREATE DATABASE Ventas; (si no existe) y USE Ventas;
- CREATE TABLE dbo.Cliente (Clienteld INT IDENTITY(1,1) CONSTRAINT PK_Cliente PRIMARY KEY, Nombre NVARCHAR(100) NOT NULL, Activo BIT NOT NULL DEFAULT 1);
- INSERT dbo.Cliente (Nombre) SELECT TOP (10000) N'Cliente ' + CAST(ROW_NUMBER() OVER (ORDER BY (SELECT NULL)) AS NVARCHAR(10)) FROM sys.all_objects a CROSS JOIN sys.all_objects b;
- CREATE TABLE dbo.Pedido (Pedidold INT IDENTITY(1,1) CONSTRAINT PK_Pedido PRIMARY KEY, Clienteld INT NOT NULL, FechaPedido DATE NOT NULL, Total DECIMAL(12,2) NOT NULL, Estado TINYINT NOT NULL DEFAULT 1, Notas CHAR(200) NOT NULL DEFAULT '');
- INSERT dbo.Pedido (Clienteld, FechaPedido, Total) SELECT TOP (500000) ABS(CHECKSUM(NEWID())) % 10000 + 1, DATEADD(DAY, -(ABS(CHECKSUM(NEWID())) % 1460), CAST(GETDATE() AS DATE)), CAST(ABS(CHECKSUM(NEWID())) % 100000 / 100.0 AS DECIMAL(12,2)) FROM sys.all_objects a CROSS JOIN sys.all_objects b CROSS JOIN sys.all_objects c;
- PASO 1 · Línea base. SET STATISTICS IO, TIME ON; activar Ctrl+M (Incluir plan de ejecución real) y ejecutar: SELECT Pedidold, FechaPedido, Total FROM dbo.Pedido WHERE Clienteld = 4217 AND FechaPedido >= '20240101'; Anotar operador (Clustered Index Scan) y logical reads (varios miles).
- PASO 2 · Índice de clave. CREATE NONCLUSTERED INDEX IX_Pedido_Cli_Fecha ON dbo.Pedido (Clienteld, FechaPedido); Repetir la consulta: aparece Index Seek + Key Lookup (Nested Loops). Anotar lecturas.
- PASO 3 · Índice covering. CREATE NONCLUSTERED INDEX IX_Pedido_Cli_Fecha ON dbo.Pedido (Clienteld, FechaPedido) INCLUDE (Total) WITH (DROP_EXISTING = ON); Repetir: Index Seek sin Key Lookup, lecturas lógicas de un dígito.

- PASO 4 · Uso de índices. `SELECT OBJECT_NAME(s.object_id) AS Tabla, i.name, s.user_seeks, s.user_scans, s.user_lookups, s.user_updates FROM sys.dm_db_index_usage_stats s JOIN sys.indexes i ON i.object_id = s.object_id AND i.index_id = s.index_id WHERE s.database_id = DB_ID() AND s.object_id = OBJECT_ID('dbo.Pedido');` Comprobar que user_seeks aumenta con el nuevo índice.
- **TRAMPAS HABITUALES:** (a) olvidar activar el plan real y ver sólo el estimado; (b) ejecutar con la caché fría la primera vez y comparar con otra caliente: repetir 2–3 veces y usar logical reads; (c) la tabla tiene tan pocas filas que el optimizador sigue eligiendo Scan: comprobar que hay 500.000 filas (`SELECT COUNT(*)`); (d) error "Clienteld" inexistente: la tabla se creó en master porque faltó USE Ventas; (e) el generador CROSS JOIN no llega a 500.000 filas en una instancia vacía: añadir otro CROSS JOIN.
- **GUÍA DE RESOLUCIÓN:** si tras el paso 3 sigue habiendo Key Lookup, la consulta pide una columna que no está en el índice (por ejemplo Estado): añadirla en INCLUDE o quitarla del SELECT. Si el índice no se usa, revisar que el predicado sea SARGable y que los tipos coincidan (FechaPedido es DATE: usar literales en formato 'yyyymmdd').

DIAPOSITIVA 58

LABORATORIO · LABORATORIO 3.2

3 · Optimización y Alta Disponibilidad

Lab 3.2 · Analizar planes de ejecución

CONTENIDO CLAVE EN PANTALLA (PROYECCIÓN)

- T-SQL · TRES CONSULTAS, TRES PLANES

SQL

```
SET STATISTICS IO ON;
CREATE NONCLUSTERED INDEX IX_Pedido_Fecha
    ON dbo.Pedido (FechaPedido);
-- A) No SARGable: función sobre la columna
SELECT PedidoId FROM dbo.Pedido
WHERE YEAR(FechaPedido) = 2024;
-- B) SARGable: rango
SELECT PedidoId FROM dbo.Pedido
WHERE FechaPedido >= '20240101'
    AND FechaPedido < '20250101';
-- C) Seek + Key Lookup (pide Total)
SELECT PedidoId, Total FROM dbo.Pedido
WHERE FechaPedido = '20240315';
```

- Trampa: compara siempre planes reales y repite cada consulta 2–3 veces.
- Entrega: tabla con operador, lecturas lógicas y desviación estimado/real de A, B y C.

Tabla Resumen en Diapositiva:

[object Object]undefined

OBJETIVO PEDAGÓGICO

Practicar la lectura de planes reales: contrastar un predicado no SARGable con uno SARGable, localizar un Key Lookup y registrar lecturas y desviaciones de cardinalidad.

GUION DEL DOCENTE Y EXPLICACIÓN TÉCNICA

- Duración orientativa:** 30–35 minutos. Requiere haber completado el Lab 3.1 (tablas Ventas.dbo.Pedido con 500.000 filas).
- Objetivo:** tres consultas casi idénticas con planes muy distintos. A) YEAR(FechaPedido) = 2024 obliga a evaluar la función sobre cada fila: Index Scan; B) rango de fechas SARGable: Index Seek; C) consulta que pide Total sobre un índice que no lo incluye: Seek + Key Lookup.
- Pedir que cada pareja rellene una tabla con operador, logical reads, Estimated vs Actual Rows y si hay warnings, y que justifique cuál es la mejor versión y por qué.
- Cerrar con una puesta en común:** ¿qué cambio de código (no de índice) mejoró más? Insistir en que reescribir predicados suele ser más barato que crear índices.

PUNTOS DE INTERACCIÓN Y PREGUNTAS PARA LA CLASE

- ¿Por qué en la consulta A el optimizador no puede hacer un Seek aunque exista el índice sobre FechaPedido?
- ¿Qué indica que Estimated Rows y Actual Rows difieran mucho en la consulta C y qué haríais para corregirlo?

INSTRUCCIONES DE LABORATORIO Y PAUTAS DE RESOLUCIÓN

- PASO 1 · SET STATISTICS IO ON; activar Ctrl+M. Crear el índice: CREATE NONCLUSTERED INDEX IX_Pedido_Fecha ON dbo.Pedido (FechaPedido);
- PASO 2 · Consulta A (no SARGable): SELECT PedidoId FROM dbo.Pedido WHERE YEAR(FechaPedido) = 2024; Resultado esperado: Index Scan sobre IX_Pedido_Fecha, lecturas lógicas altas. Anotar operador, logical reads y Estimated/Actual Rows.
- PASO 3 · Consulta B (SARGable): SELECT PedidoId FROM dbo.Pedido WHERE FechaPedido >= '20240101' AND FechaPedido < '20250101'; Resultado esperado: Index Seek con las mismas filas. Comparar lecturas con A.
- PASO 4 · Consulta C: SELECT PedidoId, Total FROM dbo.Pedido WHERE FechaPedido = '20240315'; Resultado esperado: Index Seek + Key Lookup (Total no está en el índice). Abrir las propiedades del Key Lookup (F4) y ver Number of Executions.
- PASO 5 · Resolver C: CREATE NONCLUSTERED INDEX IX_Pedido_Fecha ON dbo.Pedido (FechaPedido) INCLUDE (Total) WITH (DROP_EXISTING = ON); y repetir. El Key Lookup desaparece.
- PASO 6 · Opcional: en el plan, clic derecho → Show Execution Plan XML y localizar CardinalityEstimationModelVersion y StatementOptmLevel; luego comparar con ALTER DATABASE SCOPED CONFIGURATION SET LEGACY_CARDINALITY_ESTIMATION = ON; (volver a OFF al terminar).
- TRAMPAS HABITUALES:** (a) comparar planes estimados en lugar de reales; (b) el optimizador elige Scan en la consulta B si el año consultado abarca gran parte de la tabla: probar con un mes en vez de un año; (c) mezclar tipos (comparar con un literal NVARCHAR o con una columna de otro tipo) provoca CONVERT_IMPLICIT y warnings; (d) el plan cacheado de ejecuciones previas: usar OPTION (RECOMPILE) si hay dudas con parametrización.
- GUÍA DE RESOLUCIÓN:** si B no hace Seek, comprobar que el índice IX_Pedido_Fecha existe (sys.indexes) y que se ha ejecutado en la base Ventas; si hay desviación de estimaciones, UPDATE STATISTICS dbo.Pedido WITH FULLSCAN y repetir.

DIAPOSITIVA 59

LABORATORIO · LABORATORIO 3.3

3 · Optimización y Alta Disponibilidad

Lab 3.3 · Cuellos de botella con DMVs

CONTENIDO CLAVE EN PANTALLA (PROYECCIÓN)

- 1
- Provocar un bloqueo
- Sesión 1 abre una transacción con UPDATE sin COMMIT; Sesión 2 ejecuta un SELECT.
- 2
- Localizar la espera
- En Sesión 3 consulta sys.dm_exec_requests y sys.dm_os_waiting_tasks (LCK_M_S).
- 3
- Resolver y revisar
- Identifica al head blocker, haz ROLLBACK y revisa sys.dm_os_wait_stats.
- T-SQL · SESIONES 1 Y 2

-- Sesión 1: transacción sin cerrar

```
BEGIN TRAN;
UPDATE dbo.Cliente SET Activo = 0
WHERE ClienteId = 1;
-- Sesión 2: queda bloqueada
SELECT Nombre FROM dbo.Cliente
WHERE ClienteId = 1;
```

SQL

- T-SQL · SESIÓN 3 (DBA)

```
SELECT r.session_id, r.blocking_session_id,
       r.wait_type, r.wait_time, t.text
FROM sys.dm_exec_requests AS r
CROSS APPLY sys.dm_exec_sql_text(r.sql_handle) t
WHERE r.blocking_session_id <> 0;
-- Resolver: ROLLBACK en la sesión 1
```

SQL

OBJETIVO PEDAGÓGICO

Provocar un bloqueo controlado y localizarlo con DMVs (sys.dm_exec_requests, sys.dm_os_waiting_tasks, sys.dm_os_wait_stats), identificando al head blocker y resolviendo la incidencia.

GUIÓN DEL DOCENTE Y EXPLICACIÓN TÉCNICA

- **Duración orientativa:** 30 minutos. Se necesitan tres pestañas de consulta en SSMS (tres sesiones) contra la base Ventas.
- **Narrativa:** "la aplicación se queda colgada". Un desarrollador dejó una transacción abierta (Sesión 1); otra consulta (Sesión 2) queda bloqueada con LCK_M_S. La Sesión 3 es la del DBA que investiga solo con DMVs.
- **Conceptos reforzados:** blocking_session_id, wait_type LCK_M_*, head blocker, dm_os_waiting_tasks vs dm_exec_requests, y la lectura de dm_os_wait_stats antes/después.
- Cuando acabe el laboratorio, es el momento de mencionar READ_COMMITTED_SNAPSHOT (RCSI) como mitigación de bloqueos lector-escritor y de mantener transacciones cortas en la aplicación.

PUNTOS DE INTERACCIÓN Y PREGUNTAS PARA LA CLASE

- ¿Cómo distinguiríais entre una consulta lenta por CPU y una lenta por bloqueo mirando únicamente sys.dm_exec_requests?
- ¿Es razonable ejecutar KILL sobre el head blocker en producción? ¿Qué información necesitáis antes?

INSTRUCCIONES DE LABORATORIO Y PAUTAS DE RESOLUCIÓN

- PASO 1 · SESIÓN 1 (pestaña 1): USE Ventas; BEGIN TRAN; UPDATE dbo.Cliente SET Activo = 0 WHERE ClienteId = 1; (no ejecutar COMMIT). Anotar el SPID con SELECT @@SPID;
- PASO 2 · SESIÓN 2 (pestaña 2): USE Ventas; SELECT Nombre FROM dbo.Cliente WHERE ClienteId = 1; La consulta queda "Ejecutando..." sin devolver resultados (bloqueada por LCK_M_S).
- PASO 3 · SESIÓN 3 (pestaña 3, la del DBA): SELECT r.session_id, r.blocking_session_id, r.wait_type, r.wait_time, t.text FROM sys.dm_exec_requests AS r CROSS APPLY sys.dm_exec_sql_text(r.sql_handle) AS t WHERE r.blocking_session_id <> 0; Resultado esperado: la sesión 2 con blocking_session_id = SPID de la sesión 1 y wait_type LCK_M_S.
- PASO 4 · Ver la vista instantánea de esperas: SELECT session_id, wait_type, wait_duration_ms, blocking_session_id, resource_description FROM sys.dm_os_waiting_tasks WHERE session_id > 50; Localizar el head blocker (el SPID que aparece como bloqueante y no espera). Con sys.dm_exec_sessions comprobar login_name, host_name y program_name del responsable.
- PASO 5 · Resolver: en la Sesión 1 ejecutar ROLLBACK; (o KILL desde la sesión 3 en un caso real tras confirmar con el responsable). La Sesión 2 se completa.
- PASO 6 · Revisar esperas acumuladas: SELECT TOP (5) wait_type, wait_time_ms FROM sys.dm_os_wait_stats ORDER BY wait_time_ms DESC; y comentar que LCK_M_S de este experimento es insignificante frente a las esperas del sistema; por eso se necesitan dos snapshots.
- **TRAMPAS HABITUALES:** (a) la sesión 3 usa la misma pestaña que la 1 o 2 y queda bloqueada también: abrir pestañas distintas; (b) READ_COMMITTED_SNAPSHOT ON en la base evita el bloqueo del SELECT: comprobar con SELECT is_read_committed_snapshot_on FROM sys.databases WHERE name = N'Ventas'; (c) sin permiso VIEW SERVER STATE las DMVs fallan o devuelven vacío; (d) olvidar cerrar la transacción al final y dejar la tabla bloqueada para otros grupos.

- **GUÍA DE RESOLUCIÓN:** si no ves bloqueo, repetir con SET TRANSACTION ISOLATION LEVEL READ COMMITTED en la sesión 2 o actualizar la misma fila; para ver también las transacciones abiertas usar DBCC OPENTRAN o sys.dm_tran_active_transactions.

DIAPOSITIVA 60

HA/DR

Diseño de índices

3 · Optimización y Alta Disponibilidad

 CONTENIDO CLAVE EN PANTALLA (PROYECCIÓN)

- Clustered estrecho y creciente; NCI con INCLUDE; vigila el coste de escritura.
- Diagnóstico
- Plan real, STATISTICS IO y DMVs: mide antes de cambiar y compara después.
- Mantenimiento
- Reorganize 5–30 %, Rebuild > 30 %, estadísticas al día y fillfactor justificado.
- Define RTO y RPO primero; elige Log Shipping, FCI o AG según edición y coste.

 OBJETIVO PEDAGÓGICO

Consolidar los cuatro mensajes del módulo (diseño de índices, diagnóstico, mantenimiento y HA/DR) y repasar los errores más frecuentes antes de pasar al Módulo 4.

 GUION DEL DOCENTE Y EXPLICACIÓN TÉCNICA

- **Recapitular con el ciclo:** medir → hipótesis → cambio → volver a medir. Cada herramienta del módulo (plan real, STATISTICS IO/TIME, DMVs, Query Store) sirve a una de esas fases.
- **Índices:** un Clustered Index estrecho, único, estático y creciente; NCI diseñados para las consultas críticas, con la clave en orden igualdad → rango y columnas de lectura en INCLUDE; revisar user_updates frente a user_seeks para eliminar índices inútiles.
- **Errores comunes:** crear índices a ciegas copiando Missing Index DMVs; reconstruir todos los índices cada noche con el mismo umbral; no ajustar el nivel de compatibilidad ni el CE tras migrar; predicados no SARGable (funciones sobre columnas, conversiones implícitas); confiar en el Coste % del plan como si fuera tiempo.
- **Mantenimiento:** sys.dm_db_index_physical_stats en modo LIMITED, reorganizar entre 5 y 30 %, reconstruir por encima de 30 % en índices de más de ~1000 páginas; mantener estadísticas actualizadas (el REBUILD ya lo hace con FULLSCAN; tras REORGANIZE, actualizarlas aparte).
- **HA/DR:** empezar siempre por RTO y RPO acordados con el negocio; Log Shipping (simple, manual), FCI (instancia, almacenamiento compartido), AG (base de datos, réplicas síncronas/asíncronas) y recordar que ninguna sustituye a los backups (Módulo 4).
- **Transición:** en el Módulo 4 se automatiza el mantenimiento de índices y estadísticas, se diseña la estrategia de backups y restauración, y se practican incidencias reales (LDF lleno, tempdb, deadlocks).

 PUNTOS DE INTERACCIÓN Y PREGUNTAS PARA LA CLASE

- ¿Qué cambio de este módulo aplicaríais primero mañana en vuestro entorno real y cómo mediríais que ha funcionado?
- ¿Qué diferencia hay entre "alta disponibilidad" y "copia de seguridad" tras lo visto? Comparad con la respuesta de inicio del módulo.

6,0 H · DIAPOSITIVAS 61 – 78

MÓDULO 4: MANTENIMIENTO, COPIAS DE SEGURIDAD Y CASO FINAL

Backups, Point-in-Time, CHECKDB, Agent, incidencias, Lab 4 y Caso Integrador

DIAPOSITIVA 61

INTRODUCCIÓN MÓDULO

04 · Apertura del Módulo: Mantenimiento y Buenas Prácticas

4 · Mantenimiento y Buenas Prácticas

CONTENIDO CLAVE EN PANTALLA (PROYECCIÓN)

- MÓDULO 4 · 6 h
- Mantenimiento y Buenas Prácticas
- Recuperación, integridad, automatización y resolución de incidencias
- Modelos de recuperación y backups
- Restauración point-in-time

Integridad física: DBCC CHECKDB

SQL

- Mantenimiento automatizado
- Incidencias: LDF, TempDB, bloqueos
- Laboratorio 4 y caso integrador

OBJETIVO PEDAGÓGICO

Situar al alumnado en el último bloque del curso: pasar de «saber crear y consultar» a «saber proteger, mantener y recuperar» una instancia en producción.

GUION DEL DOCENTE Y EXPLICACIÓN TÉCNICA

- **Este módulo es el que separa a un desarrollador con conocimientos de SQL Server de un DBA:** aquí se juega la continuidad del negocio. Todo lo anterior (arquitectura, seguridad, índices, HA) converge en una pregunta: ¿qué ocurre cuando algo falla a las 3 de la madrugada?
- **Recorrido:** (1) modelos de recuperación y cómo condicionan el Transaction Log; (2) estrategia de backups Full + Diferencial + Log y su verificación; (3) restauración point-in-time; (4) integridad física con DBCC CHECKDB; (5) mantenimiento automatizado con SQL Server Agent; (6) incidencias típicas (LDF lleno, TempDB saturada, bloqueos y deadlocks).
- **Cerraremos con el Laboratorio 4 (pérdida de datos simulada y recuperación a un instante exacto) y un caso integrador que evalúa todo el curso:** seguridad, índices, backups y RPO/RTO.
- **Recordar el hilo conductor:** el Transaction Log (WAL) que vimos en el Módulo 1 es la pieza que hace posible la durabilidad (la «D» de ACID), el crash recovery y la restauración a un punto en el tiempo. Casi todo lo de este módulo es una consecuencia del WAL.
- **Tiempo orientativo:** ~6 h (teoría ~3,5 h, laboratorio 4 ~1,5 h, caso integrador ~1 h). Si el grupo va justo de tiempo, la diapositiva de mantenimiento de índices puede darse como lectura y recuperarse en el caso final.

PUNTOS DE INTERACCIÓN Y PREGUNTAS PARA LA CLASE

- ¿Quién ha sufrido alguna vez una pérdida de datos o ha tenido que restaurar una base de datos? ¿Qué falló: el backup, el proceso o la comunicación?
- ¿Cuál es el backup más importante: el que se hace o el que se ha probado restaurar?

DIAPOSITIVA 62

BACKUPS

Modelos de recuperación

4 · Mantenimiento y Buenas Prácticas

 CONTENIDO CLAVE EN PANTALLA (PROYECCIÓN)

Minimal logging: registra sólo la asignación de extents, **no** cada fila. Es más rápido y genera menos log, pero log backup con carga masiva **no** permite STOPAT en su interior. SQL

Tabla Resumen en Diapositiva:

[object Object]undefined

 OBJETIVO PEDAGÓGICO

Entender que el recovery model no cambia lo que se registra para el crash recovery, sino cuándo se puede truncar el log y hasta qué punto se puede restaurar.

 GUION DEL DOCENTE Y EXPLICACIÓN TÉCNICA

- **Recovery model Simple:** el log se sigue usando (rollback y crash recovery) pero se trunca automáticamente en cada checkpoint cuando no hay otro motivo de retención. No existe BACKUP LOG y sólo se puede restaurar al último Full o Diferencial: RPO = tiempo desde el último backup.
- **Full:** toda operación se registra completa y el log NO se trunca hasta que un BACKUP LOG marca los VLFs como reutilizables (log_reuse_wait_desc = LOG_BACKUP). Permite STOPAT y un RPO cercano a cero con tail-log backup. Ojo: hasta el primer Full backup la base se comporta como Simple (la log chain aún no se ha iniciado).
- **Bulk-Logged:** igual que Full salvo en operaciones masivas (BULK INSERT, bcp, INSERT...SELECT con TABLOCK, SELECT INTO, CREATE/ALTER INDEX) que usan minimal logging: sólo se registran las asignaciones de extents, no las filas. El log backup posterior copia además esos extents (bitmap ML), por lo que puede ser grande.
- **Limitación clave de Bulk-Logged:** un log backup que contiene operaciones minimally logged no admite STOPAT dentro de él (error 4341) y, si el MDF se pierde, no se puede hacer tail-log. Por eso se activa sólo durante la ventana ETL y se vuelve a Full inmediatamente, con un log backup antes y otro después.
- **Cambiar de modelo:** ALTER DATABASE Ventas SET RECOVERY FULL. De Simple a Full hay que tomar un Full (o Diferencial) para iniciar la cadena. De Full a Simple se rompe la log chain: tras volver a Full, nuevo Full obligatorio.
- **Dato práctico:** la base model viene en Full, así que toda base nueva nace en Full. Si nadie programa BACKUP LOG, el LDF crece sin límite hasta llenar el disco (causa n° 1 de «LDF lleno»; lo veremos en las incidencias). Consulta: SELECT name, recovery_model_desc, log_reuse_wait_desc FROM sys.databases.

 PUNTOS DE INTERACCIÓN Y PREGUNTAS PARA LA CLASE

- Si una base es Simple, ¿cuántos datos puedes perder como máximo si el disco falla justo antes del siguiente backup? ¿Es aceptable para un ERP?
- ¿Por qué cambiar a Bulk-Logged «sólo un rato» es una decisión con riesgo para el RPO?
- ¿Qué recovery model elegirías para un entorno de desarrollo y cuál para la base Ventas de producción?

DIAPOSITIVA 63

BACKUPS

Estrategia: Full + Diferencial + Log chain

4 · Mantenimiento y Buenas Prácticas

CONTENIDO CLAVE EN PANTALLA (PROYECCIÓN)

- | | |
|--|---|
| <ul style="list-style-type: none"> • Domingo • Lunes • Martes • Miércoles • Jueves • Full • Diferencial | <ul style="list-style-type: none"> • Log de transacciones • Dom 22:00 • Lun 22:00 • Mar 22:00 • cada 15 min • Incidente · Mié 10:37 • Secuencia de restauración (STOPAT = 10:36) |
|--|---|

1 · RESTORE **FULL**

SQL

- Domingo · NORECOVERY

2 · RESTORE **DIFF**

SQL

- Martes · NORECOVERY

3 · RESTORE **LOG(s)**

SQL

- | | |
|--|--|
| <ul style="list-style-type: none"> • Mar 22:15 → Mié 10:30 · NORECOVERY | <ul style="list-style-type: none"> • 4 · LOG final • WITH STOPAT = 10:36, RECOVERY |
|--|--|

OBJETIVO PEDAGÓGICO

Diseñar una estrategia de backups que combine los tres tipos para cumplir un RPO y un RTO concretos, y entender la log chain y los LSN.

GUION DEL DOCENTE Y EXPLICACIÓN TÉCNICA

- **Full:** copia todas las páginas asignadas más la parte activa del log necesaria para dejar la base consistente al terminar. Es la base de cualquier restauración; en el ejemplo, cada domingo a las 22:00.
- **Diferencial:** copia los extents modificados desde el último Full, que el motor localiza con el bitmap DCM (Differential Changed Map). Es acumulativo: para restaurar sólo se necesita el último diferencial, no todos. Crece a lo largo de la semana; si supera ~50 % del Full conviene un nuevo Full.
- **Log backup:** copia el log desde el último BACKUP LOG y permite truncar. Cada log backup enlaza con el anterior mediante LSN (el first_lsn de uno es el last_lsn del anterior); esa secuencia ininterrumpida es la log chain. Se rompe al pasar a Simple o si se pierde/borra un .trn.
- **RPO y RTO:** el RPO lo marca la frecuencia del log backup (cada 15 min → hasta 15 min de pérdida; con tail-log backup, casi cero). El RTO depende del tamaño del Full, del diferencial y de cuántos logs hay que reaplicar: más diferenciales = menos logs que reproducir = restauración más rápida.
- **Ejemplo del diagrama:** fallo el miércoles a las 10:37. Restauramos el Full del domingo (NORECOVERY), el diferencial del martes (NORECOVERY), los logs del martes 22:15 al miércoles 10:30 (NORECOVERY) y el último con STOPAT 10:36 y RECOVERY. Cualquier log que falte interrumpe la secuencia.
- **COPY_ONLY** no restablece el bitmap DCM ni afecta a la cadena: es lo que se usa para copias puntuales (por ejemplo, pasar la base a pre-producción) sin invalidar el plan. En msdb.dbo.backupset las columnas first_lsn, last_lsn, database_backup_lsn y differential_base_lsn permiten auditar la cadena.

PUNTOS DE INTERACCIÓN Y PREGUNTAS PARA LA CLASE

- Con un Full semanal, diferencial diario y log cada 15 minutos, ¿cuál es el peor caso de pérdida de datos? ¿Y qué ocurre si falla el disco donde están también los backups?
- ¿Qué backups hay que restaurar si el fallo ocurre el jueves a las 09:00 y el diferencial del miércoles está corrupto?
- ¿Cómo equilibrarías RTO y coste de almacenamiento al decidir entre más diferenciales o más logs?

DIAPOSITIVA 64

BACKUPS

T-SQL de backup: Full, Diff y Log

4 · Mantenimiento y Buenas Prácticas

CONTENIDO CLAVE EN PANTALLA (PROYECCIÓN)

- T-SQL

SQL

```
-- Full semanal con compresión y verificación de páginas
BACKUP DATABASE Ventas
TO DISK = N'/var/opt/mssql/backup/Ventas_FULL.bak'
WITH COMPRESSION, CHECKSUM, INIT, STATS = 10;

-- Diferencial diario (cambios desde el último Full)
BACKUP DATABASE Ventas
TO DISK = N'/var/opt/mssql/backup/Ventas_DIFF.bak'
WITH DIFFERENTIAL, COMPRESSION, CHECKSUM;

-- Log cada 15 min (sólo Full / Bulk-Logged)
BACKUP LOG Ventas
TO DISK = N'/var/opt/mssql/backup/Ventas_LOG_01.trn'
WITH COMPRESSION, CHECKSUM;

-- Copia puntual que no altera la cadena
BACKUP DATABASE Ventas
TO DISK = N'/var/opt/mssql/backup/Ventas_CO.bak'
WITH COPY_ONLY, COMPRESSION, CHECKSUM;
```

- COMPRESSION + CHECKSUM
- Menos E/S y tamaño; valida páginas al leer.
- COPY_ONLY
- Copia ad hoc sin alterar diferencial ni log chain.
- TO URL (Azure Blob)
- Copia externa en la nube con credencial SAS.

OBJETIVO PEDAGÓGICO

Dominar la sintaxis de BACKUP con las opciones que se usan en producción (COMPRESSION, CHECKSUM, COPY_ONLY, TO URL) y saber por qué se usan.

GUION DEL DOCENTE Y EXPLICACIÓN TÉCNICA

- BACKUP DATABASE ... TO DISK con WITH COMPRESSION reduce el tamaño (típicamente 3–5x según los datos) y la E/S de escritura, a costa de algo de CPU. Está disponible en Standard y Enterprise desde SQL Server 2008 R2/2012. Se puede forzar por defecto con sp_configure «backup compression default».
- CHECKSUM hace que el motor valide el checksum de cada página al leerla y calcule un checksum global del backup. Si encuentra una página corrupta, el backup falla (a menos que se use CONTINUE_AFTER_ERROR), lo cual es una detección temprana de corrupción. Es obligatorio en toda la estrategia.
- INIT sobrescribe el fichero; sin él se anexan sets al mismo medio (NOINIT). STATS = 10 muestra el avance. Convención de nombres con fecha y hora (Ventas_FULL_20250312_2200.bak) para evitar sobrescrituras y facilitar la limpieza.
- DIFFERENTIAL:** sólo cambios desde el último Full «base». Un BACKUP LOG sólo es posible en Full/Bulk-Logged y tras un primer Full; si la base está en Simple devuelve el error 4208.
- COPY_ONLY: copia independiente que no actualiza differential_base_lsn ni trunca el log (en BACKUP LOG). Úsalo para extraer copias para desarrollo o auditoría.
- BACKUP ... TO URL (Azure Blob Storage) permite enviar copias fuera del datacenter con una credencial SAS (CREATE CREDENTIAL sobre la URL del contenedor). Desde 2022 también se admite S3. Regla 3-2-1: 3 copias, 2 soportes distintos, 1 fuera de sitio. Para backups grandes en Azure, usar striping (varios TO URL) por el límite de bloque.
- En Docker/Linux las rutas son /var/opt/mssql/backup/...; el proceso mssql necesita permisos de escritura en esa carpeta (usuario mssql). Un backup en el mismo disco que los datos no es un backup.

PUNTOS DE INTERACCIÓN Y PREGUNTAS PARA LA CLASE

- ¿Qué ventaja tiene CHECKSUM frente a lanzar un backup sin verificación de páginas?
- ¿Cuándo usarías COPY_ONLY y qué se rompería si hicieras ese backup sin la opción?
- ¿Dónde guardarías los ficheros .bak para cumplir la regla 3-2-1?

DIAPOSITIVA 65

BACKUPS

Tail-log, verificación e historial msdb

4 · Mantenimiento y Buenas Prácticas

CONTENIDO CLAVE EN PANTALLA (PROYECCIÓN)

• T-SQL

```
-- Tail-log: salvar el final del log antes de restaurar
BACKUP LOG Ventas
TO DISK = N'/var/opt/mssql/backup/Ventas_TAIL.trn'
WITH NO_TRUNCATE, NORECOVERY, CHECKSUM;

-- Verificar el backup sin restaurarlo
RESTORE VERIFYONLY
FROM DISK = N'/var/opt/mssql/backup/Ventas_FULL.bak'
WITH CHECKSUM;
```

SQL

• T-SQL · HISTORIAL EN MSDB

```
SELECT TOP (10) bs.database_name,
               bs.type,      -- D=Full I=Diff L=Log
               bs.backup_finish_date,
               bs.is_copy_only,
               bmf.physical_device_name
FROM msdb.dbo.backupset AS bs
JOIN msdb.dbo.backupmediafamily AS bmf
  ON bmf.media_set_id = bs.media_set_id
WHERE bs.database_name = N'Ventas'
ORDER BY bs.backup_finish_date DESC;
```

SQL

VERIFYONLY **no** basta: sólo comprueba que el fichero es legible. La prueba real es restaurar en otro servidor y ejecutar DBCC CHECKDB; planifícala (p. ej. mensualmente).

SQL

OBJETIVO PEDAGÓGICO

Saber salvar la cola del log antes de restaurar, verificar que un backup es legible y consultar el historial de backups en msdb.

GUION DEL DOCENTE Y EXPLICACIÓN TÉCNICA

- **Tail-log backup:** es el log backup que se hace ANTES de restaurar para capturar las transacciones que aún no estaban en ningún backup. WITH NORECOVERY deja la base en estado RESTORING e impide nuevas transacciones; WITH NO_TRUNCATE permite hacerlo aunque la base esté dañada (siempre que el LDF sea accesible). Es lo que da un RPO cercano a cero.
- Si el LDF también se ha perdido, el tail-log es imposible y el RPO queda en el último log backup. Por eso conviene separar MDF y LDF en discos distintos, y por eso los log backups frecuentes importan más que los Full.
- **RESTORE VERIFYONLY comprueba que el set de backup está completo y es legible (cabecera, checksums si se tomó WITH CHECKSUM). NO restaura datos ni garantiza que la base sea consistente:** es una verificación mínima. La prueba real es restaurar en otro servidor y lanzar DBCC CHECKDB.
- msdb.dbo.backupset guarda un registro por backup (type: D = Full, I = Diferencial, L = Log; is_copy_only, first_lsn, last_lsn, backup_size, compressed_backup_size) y backupmediafamily la ruta física. msdb.dbo.restorehistory documenta las restauraciones. Son la fuente de verdad para reconstruir la cadena cuando alguien pregunta «¿cuál fue el último backup?».
- **Limpieza:** msdb crece con el historial; programar sp_delete_backuphistory @oldest_date (lo veremos en mantenimiento) y vigilar el tamaño de msdb. Alerta: un backup «reciente» en msdb no implica que el fichero exista aún en disco.
- RESTORE HEADERONLY, RESTORE FILELISTONLY y RESTORE LABELONLY inspeccionan el contenido de un .bak sin restaurarlo; FILELISTONLY es imprescindible para saber qué nombres lógicos usar en MOVE.

PUNTOS DE INTERACCIÓN Y PREGUNTAS PARA LA CLASE

- ¿Qué diferencia hay entre «el backup se ha verificado» y «la base se puede restaurar»?
- ¿Qué haces si el servidor ha caído, el MDF está corrupto pero el LDF está intacto?
- ¿Cómo detectarías con una consulta que lleva dos semanas sin hacerse un log backup en alguna base?

DIAPOSITIVA 66

RESTAURACIÓN

Restauración point-in-time (STOPAT)

4 · Mantenimiento y Buenas Prácticas

CONTENIDO CLAVE EN PANTALLA (PROYECCIÓN)

- T-SQL

SQL

```
-- 1) Full. MOVE reubica los ficheros lógicos
RESTORE DATABASE Ventas
FROM DISK = N'/var/opt/mssql/backup/Ventas_FULL.bak'
WITH NORECOVERY,
    MOVE N'Ventas' TO N'/var/opt/mssql/data/Ventas.mdf',
    MOVE N'Ventas_log' TO N'/var/opt/mssql/data/Ventas.ldf';

-- 2) Último diferencial
RESTORE DATABASE Ventas
FROM DISK = N'/var/opt/mssql/backup/Ventas_DIFF.bak'
WITH NORECOVERY;

-- 3) Logs en orden; el último, con STOPAT
RESTORE LOG Ventas
FROM DISK = N'/var/opt/mssql/backup/Ventas_LOG_01.trn'
WITH NORECOVERY;

RESTORE LOG Ventas
FROM DISK = N'/var/opt/mssql/backup/Ventas_LOG_02.trn'
WITH STOPAT = N'2025-03-12T10:36:00', RECOVERY;
```

- NORECOVERY
- Deja la BD en RESTORING; permite encadenar más backups.
- RECOVERY
- Deshace lo no confirmado y abre la BD (ONLINE).
- STANDBY
- Sólo lectura entre restauraciones; usa fichero undo.

OBJETIVO PEDAGÓGICO

Ser capaz de escribir y razonar la secuencia completa de RESTORE (Full → Diff → Logs) para volver a un instante concreto, entendiendo NORECOVERY, RECOVERY, STANDBY y MOVE.

GUION DEL DOCENTE Y EXPLICACIÓN TÉCNICA

- **Orden estricto:** 1) RESTORE DATABASE del Full con NORECOVERY; 2) el último Diferencial con NORECOVERY (opcional pero acelera); 3) todos los RESTORE LOG en orden de LSN con NORECOVERY; 4) el último log con STOPAT y RECOVERY (o sólo RECOVERY si se quiere hasta el final).
- NORECOVERY deja la base en RESTORING y NO deshace transacciones sin confirmar, para poder seguir aplicando backups. RECOVERY (opción por defecto) ejecuta la fase de undo, abre la base y ya no admite más backups de la cadena. STANDBY = fichero undo + base de sólo lectura entre restauraciones (útil en Log Shipping para consultar la réplica).
- STOPAT = fecha/hora: el motor re replica el log hasta ese instante y descarta las transacciones posteriores (la hora es la del servidor, no UTC). Alternativas: STOPATMARK / STOPBEFOREMARK con transacciones marcadas (BEGIN TRAN ... WITH MARK) y STOPAT con LSN en 2022.
- MOVE reubica los ficheros lógicos al restaurar (otro volumen o servidor). Los nombres lógicos se ven con RESTORE FILELISTONLY. En Linux/Docker las rutas son /var/opt/mssql/data/...; si no se usa MOVE, el motor intenta usar las rutas originales y falla si no existen.
- **REPLACE sobrescribe una base existente:** úsalo sólo si estás seguro. Antes, hacer siempre un tail-log. Para restauraciones parciales existen RESTORE ... PAGE (restauración de página, online en Enterprise) y RESTORE con PARTIAL / filegroups.
- **Progreso:** STATS = 5 o sys.dm_exec_requests.percent_complete. El tiempo de restauración incluye la fase de recovery (redo/undo): un log de transacción largo con una transacción grande abierta alarga el undo. El Instant File Initialization acelera la creación de los MDF/NDF, no del LDF.
- **Tras RECOVERY:** validar el negocio (COUNT(*) de tablas clave), ejecutar DBCC CHECKDB y recrear logins huérfanos con ALTER USER ... WITH LOGIN o sp_change_users_login (mismatch de SID al restaurar en otro servidor).

PUNTOS DE INTERACCIÓN Y PREGUNTAS PARA LA CLASE

- ¿Qué pasa si restauras el Full con RECOVERY y luego intentas aplicar un diferencial?
- Un compañero ejecutó DELETE sin WHERE a las 10:36. ¿Qué STOPAT usarías y qué harías con los datos introducidos entre las 10:36 y las 10:50?
- ¿Por qué conviene restaurar con otro nombre en un servidor distinto antes de tocar producción?

DIAPOSITIVA 67

INTEGRIDAD

Integridad física: DBCC CHECKDB

4 · Mantenimiento y Buenas Prácticas

CONTENIDO CLAVE EN PANTALLA (PROYECCIÓN)

- 823
- Error de E/S del sistema operativo
- 824
- Error lógico: checksum o página rota
- 825
- Lectura con éxito tras reintentos: disco en riesgo
- T-SQL

SQL

```
-- Verificación de páginas a nivel de BD
ALTER DATABASE Ventas SET PAGE_VERIFY CHECKSUM;

DBCC CHECKDB (N'Ventas') WITH NO_INFOMSGS, ALL_ERRORMSG;

-- Páginas dañadas registradas por el motor
SELECT database_id, file_id, page_id, event_type
FROM msdb.dbo.suspect_pages;
```

- Opciones de reparación
- REPAIR_REBUILD: sin pérdida de datos.
- ALLOW_DATA_LOSS: último recurso, tras backup.
- Antes: restaurar desde backup.

OBJETIVO PEDAGÓGICO

Comprender cómo se detecta la corrupción (PAGE_VERIFY CHECKSUM, DBCC CHECKDB, errores 823/824/825) y por qué la reparación con pérdida de datos es el último recurso.

GUIÓN DEL DOCENTE Y EXPLICACIÓN TÉCNICA

- PAGE_VERIFY CHECKSUM (por defecto desde 2005 en bases nuevas) escribe un checksum en la cabecera de cada página al grabarla a disco y lo valida al leerla. Detecta corrupción introducida por el subsistema de E/S. Comprobar con SELECT name, page_verify_option_desc FROM sys.databases; las bases migradas de 2000 pueden estar en TORN_PAGE_DETECTION o NONE.
- **Los errores típicos:** 823 = fallo de E/S del sistema operativo al leer/escribir (CRC, disco o controladora); 824 = error lógico de consistencia (checksum o torn page incorrectos); 825 = la lectura falló pero tuvo éxito tras reintentos (read-retry): es una alerta temprana de un disco que se está degradando. Todos son gravedad 24: crear alertas del Agent para 823, 824, 825 y gravedades 19–25.
- DBCC CHECKDB combina CHECKALLOC (asignación), CHECKTABLE (estructuras de tablas e índices), CHECKCATALOG y validación de Service Broker/vistas indexadas. Trabaja sobre un snapshot interno (sparse, sin bloqueos) y usa mucho TempDB y E/S. Con WITH PHYSICAL_ONLY es mucho más rápido (sólo estructura física y checksums); ejecutar la verificación completa al menos semanalmente.
- msdb.dbo.suspect_pages registra las páginas con error 823/824 (event_type 1–3: error de E/S, mal checksum, torn page; 4 = restaurada; 5 = reparada; 7 = desasignada). En un Availability Group, la réplica puede reparar páginas automáticamente (sys.dm_hadr_auto_page_repair).
- **Reparación:** REPAIR_REBUILD arregla problemas menores (por ejemplo, índices non-clustered) sin pérdida; REPAIR_ALLOW_DATA_LOSS borra páginas y filas para restaurar consistencia estructural, sin respetar integridad referencial ni reglas de negocio. Exige SINGLE_USER y siempre después de hacer un backup. Es el último recurso.
- **Primera opción ante corrupción:** restaurar desde backup (o RESTORE ... PAGE si afecta a pocas páginas y existe log chain) y reemplazar logs; así no se pierden datos. Ejecuta CHECKDB sobre una copia restaurada para descargar producción y, de paso, probar los backups.

PUNTOS DE INTERACCIÓN Y PREGUNTAS PARA LA CLASE

- ¿Por qué un backup WITH CHECKSUM no sustituye a DBCC CHECKDB?
- Aparece un error 825 en el errorlog, pero las aplicaciones funcionan. ¿Qué haces y con qué urgencia?
- ¿Cuándo es defendible usar REPAIR_ALLOW_DATA_LOSS, y qué comunicarías al negocio después?

DIAPOSITIVA 68

MANTENIMIENTO

Reorganize vs Rebuild y estadísticas

4 · Mantenimiento y Buenas Prácticas

CONTENIDO CLAVE EN PANTALLA (PROYECCIÓN)

- T-SQL

```
SELECT OBJECT_NAME(ips.object_id) AS Tabla, i.name,
       ips.avg_fragmentation_in_percent AS Frag
FROM sys.dm_db_index_physical_stats
     (DB_ID(), NULL, NULL, NULL, 'LIMITED') AS ips
JOIN sys.indexes AS i
     ON i.object_id = ips.object_id
    AND i.index_id = ips.index_id
WHERE ips.page_count > 1000 AND i.index_id > 0;

ALTER INDEX IX_Pedido_Fecha ON dbo.Pedido REORGANIZE;
ALTER INDEX IX_Pedido_Fecha ON dbo.Pedido REBUILD;
UPDATE STATISTICS dbo.Pedido WITH FULLSCAN;
```

SQL

- Estadísticas primero: actualízalas por umbral de cambios (dm_db_stats_properties) y en horas valle; suelen aportar más que reconstruir índices.

Tabla Resumen en Diapositiva:

[object Object]undefined

OBJETIVO PEDAGÓGICO

Elegir entre REORGANIZE y REBUILD según el nivel de fragmentación y entender por qué las estadísticas actualizadas importan más que la fragmentación.

GUION DEL DOCENTE Y EXPLICACIÓN TÉCNICA

- Fragmentación:** sys.dm_db_index_physical_stats con modo LIMITED (barato, sólo nivel hoja padre) mide avg_fragmentation_in_percent (fragmentación lógica por Page Splits). Regla clásica: < 5 % no hacer nada; 5–30 % REORGANIZE; > 30 % REBUILD; ignorar índices de menos de ~1000 páginas (no merece la pena).
- REORGANIZE:** compacta y reordena las páginas hoja en el sitio, siempre ONLINE, en pequeñas transacciones, se puede interrumpir sin perder el trabajo hecho, y NO actualiza estadísticas. Usa poco log pero lo genera de forma sostenida.
- REBUILD:** crea un índice nuevo completo y elimina el viejo; actualiza las estadísticas del índice con el equivalente a FULLSCAN, aplica el fill factor y puede reducir el espacio. Es offline por defecto; ONLINE = ON sólo en Enterprise (con WAIT_AT_LOW_PRIORITY desde 2014 y RESUMABLE desde 2017). Registra mucho log en Full: vigilar el LDF y los log backups durante la ventana.
- Realidad moderna:** con SSD/SAN y Buffer Pool grande la fragmentación influye menos que antes (sobre todo en scans y read-ahead). Lo realmente importante son las estadísticas: el optimizador estima cardinalidades con el histograma; estadísticas obsoletas = planes malos.
- Auto update statistics se dispara con ~20 % de filas modificadas (+500) en bases antiguas; con compatibilidad 130+ el umbral dinámico es SQRT(1000 × filas), mucho más sensible en tablas grandes. Se consulta con sys.dm_db_stats_properties (modification_counter, last_updated). UPDATE STATISTICS ... WITH FULLSCAN en horas valle para tablas críticas.
- Recordatorio:** REBUILD de todos los índices cada noche es un antipatrón de los Maintenance Plans clásicos; genera log, bloquea y no suele mejorar nada. Mejor soluciones por umbrales (Ola Hallengren) y fill factor adecuado para reducir los Page Splits.

PUNTOS DE INTERACCIÓN Y PREGUNTAS PARA LA CLASE

- ¿Por qué un REBUILD no necesita luego un UPDATE STATISTICS sobre el mismo índice, pero un REORGANIZE sí?
- ¿Qué riesgo tiene un REBUILD nocturno de todos los índices en una base en recovery Full?
- Si la fragmentación está al 40 % en un índice de 200 páginas, ¿merece la pena actuar?

DIAPOSITIVA 69

MANTENIMIENTO

Mantenimiento automatizado con Agent

4 · Mantenimiento y Buenas Prácticas

 CONTENIDO CLAVE EN PANTALLA (PROYECCIÓN)

- SQL Server Agent jobs
- Pasos T-SQL o CmdExec, schedules e historial en msdb.
- Maintenance Plans
- Asistente gráfico (SSIS): simple, pero poco flexible.
- Scripts Ola Hallengren
- DatabaseBackup, IntegrityCheck e IndexOptimize por umbrales.
- Limpieza de msdb
- sp_delete_backuphistory y sp_purge_jobhistory semanales.
- Database Mail y operadores
- El operador recibe el correo cuando un job falla.
- Alertas del Agent
- Gravedad 19–25 y errores 823, 824, 825 y 9002.

 OBJETIVO PEDAGÓGICO

Conocer las piezas para automatizar el mantenimiento (Agent jobs, Maintenance Plans, scripts de Ola Hallengren, limpieza de msdb, Database Mail y alertas) y saber elegir entre ellas.

 GUION DEL DOCENTE Y EXPLICACIÓN TÉCNICA

- **SQL Server Agent ejecuta jobs:** pasos (T-SQL, CmdExec, PowerShell, SSIS), schedules, notificaciones y reintentos. El estado vive en msdb (sysjobs, sysjobsteps, sysjobhistory). No existe en Express. En Docker/Linux se activa con MSSQL_AGENT_ENABLED=true (o mssql-conf set sqlagent.enabled true).
- **Maintenance Plans:** asistente gráfico, genera paquetes SSIS. Son fáciles pero poco flexibles: la tarea «Rebuild Index» reconstruye todo sin mirar fragmentación y la tarea «Shrink Database» es un antipatrón (fragmenta de nuevo y provoca crecimiento posterior).
- **Scripts de Ola Hallengren (ola.hallengren.com):** DatabaseBackup, DatabaseIntegrityCheck e IndexOptimize como procedimientos almacenados con parámetros (@FragmentationLevel1 = 5, @FragmentationLevel2 = 30, @UpdateStatistics = «ALL», @LogToTable). Son el estándar de facto, se auditan en la tabla CommandLog y funcionan con jobs de Agent. Recomendados sobre los Maintenance Plans.
- **Limpieza de msdb:** sp_delete_backuphistory @oldest_date, sp_purge_jobhistory, sysmail_delete_mailitems_sp y sysmail_delete_log_sp. Un msdb con años de historial ralentiza SSMS y las restauraciones (el asistente consulta backupset).
- **Database Mail (sp_configure «Database Mail XPs», perfil + cuenta SMTP)** permite que el Agent notifique. Los operadores (sp_add_operator) definen quién recibe correos de fallo de job. Alertas del Agent: por gravedad 19–25 y por errores concretos (823, 824, 825, 9002); la alerta dispara un job o notifica a un operador.
- **Calendario de referencia en producción:** log backup cada 15 min; diferencial diario; Full semanal; CHECKDB semanal; IndexOptimize diario/semanal; limpieza de historial semanal. Todo con notificación de fallo y vigilancia de que el job «no se ha ejecutado» (un job que nunca corre no falla).

 PUNTOS DE INTERACCIÓN Y PREGUNTAS PARA LA CLASE

- ¿Qué ventajas ofrece una solución basada en scripts frente a un Maintenance Plan gráfico? ¿Y desventajas?
- ¿Cómo te enterarías de que un job de backup lleva tres noches sin ejecutarse?
- ¿Qué errores y gravedades darías de alta como alertas en una instancia nueva?

DIAPOSITIVA 70

INCIDENCIAS

Incidencia: log de transacciones lleno

4 · Mantenimiento y Buenas Prácticas

CONTENIDO CLAVE EN PANTALLA (PROYECCIÓN)

- T-SQL

```
SELECT name, recovery_model_desc,
       log_reuse_wait_desc
FROM sys.databases
WHERE name = N'Ventas';

DBCC SQLPERF (LOGSPACE);
DBCC OPENTRAN (N'Ventas');

BACKUP LOG Ventas
TO DISK = N'/var/opt/mssql/backup/Log2.trn';
```

SQL

- Nunca borres el .ldf: la base puede quedar sin recuperar. Resuelve la causa raíz (log_reuse_wait_desc) y reduce el fichero sólo una vez, después.

Tabla Resumen en Diapositiva:

[object Object]undefined

OBJETIVO PEDAGÓGICO

Diagnosticar un LDF lleno (error 9002) mediante log_reuse_wait_desc, tratar la causa raíz y evitar «soluciones» peligrosas como borrar el fichero o encadenar SHRINKFILE.

GUIÓN DEL DOCENTE Y EXPLICACIÓN TÉCNICA

- Síntoma:** error 9002 «The transaction log for database is full due to ...». El motivo se indica al final del mensaje y en sys.databases.log_reuse_wait_desc: es lo que impide que los VLFs inactivos se marquen como reutilizables (truncado lógico).
- LOG_BACKUP: la causa más habitual; la base está en Full/Bulk-Logged y no hay BACKUP LOG programado (o ha fallado). Solución: hacer log backup y programarlos cada 15 min.
- ACTIVE_TRANSACTION: una transacción larga o abandonada mantiene activo el MinLSN; el log no se puede truncar aunque se hagan log backups. DBCC OPENTRAN o sys.dm_tran_active_transactions + sys.dm_exec_sessions localizan al responsable; cerrar la transacción en el cliente es preferible a KILL (el rollback puede tardar tanto como el trabajo hecho).
- AVAILABILITY_REPLICA y REPLICATION: una réplica del AG atrasada (sys.dm_hadr_database_replica_states) o un Log Reader Agent parado retienen el log hasta que el consumidor lo lea. También hay OLDEST_PAGE, CHECKPOINT, ACTIVE_BACKUP_OR_RESTORE y DATABASE_MIRRORING.
- Medición:** DBCC SQLPERF (LOGSPACE) o sys.dm_db_log_space_usage; sys.dm_db_log_info (2016 SP2+) muestra los VLFs. Un autogrowth pequeño (1 MB o 10 %) crea cientos de VLFs y ralentiza recuperación y backups: configurar crecimiento fijo (p. ej. 512 MB–1 GB) y tamaño inicial adecuado.
- Qué NO hacer:** borrar o renombrar el .ldf, pasar a Simple «para que se vacíe» sin entender que se rompe la log chain, o reducir (SHRINKFILE) en bucle. En emergencia puede añadirse un segundo fichero de log en otro volumen y retirarlo después. La reducción es puntual, tras resolver la causa.

PUNTOS DE INTERACCIÓN Y PREGUNTAS PARA LA CLASE

- ¿Por qué hacer un BACKUP LOG no libera espacio si hay una transacción abierta desde ayer?
- ¿Qué implica cambiar la base a Simple en mitad de la incidencia para el RPO del negocio?
- ¿Cómo evitarías que se repita (monitorización, tamaño inicial, autogrowth, alertas)?

Incidencia: TempDB saturada

CONTENIDO CLAVE EN PANTALLA (PROYECCIÓN)

- Version store
- RCSI y snapshot guardan versiones; una transacción larga las retiene.
- Spills y temporales
- Sort/hash sin memoria suficiente y #tablas grandes.
- PFS / GAM / SGAM
- Contención PAGELATCH en páginas de asignación.
- Varios ficheros
- Ficheros de igual tamaño: 1 por core, hasta 8.
- T-SQL

SQL

```
-- ¿Quién consume TempDB?
SELECT TOP (5) session_id,
    internal_objects_alloc_page_count AS p_int
FROM sys.dm_db_session_space_usage
ORDER BY p_int DESC;

-- ¿Contención de asignación?
SELECT session_id, wait_type
FROM sys.dm_os_waiting_tasks
WHERE wait_type LIKE N'PAGELATCH%'
    AND resource_description LIKE N'2:%';
```

- Ficheros iguales y pre-dimensionados; no hagas shrink rutinario.

OBJETIVO PEDAGÓGICO

Identificar qué consume TempDB (objetos de usuario, internos y version store) y reconocer y mitigar la contención de páginas de asignación PFS/GAM/SGAM.

GUION DEL DOCENTE Y EXPLICACIÓN TÉCNICA

- TempDB se recrea en cada arranque a partir de model; usa un log que no hace redo y es siempre «minimal». Todas las sesiones la comparten: una consulta descontrolada o una transacción larga puede llenarla y afectar a toda la instancia (errores 1105 y 9002 sobre tempdb).
- **Consumidores:** (1) objetos de usuario (#tablas, @variables tabla, cursores); (2) objetos internos (sorts, hash joins, spools, spills a disco por una mala estimación del memory grant: aparecen como Sort/Hash Warning en el plan); (3) version store, con RCSI, SNAPSHOT, triggers, online index operations: lo limpia una tarea de fondo, pero una transacción larga abierta impide la limpieza.
- **Diagnóstico:** sys.dm_db_file_space_usage (user_object_reserved_page_count, internal_object_reserved_page_count, version_store_reserved_page_count), sys.dm_db_session_space_usage y sys.dm_db_task_space_usage por sesión; sys.dm_tran_active_snapshot_database_transactions para la transacción más antigua del version store.
- **Contención de asignación:** PFS (página 1, cada 8088 páginas), GAM (página 2) y SGAM (página 3) se protegen con latches. Con muchas sesiones creando/destruyendo objetos temporales aparecen esperas PAGELATCH_UP/EX sobre 2:1:1, 2:1:2, 2:1:3 (sys.dm_os_waiting_tasks). Con 2019 se añade contención de metadatos, mitigable con MEMORY_OPTIMIZED TEMPDB_METADATA.
- **Mitigación:** varios ficheros de datos del MISMO tamaño y FILEGROWTH (1 por núcleo lógico hasta 8; si persiste, de 4 en 4); el instalador (2016+) propone el número y los TF 1117/1118 son comportamiento por defecto en tempdb. Pre-dimensionar y ponerla en almacenamiento rápido.
- **No reducir tempdb de forma rutinaria:** reiniciar la instancia la devuelve a su tamaño configurado. Si se llena, localizar la sesión (KILL con cautela), corregir la consulta (índices, estadísticas, memory grant) o acortar la transacción que retiene el version store.

PUNTOS DE INTERACCIÓN Y PREGUNTAS PARA LA CLASE

- ¿Qué diferencia hay entre un spill a TempDB y la creación explícita de una #tabla? ¿Cómo lo reconoces en el plan?
- Si activamos RCSI en Ventas, ¿qué cambia en el consumo de TempDB y qué vigilarías?
- ¿Por qué los ficheros de tempdb deben tener el mismo tamaño?

DIAPOSITIVA 72

INCIDENCIAS

Bloqueos prolongados y deadlocks

4 · Mantenimiento y Buenas Prácticas

CONTENIDO CLAVE EN PANTALLA (PROYECCIÓN)

- T-SQL

SQL

```
-- Peticiones bloqueadas y su bloqueador
SELECT r.session_id, r.blocking_session_id,
       r.wait_type, r.wait_time, t.text
FROM sys.dm_exec_requests AS r
CROSS APPLY sys.dm_exec_sql_text(r.sql_handle) AS t
WHERE r.blocking_session_id <> 0;

-- Locks en espera
SELECT resource_type, request_mode,
       request_status, request_session_id
FROM sys.dm_tran_locks
WHERE request_status = N'WAIT';

-- Grafo de deadlock (system_health)
SELECT CAST(xe.event_data AS XML) AS grafo
FROM sys.fn_xe_file_target_read_file
     (N'system_health*.xel', NULL, NULL, NULL) AS xe
WHERE xe.object_name = N'xml_deadlock_report';
```

- Bloqueo prolongado
- Transacción abierta sin COMMIT: el resto espera.
- Deadlock (error 1205)
- Ciclo de esperas; el Lock Monitor elige víctima.
- RCSI y KILL con cautela
- Los lectores no bloquean; KILL, el último recurso.

OBJETIVO PEDAGÓGICO

Distinguir bloqueo prolongado de deadlock, localizar al bloqueador raíz con DMVs, leer el grafo de deadlock de system_health y aplicar mitigaciones (RCSI, índices, orden de acceso) sin abusar de KILL.

GUION DEL DOCENTE Y EXPLICACIÓN TÉCNICA

- Bloqueo (blocking):** una sesión pide un lock incompatible con el que otra mantiene y espera (LCK_M_S, LCK_M_X, LCK_M_U...). Es normal y breve; es incidencia cuando se prolonga porque el bloqueador tiene una transacción abierta (sesión sleeping con open_transaction_count > 0).
- Diagnóstico:** sys.dm_exec_requests.blocking_session_id muestra quién bloquea a cada petición; el bloqueador raíz es la sesión que bloquea a otras y no está bloqueada (blocking_session_id = 0). sys.dm_tran_locks (request_status = WAIT / GRANT) detalla recurso y modo; sys.dm_os_waiting_tasks da la cadena de esperas; sys.dm_exec_sql_text(sql_handle) y most_recent_sql_handle en dm_exec_connections revelan la consulta.
- Deadlock:** dos o más sesiones se esperan en ciclo. El Lock Monitor (cada 5 s, más a menudo si hay deadlocks) elige una víctima (DEADLOCK_PRIORITY, luego menor coste de rollback) y devuelve el error 1205 al cliente. El grafo (xml_deadlock_report) está en la sesión Extended Events system_health (siempre activa, archivos .xel); también TF 1222 lo escribe en el errorlog.
- Mitigación:** transacciones cortas, acceder a los objetos siempre en el mismo orden, índices adecuados (menos filas bloqueadas, evita table scans bajo lock y la escalada a lock de tabla a las ~5000 locks), reintentos con TRY...CATCH ante el error 1205 y LOCK_TIMEOUT.
- Niveles de aislamiento:** READ COMMITTED (por defecto) bloquea lectores contra escritores. READ_COMMITTED_SNAPSHOT (RCSI) hace que los lectores lean la última versión confirmada del version store (en TempDB) sin bloquear: ALTER DATABASE Ventas SET READ_COMMITTED_SNAPSHOT ON WITH ROLLBACK IMMEDIATE. Coste: TempDB y 14 bytes por fila. SNAPSHOT añade lecturas consistentes por transacción y conflictos de actualización (error 3960).
- KILL <session_id>** termina la sesión y revierte su transacción: el rollback puede ser tan largo como lo ya hecho (KILL ... WITH STATUSONLY indica el avance) y mientras tanto sigue reteniendo locks. Nunca matar sesiones del sistema ni sin entender qué hace el proceso (backup, restore, rebuild). Es el último paso, tras intentar contactar al propietario.

PUNTOS DE INTERACCIÓN Y PREGUNTAS PARA LA CLASE

- ¿Cómo distingues desde las DMVs entre un bloqueo largo y un deadlock? ¿Qué ve el usuario en cada caso?
- ¿Qué gana y qué pierde Ventas al activar RCSI? ¿Quién paga el coste?
- Encuentras que el bloqueador raíz es una sesión «sleeping» con una transacción abierta de hace 3 horas. ¿Qué haces antes de ejecutar KILL?

DIAPOSITIVA 73

T-SQL

LABORATORIO · LABORATORIO 4 · PREPARACIÓN

4 · Mantenimiento y Buenas Prácticas

 CONTENIDO CLAVE EN PANTALLA (PROYECCIÓN)

- Preparar la BD y la cadena de backups

```
USE master;
ALTER DATABASE Ventas SET RECOVERY FULL;

BACKUP DATABASE Ventas
TO DISK = N'/var/opt/mssql/backup/Ventas_L4_FULL.bak'
WITH COMPRESSION, CHECKSUM, INIT;

USE Ventas;
INSERT dbo.Cliente (Nombre) VALUES (N'Lab4 A');
BACKUP LOG Ventas
TO DISK = N'/var/opt/mssql/backup/Ventas_L4_LOG1.trn'
WITH CHECKSUM, INIT;

INSERT dbo.Cliente (Nombre) VALUES (N'Lab4 B');
BACKUP DATABASE Ventas
TO DISK = N'/var/opt/mssql/backup/Ventas_L4_DIFF.bak'
WITH DIFFERENTIAL, CHECKSUM, INIT;
```

SQL

- 1

```
Full en recovery Full
ALTER DATABASE y BACKUP DATABASE con CHECKSUM.
```

SQL

- 2

```
Log backup 1
Tras insertar datos, inicia la log chain.
```

SQL

- 3
- Diferencial
- Tras más actividad: WITH DIFFERENTIAL.
- 4

```
Segundo log backup
Un cliente más y BACKUP LOG; anota el COUNT.
```

SQL

 OBJETIVO PEDAGÓGICO

Dejar la base Ventas lista para el ejercicio de desastre: recovery model Full y una log chain completa (Full, log, diferencial, log) con actividad entre backups.

 GUION DEL DOCENTE Y EXPLICACIÓN TÉCNICA

- Contexto:** el laboratorio simula el escenario del diagrama de la estrategia: un Full, actividad, backups intermedios y un desastre posterior. Todo se hace sobre Ventas (la base creada en los laboratorios anteriores) en el contenedor Docker de SQL Server.
- Importancia del orden:** sin Full previo, BACKUP LOG falla (error 4214 / «no hay backup actual de la base»); un diferencial exige un Full base. Por eso el primer paso es el Full, y por eso la log chain empieza aquí.
- WITH COMPRESSION, CHECKSUM, INIT:** INIT sobrescribe el fichero si se repite el ejercicio; CHECKSUM valida páginas. Las rutas son /var/opt/mssql/backup/ dentro del contenedor; hay que montarla como volumen (docker run -v ...) si se quiere ver desde el host.
- La actividad entre backups (INSERT en dbo.Cliente) sirve para que cada tipo de backup contenga datos distintos y para poder comprobar después hasta dónde se restauró.
- Anotar la hora (SYSDATETIME) justo antes del desastre es el dato más importante de la práctica:** será el STOPAT. Pedir al alumnado que lo copie en un bloc de notas.
- Tiempo:** ~20 min. Mientras corren los backups, aprovechar para repasar msdb.dbo.backupset y la columna type.

 PUNTOS DE INTERACCIÓN Y PREGUNTAS PARA LA CLASE

- ¿Qué ocurre si se lanza BACKUP LOG antes del primer Full? ¿Por qué?
- ¿Qué contiene el diferencial respecto del log backup anterior?

 INSTRUCCIONES DE LABORATORIO Y PAUTAS DE RESOLUCIÓN

- Paso 1 (SSMS o sqlcmd conectado al contenedor):** ejecutar USE master; ALTER DATABASE Ventas SET RECOVERY FULL; y comprobar con SELECT recovery_model_desc FROM sys.databases WHERE name = N'Ventas'.
- Paso 2:** tomar el Full inicial (Ventas_L4_FULL.bak). Verificar que aparece en msdb.dbo.backupset con type = D.
- Paso 3:** insertar un cliente y tomar el primer log backup (Ventas_L4_LOG1.trn); insertar otro y tomar el diferencial (Ventas_L4_DIFF.bak).

- **Paso 4:** insertar un tercer cliente y tomar un segundo log backup (Ventas_L4_LOG2.trn). Ejecutar `SELECT COUNT(*) FROM dbo.Pedido` y anotar el resultado (baseline).
- **Trampas habituales:** (a) el usuario mssql del contenedor no tiene permiso sobre la carpeta del volumen montado (error 3201/«Operating system error 5»); (b) la base estaba en SIMPLE y no se cambió (BACKUP LOG devuelve el error 4208); (c) la columna Nombre de dbo.Cliente tiene otras columnas NOT NULL: adaptar el INSERT; (d) rutas Windows (C:\...) en un contenedor Linux.
- **Guía de resolución:** `SELECT bs.type, bs.backup_finish_date, bmf.physical_device_name FROM msdb.dbo.backupset bs JOIN msdb.dbo.backupmediafamily bmf ON bmf.media_set_id = bs.media_set_id WHERE bs.database_name = N'Ventas' ORDER BY bs.backup_finish_date`; debe mostrar D, L, L, L en ese orden. Si falta alguno, repetir ese paso.

DIAPOSITIVA 74

T-SQL

LABORATORIO · LABORATORIO 4 · DESASTRE

4 · Mantenimiento y Buenas Prácticas

 CONTENIDO CLAVE EN PANTALLA (PROYECCIÓN)

- Simular la pérdida de datos y el tail-log
- 1

Anota HoraOK
SELECT SYSDATETIME() antes del accidente.

SQL

- 2
- Provoca el error
- DELETE sin WHERE sobre dbo.Pedido.
- 3
- Salva la cola
- Tail-log con NORECOVERY: la BD pasa a RESTORING.

```
USE Ventas;
SELECT COUNT(*) AS Pedidos FROM dbo.Pedido;
SELECT SYSDATETIME() AS HoraOK;      -- anótala
WAITFOR DELAY '00:00:05';

DELETE FROM dbo.Pedido;              -- "accidente"

USE master;
BACKUP LOG Ventas
TO DISK = N'/var/opt/mssql/backup/Ventas_L4_TAIL.trn'
WITH NORECOVERY, CHECKSUM, INIT;
```

SQL

Trampa: si hay conexiones abiertas a Ventas, usa **ALTER DATABASE ... SET SINGLE_USER WITH ROLLBACK IMMEDIATE** antes del tail-log.

SQL

 OBJETIVO PEDAGÓGICO

Provocar un error humano (DELETE sin WHERE), registrar la hora previa y salvar las transacciones con un tail-log backup antes de restaurar.

 GUION DEL DOCENTE Y EXPLICACIÓN TÉCNICA

- El error humano es la causa de pérdida de datos más frecuente en producción (más que el fallo hardware). Simulamos un DELETE FROM dbo.Pedido sin WHERE: las filas desaparecen de forma lógica pero el log las contiene.
- Capturar SYSDATETIME() justo antes, y esperar unos segundos (WAITFOR DELAY) antes del DELETE para que haya margen entre HoraOK y el accidente al fijar STOPAT. La hora es la del servidor: en Docker normalmente UTC.
- Tail-log backup con NORECOVERY: toma la cola del log (incluye el DELETE y todo lo anterior) y deja la base en RESTORING, bloqueando nuevas escrituras mientras se restaura. En producción es el paso que evita perder los últimos minutos.
- Explicar la lección: lo que se restaura es la base anterior al DELETE, pero los datos legítimos hechos entre el accidente y la detección se perderían; alternativa: restaurar con otro nombre (RESTORE ... AS Ventas_Recup) y copiar sólo las filas borradas con INSERT...SELECT.
- Verificar el estado tras el tail-log: SELECT name, state_desc FROM sys.databases WHERE name = N'Ventas'; debe mostrar RESTORING.

 PUNTOS DE INTERACCIÓN Y PREGUNTAS PARA LA CLASE

- ¿Por qué hay que hacer el tail-log ANTES de empezar a restaurar?
- ¿Qué alternativa tendrías si sólo quisieras recuperar las filas borradas sin revertir toda la base?

 INSTRUCCIONES DE LABORATORIO Y PAUTAS DE RESOLUCIÓN

- **Paso 1:** en una pestaña de SSMS ejecutar USE Ventas; SELECT COUNT(*) AS Pedidos FROM dbo.Pedido; y anotar el valor.
- **Paso 2:** ejecutar SELECT SYSDATETIME() AS HoraOK; copiar el valor (p. ej. 2025-03-12 10:36:05.1234567) y esperar 5 s con WAITFOR DELAY.
- **Paso 3:** ejecutar el «accidente»: DELETE FROM dbo.Pedido; y comprobar COUNT(*) = 0.
- **Paso 4:** desde master tomar el tail-log (Ventas_L4_TAIL.trn) WITH NORECOVERY, CHECKSUM. Si la sesión que borró sigue conectada a Ventas, cerrarla antes o añadir ALTER DATABASE ... SET SINGLE_USER WITH ROLLBACK IMMEDIATE.
- **Trampas habituales:** (a) dejar USE Ventas activo y no poder pasar a RESTORING (hay conexiones abiertas); (b) olvidar anotar la hora; (c) usar NORECOVERY en el tail-log y luego querer consultar la base (está en RESTORING, es lo esperado); (d) confundir hora local y UTC.
- **Guía de resolución:** si el tail-log falla por conexiones, ejecutar ALTER DATABASE Ventas SET SINGLE_USER WITH ROLLBACK IMMEDIATE y repetir. Si se olvidó la hora, buscarla con fn_dblog o, tras restaurar a otro nombre, con una consulta por la fecha de la última fila válida.

DIAPOSITIVA 75

T-SQL

LABORATORIO · LABORATORIO 4 · RESTAURACIÓN

4 · Mantenimiento y Buenas Prácticas

 CONTENIDO CLAVE EN PANTALLA (PROYECCIÓN)

- Restaurar a un punto en el tiempo

```
USE master;
RESTORE DATABASE Ventas
FROM DISK = N'/var/opt/mssql/backup/Ventas_L4_FULL.bak'
WITH NORECOVERY, REPLACE;

RESTORE DATABASE Ventas
FROM DISK = N'/var/opt/mssql/backup/Ventas_L4_DIFF.bak'
WITH NORECOVERY;

RESTORE LOG Ventas
FROM DISK = N'/var/opt/mssql/backup/Ventas_L4_LOG1.trn'
WITH NORECOVERY;

RESTORE LOG Ventas
FROM DISK = N'/var/opt/mssql/backup/Ventas_L4_TAIL.trn'
WITH STOPAT = N'2025-03-12T10:36:00', RECOVERY;

SELECT COUNT(*) AS Pedidos FROM Ventas.dbo.Pedido;
```

SQL

- Verifica el resultado

```
COUNT(*) igual al baseline
state_desc = ONLINE
DBCC CHECKDB sin errores
```

SQL

- Trampa: STOPAT va en HoraOK, antes del DELETE, y sin NORECOVERY en el último paso.

 OBJETIVO PEDAGÓGICO

Ejecutar la secuencia completa de restauración con STOPAT, recuperar Pedido y verificar el resultado comparándolo con el baseline.

 GUION DEL DOCENTE Y EXPLICACIÓN TÉCNICA

- Secuencia:** Full (NORECOVERY, REPLACE) → diferencial (NORECOVERY) → log 1 (NORECOVERY) → log 2 (NORECOVERY, si se tomó) → tail-log con STOPAT = HoraOK y RECOVERY. Cada paso debe devolver «RESTORE DATABASE successfully processed» antes del siguiente.
- El STOPAT ha de ir en el último log que cubra la hora y estar ANTES del DELETE. Si la hora indicada es anterior al inicio de ese log, el motor devuelve el error 4326 («too early to apply»); si es posterior al final, avisa de que el log termina antes (y la base queda recuperada al final del log).
- REPLACE es necesario porque la base existe (en RESTORING) y se sobrescribe. Si alguien omite NORECOVERY en el Full, el siguiente RESTORE falla («the database is online / el log no se puede restaurar»):** hay que volver a empezar.
- Verificación:** SELECT COUNT(*) FROM dbo.Pedido debe igualar el baseline anotado antes del DELETE. Después: DBCC CHECKDB (Ventas) WITH NO_INFOMSGS y una consulta a msdb.dbo.restorehistory para ver la secuencia.
- Cierre pedagógico:** discutir el RTO real medido (cuánto tardó) y el RPO (qué se perdió, en este caso nada gracias al tail-log); conectar con el caso integrador.

 PUNTOS DE INTERACCIÓN Y PREGUNTAS PARA LA CLASE

- ¿Qué diferencia habría si en vez de STOPAT usáramos sólo RECOVERY con el tail-log completo?
- Si el COUNT(*) es menor que el baseline, ¿qué hipótesis comprobarías primero?

 INSTRUCCIONES DE LABORATORIO Y PAUTAS DE RESOLUCIÓN

- Paso 1:** desde master, RESTORE DATABASE Ventas FROM DISK = Ventas_L4_FULL.bak WITH NORECOVERY, REPLACE.
- Paso 2:** RESTORE DATABASE Ventas FROM DISK = Ventas_L4_DIFF.bak WITH NORECOVERY.
- Paso 3:** RESTORE LOG con Ventas_L4_LOG1.trn (NORECOVERY); si se tomó el segundo log (LOG2), restaurarlo también en orden.
- Paso 4:** RESTORE LOG con Ventas_L4_TAIL.trn WITH STOPAT = N'', RECOVERY (usar el valor anotado; formato aaaa-mm-ddThh:mm:ss).
- Paso 5:** comprobar COUNT(*) FROM dbo.Pedido = baseline, state_desc = ONLINE y ejecutar DBCC CHECKDB.
- Trampas habituales:** (a) aplicar un log fuera de orden o saltarse uno: error 4305 («log too recent / LSN too early»); (b) STOPAT posterior al DELETE: la tabla sigue vacía; (c) restaurar el Full sin NORECOVERY: la base se abre y la cadena se corta; (d) nombres lógicos distintos al usar MOVE (revisar con RESTORE FILELISTONLY).
- Guía de resolución:** si se cometió el error (c), repetir desde el paso 1 con REPLACE. Si STOPAT queda tarde, repetir toda la secuencia con una hora anterior (restaurando de nuevo desde el Full). Comprobar los LSN con RESTORE HEADERONLY o en msdb.dbo.backupset (first_lsn / last_lsn).

DIAPOSITIVA 76

LABORATORIO PRÁCTICO

Laboratorio: Caso Práctico Integrador Final

4 · Mantenimiento y Buenas Prácticas

 CONTENIDO CLAVE EN PANTALLA (PROYECCIÓN)

- Caso práctico integrador final
- 15 min

RPO: log backup cada 15 minutos

SQL

- 60 min
- 3 roles
- RTO: restaurar y validar en una hora
- Mínimo privilegio: app, informes, DBA
- 1 / sem

DBCC CHECKDB semanal con alerta

SQL

- Requisitos de Ventas S.A.
- Mantenimiento: jobs, CHECKDB y alertas.
- Seguridad: logins y roles sin sysadmin.
- Evaluación y entregables
- Índices: 2 justificados con su plan.
- Peso: 25 · 25 · 30 · 20 %.
- Backups: Full, Diff y Log; prueba de restore.
- Script T-SQL comentado.
- Informe con RPO y RTO medidos.

 OBJETIVO PEDAGÓGICO

Integrar todo el curso en un único entregable: dejar una instancia lista para producción con seguridad, rendimiento, estrategia de backups y mantenimiento que cumpla un RPO y un RTO dados.

 GUION DEL DOCENTE Y EXPLICACIÓN TÉCNICA

- Escenario:** «Ventas S.A.» pone en producción la base Ventas (Cliente, Pedido, Producto) en una instancia nueva. Se pide un plan de operación defendible ante el responsable de sistemas, con un RPO $\leq$ 15 min y un RTO $\leq$ 60 min.
- Requisitos de seguridad (Módulo 2):** login SQL o de dominio para la aplicación con permisos mínimos (db_datareader/db_datawriter o permisos sobre schema), rol de sólo lectura para informes, ningún miembro de sysadmin salvo el DBA; auditoría básica de logins fallidos.
- Requisitos de rendimiento (Módulo 3):** al menos 2 índices justificados con plan de ejecución antes/después (por ejemplo, Pedido por Clienteld + Fecha con INCLUDE de Total) y una consulta analítica que pase de scan a seek; evidencias con SET STATISTICS IO.
- Requisitos de backups (Módulo 4):** recovery Full, Full semanal, Diferencial diario, Log cada 15 min con COMPRESSION y CHECKSUM, copia fuera del servidor (Azure Blob o ruta de red), restauración de prueba en otro nombre (Ventas_Test) con medición del tiempo real.
- Mantenimiento:** jobs de Agent para backups, CHECKDB semanal, mantenimiento de índices y estadísticas por umbrales, limpieza de msdb, alertas 823/824/825 y gravedad 19–25, Database Mail y operador DBA.
- Evaluación:** seguridad 25 %, rendimiento 25 %, backups y restauración 30 %, mantenimiento y alertas 20 %. Entregables: script T-SQL (idempotente y comentado), informe breve de 1–2 páginas con RPO/RTO medidos y capturas de la restauración.

 PUNTOS DE INTERACCIÓN Y PREGUNTAS PARA LA CLASE

- ¿Cómo demostrarías con datos que se cumple el RPO de 15 minutos? ¿Y el RTO de 60 minutos?
- ¿Qué riesgo queda sin cubrir con este diseño (p. ej. fallo del sitio completo) y cómo lo mitigarías con lo visto en HA/DR?
- ¿Qué pondrías en el runbook para que otra persona pueda restaurar a las 3 de la madrugada?

 INSTRUCCIONES DE LABORATORIO Y PAUTAS DE RESOLUCIÓN

- Dinámica:** trabajo individual o en parejas, ~60 min más 10 min de puesta en común. El docente recorre los grupos con la rúbrica.
- Paso 1 (10 min):** seguridad: CREATE LOGIN + CREATE USER + roles; probar con EXECUTE AS que la app no puede DROP ni leer fuera de su schema.
- Paso 2 (15 min):** índices: capturar el plan antes (Table Scan / Key Lookup), crear el índice (con INCLUDE), capturar después (Index Seek) y comparar lecturas lógicas.
- Paso 3 (20 min):** backups: recovery Full, Full + Diff + Log con CHECKSUM, simular un fallo y restaurar a otro nombre con MOVE, medir el RTO.
- Paso 4 (15 min):** mantenimiento: crear al menos un job (log backup cada 15 min), una alerta (error 823 o gravedad 24) y el operador DBA.
- Trampas habituales:** backups en el mismo volumen que los datos; no probar la restauración; dar sysadmin a la aplicación «para ir más rápido»; índices duplicados; olvidar el INCLUDE y mantener el Key Lookup; no iniciar la log chain (falta del primer Full).
- Guía de resolución:** seguridad correcta = login → user → rol/permisos sin sysadmin; rendimiento = lecturas lógicas menores y plan con Seek; backups = cadena msdb completa (D, I, L) y tiempo de restore medido < 60 min; mantenimiento = jobs con historial en msdb y alertas visibles en sysjobs / sysalerts.

DIAPOSITIVA 77

RESUMEN

Checklist de buenas prácticas del DBA

4 · Mantenimiento y Buenas Prácticas

 CONTENIDO CLAVE EN PANTALLA (PROYECCIÓN)

Backups probados

Full + Diff + Log con CHECKSUM, copia externa y restore mensual.

SQL

- Recovery Full en producción
- Log backups cada 15 min según el RPO.
- Integridad vigilada
- PAGE_VERIFY CHECKSUM y CHECKDB semanal.
- Mantenimiento por umbrales
- Índices y estadísticas con scripts, no a ciegas.
- Capacidad controlada
- Autogrowth fijo, tempdb igual y espacio monitorizado.
- Alertas operativas
- Errores 823/824/825 y gravedad 19–25 por correo.
- Mínimo privilegio
- Roles ajustados y auditoría de sysadmin.
- Runbook y DR
- Procedimientos documentados y RPO/RTO medidos.

 OBJETIVO PEDAGÓGICO

Consolidar en una lista accionable los hábitos que protegen la instancia y que el alumnado puede aplicar desde el primer día de trabajo.

 GUION DEL DOCENTE Y EXPLICACIÓN TÉCNICA

- **Backups:** el estándar es recovery Full con Full + Diff + Log según RPO/RTO, siempre con CHECKSUM y compresión, guardados fuera del servidor (3-2-1) y con una restauración de prueba mensual. «Un backup no probado es una esperanza».
- **Integridad:** PAGE_VERIFY CHECKSUM en todas las bases, DBCC CHECKDB al menos semanal (idealmente sobre una copia restaurada) y alertas 823, 824, 825 y gravedad 19–25 enviadas por Database Mail.
- **Mantenimiento:** reorganizar/reconstruir por umbrales (5 %/30 %, > 1000 páginas), estadísticas actualizadas, limpieza de msdb y revisión del historial de jobs. Evitar SHRINK y rebuild ciegos.
- **Capacidad y rendimiento:** autogrowth en MB fijos y tamaños iniciales realistas, ficheros de tempdb iguales, monitorización de espacio, log_reuse_wait_desc y esperas con DMVs. Baseline para detectar desviaciones.
- **Seguridad:** mínimo privilegio, sin cuentas compartidas, sa deshabilitada o con contraseña robusta, auditoría de sysadmin y parches (CU) al día con ventana de mantenimiento.
- **Operación:** runbooks documentados, contactos de escalado, pruebas de DR periódicas (RPO/RTO medidos) y control de cambios. La automatización reduce errores humanos, la principal causa de incidentes.

 PUNTOS DE INTERACCIÓN Y PREGUNTAS PARA LA CLASE

- ¿Qué elemento del checklist es el más barato de implantar y el más caro de olvidar?
- ¿Cuáles de estos hábitos ya se aplican en vuestra organización y cuáles no? ¿Qué impide hacerlo?

DIAPOSITIVA 78

CIERRE DEL CURSO

Gracias: ahora piensas como un DBA

4 · Mantenimiento y Buenas Prácticas

 CONTENIDO CLAVE EN PANTALLA (PROYECCIÓN)

- Entiende el motor
- Buffer Pool, WAL y VLFs explican rendimiento y recuperación.
- Protege los datos
- Mínimo privilegio y backups con CHECKSUM, probados.
- Optimiza con datos
- Índices, estadísticas y DMVs antes de tocar nada.
- Automatiza y vigila
- Jobs, alertas y runbooks: menos errores humanos.

 OBJETIVO PEDAGÓGICO

Cerrar el curso resumiendo las ideas que el alumnado debe recordar y animarle a seguir practicando con un entorno propio.

 GUION DEL DOCENTE Y EXPLICACIÓN TÉCNICA

- **Resumen en cuatro ideas. 1) Entender el motor:** Relational Engine, Storage Engine, Buffer Pool y el Transaction Log (WAL) explican por qué SQL Server rinde y se recupera como lo hace; casi todo problema se resuelve mejor sabiendo qué hay debajo.
- **2) Proteger los datos:** mínimo privilegio, estrategia de backups alineada con RPO/RTO, CHECKSUM, tail-log, restauraciones de prueba y DBCC CHECKDB. La disciplina vale más que las herramientas.
- **3) Optimizar con evidencia:** medir primero (DMVs, planes de ejecución, esperas) y actuar después (índices adecuados, estadísticas actualizadas), evitando «recetas» como rebuild nocturno o shrink.
- **4) Automatizar y vigilar:** jobs del Agent, alertas, Database Mail y runbooks. Un DBA eficaz es el que consigue que lo previsible no requiera intervención y que lo imprevisto se detecte pronto.
- **Próximos pasos recomendados:** montar un laboratorio personal en Docker; practicar Always On Availability Groups, TDE y Always Encrypted; explorar Query Store y Extended Events; preparar las certificaciones DP-300 (Azure Database Administrator) o similares.
- Cerrar con la evaluación del caso integrador, la encuesta del curso y un espacio para dudas. Agradecer la participación y compartir los scripts y la guía del docente.

 PUNTOS DE INTERACCIÓN Y PREGUNTAS PARA LA CLASE

- ¿Qué es lo primero que cambiarías mañana en tu instancia de producción tras este curso?
- ¿Qué parte te gustaría profundizar: alta disponibilidad, tuning de consultas o seguridad avanzada?